

AlphaZero-style MCTS for Yoh

The network, the search, the self-play learning loop, and the code that runs them

2026-09-26

Table of contents

1	What this document covers	3
2	The game, as far as the search is concerned	4
3	Position and move encoding	4
4	The network	6
4.1	Architecture	6
4.2	Training	7
4.3	Export: folding BatchNorm	8
4.4	The CPU backend	9
4.5	The libtorch backend	9
4.6	The seam: <code>nn::Evaluator</code>	9
4.7	The network cache	10
5	Monte-Carlo tree search with a network	10
5.1	One simulation	10
5.2	PUCT, in KataGo's formulation	11
5.2.1	The exploration coefficient	11
5.2.2	The utility-stdev factor $\hat{o}$	11
5.3	First play urgency	12
5.4	Backup and the sign convention	13
5.5	Virtual loss, batching and collisions	13
5.6	Graph search	14
5.6.1	The node key	15
5.6.2	Tree reuse and garbage collection	15
5.7	At the root	15
5.7.1	Dirichlet noise	16
5.7.2	Forced playouts	16
5.7.3	Policy target pruning	16
5.7.4	LCB move selection	17
5.8	Playout cap randomization	17
5.9	Choosing the move, and the policy target	17
6	Learning by self-play	18
6.1	The idea	18
6.2	The pieces and how they talk	19

6.3	Self-play: many games, one GPU	19
6.3.1	Why many games at once	19
6.3.2	The self-play settings, and why	21
6.3.3	What a training row contains	21
6.3.4	Hot-swapping the network	22
6.4	The trainer	22
6.4.1	The replay window	22
6.4.2	How much to train: the reuse ratio	23
6.4.3	The loss	24
6.4.4	Optimiser and numerics	24
6.4.5	Data augmentation: the 8 symmetries	25
6.4.6	Averaging the weights before export	26
6.4.7	Export and publication	26
6.5	Measuring progress	26
6.5.1	The match protocol	26
6.5.2	From a score to Elo	27
6.6	Running it on the cluster	28
6.6.1	The cluster	28
6.6.2	The image	28
6.6.3	The job script	29
6.6.4	Splitting the GPUs over several nodes	29
6.6.5	Stopping cleanly	30
6.6.6	What to watch	30
6.7	Sizing for the cluster	31
6.7.1	What was measured	31
6.7.2	From the measurements to the settings	31
6.8	Parameters of the learning loop	32
6.9	KataGo’s auxiliary heads	34
6.9.1	Why predicting more helps	34
6.9.2	Ownership	35
6.9.3	Score margin	35
6.9.4	Opponent’s reply	36
6.9.5	Short-term value targets	36
6.9.6	How it would fit into this code	36
7	Code map	37
8	Parameter reference	38
8.1	Budget and parallelism	38
8.2	PUCT	38
8.3	First play urgency	39
8.4	Graph and root	39
8.5	Self-play	39
9	Measurements	40
9.1	What graph search actually buys	40
9.2	Node allocation	41
9.3	What LCB and pruning change	41
10	References	41

11 Annotated source	42
11.1 The search	42
11.1.1 <code>player/alphazero_mcts.h</code>	42
11.1.2 <code>player/alphazero_mcts.cpp</code>	48
11.1.3 <code>player/alphazero_mcts_player.h</code>	63
11.1.4 <code>player/alphazero_mcts_player.cpp</code>	65
11.1.5 <code>player/nn_cache.h</code>	68
11.2 The network interface and its backends	70
11.2.1 <code>nnue/nn_evaluator.h</code>	70
11.2.2 <code>nnue/nn_evaluator.cpp</code>	73
11.2.3 <code>nnue/batched_evaluator.h</code>	73
11.2.4 <code>nnue/batched_evaluator.cpp</code>	75
11.2.5 <code>nnue/resnet.h</code>	77
11.2.6 <code>nnue/resnet.cpp</code>	79
11.2.7 <code>nnue/resnet_torch.h</code>	85
11.2.8 <code>nnue/resnet_torch.cpp</code>	86
11.3 Training and export	87
11.3.1 <code>nnue/cnn_resnet_value_policy_chunked.py</code>	87
11.3.2 <code>nnue/resnet_export.py</code>	102
11.4 The self-checks	106
11.4.1 <code>test/resnet_check.cpp</code>	106
11.4.2 <code>test/alphazero_mcts_check.cpp</code>	108
11.5 The learning loop	111
11.5.1 <code>player/alphazero_selfplay.h</code>	111
11.5.2 <code>player/alphazero_selfplay.cpp</code>	113
11.5.3 <code>test/alphazero_learn_main.cpp</code>	118
11.5.4 <code>nnue/alphazero_learn.py</code>	122
11.5.5 <code>config/alphazero_mcts_selfplay_player.cfg</code>	136
11.5.6 <code>config/alphazero_mcts_eval_player.cfg</code>	136
11.5.7 <code>nnue/alphazero_learn.def</code>	137
11.5.8 <code>nnue/alphazero_common.sh</code>	138
11.5.9 <code>nnue/alphazero_learn.sh</code>	140
11.5.10 <code>nnue/alphazero_selfplay.sh</code>	142

1 What this document covers

The Yolah engine has two halves that only meet through a very thin interface. On one side a two-headed residual network, trained in PyTorch, that looks at a position and answers two questions: *who is winning?* and *what would a strong player consider here?* On the other side a Monte-Carlo tree search that calls that network a few hundred times per move and turns its two opinions into an actual move — one much stronger than either opinion alone.

This document explains both halves and the seam between them: the algorithm and its formulas first, then the code that implements them, file by file. The search is the one in `player/alphazero_mcts.{h,cpp}`; it started as plain AlphaZero and has since been moved onto KataGo’s formulation of PUCT, its first-play-urgency rule, LCB move selection, graph search, and the two self-play tricks from the KataGo paper (forced playouts and policy target pruning). Section 6 then closes the loop: the engine plays against itself on a GPU cluster, learns from those games and gets stronger, for weeks, with its progress measured by regular matches against its former selves.

Conventions used throughout:

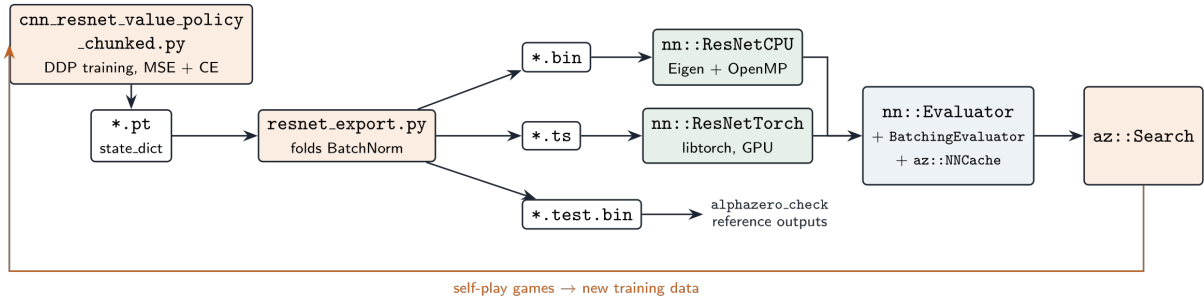


Figure 1: From the training script to the search: every artefact and who consumes it.

- s is a position, a a legal move, $s \cdot a$ the position after it.
- $v \in (-1, 1)$ is the network's value, always **for the player to move**.
- $z \in \{-1, 0, +1\}$ is the final game result, also for the player to move.
- $P(s, a)$ is the network's prior, $Q(s, a)$ the search's running estimate, N a visit count, W a sum of values.

2 The game, as far as the search is concerned

Yolah is played on 8×8 with four stones per side. A stone slides like a queen, and **the square it leaves becomes a permanent hole**. The player who moves scores one point per move; the game ends when neither side can move, and the higher score wins.

Two consequences drive the design of the search.

The state graph is acyclic. In `Yolah::play` the move does `empty ^= square_bb(m.from_sq())`, and `from_sq` always holds a stone, never a hole — so the hole count increases by exactly one on every real move and can never decrease. A position can therefore never repeat. No repetition detection, no cycle handling, no draw-by-repetition: the search space is a DAG, and a depth-first walk over it always terminates.

Move orders commute. The four stones of a side move independently, so playing a then b with two different stones reaches exactly the same position as b then a . The same position is reachable through many paths, which is what makes both the network cache and the graph search worth having.

The one wrinkle: a player with no legal move passes (`Move::none()`) and scores nothing, while the opponent keeps moving. Two positions can therefore share a board and a side to move while differing in how the points so far split between the players — and the final result depends on that split. This matters for hashing, and Section 5.6.1 comes back to it.

3 Position and move encoding

The network sees four 8×8 binary planes: black stones, white stones, holes, and a constant plane for the side to move.

Two details are easy to get wrong and are worth stating explicitly.

The planes come out mirrored, and not on purpose. Squares are numbered `SQ_A1 = 0` through `SQ_H8 = 63`, and bit s of a bitboard is square s . The training cache turns a bitboard into a plane like this (`preprocess.py`):

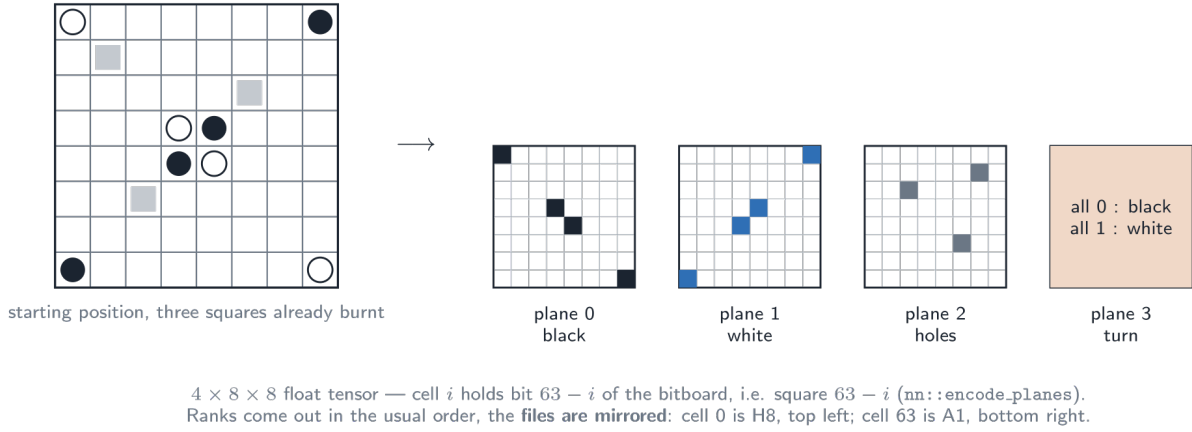


Figure 2: `nn::encode_planes` (C++) and `encode_cnn` (Python) must agree bit for bit.

```
np.unpackbits(np.array([n], dtype='>u8').view(np.uint8)).reshape(8, 8)
```

`>u8` writes the integer big-endian — most significant byte first — and `np.unpackbits` takes the most significant bit of each byte first. The 64 values therefore come out in the order bit 63, bit 62, ..., bit 0, so element i is bit $63 - i$, which is square $63 - i$. That is the whole origin of the $63 - i$, and `nn::encode_planes` has no choice but to repeat it:

```
plane[i] = static_cast<float>((bb >> (63 - i)) & 1);
```

Geometrically: write $s = 8\text{rank} + \text{file}$ and reshape row-major, and you get $\text{row} = 7 - \text{rank}$, $\text{col} = 7 - \text{file}$. The ranks land the usual way round, rank 8 on the top row — but **the files are mirrored**, H on the left and A on the right. Cell 0 is H8 and cell 63 is A1.

Nothing about the network minds. A fixed mirror of every input is just a relabelling of the board, and the convolutions learn whichever labelling they are shown. What matters is that training and inference agree *exactly*: an encoder that helpfully “fixes” the order would feed the trained weights a mirrored position, and the failure would be quiet — a network that has simply become much weaker.

There is one place it could have bitten, and happens not to. The policy head indexes actions by the **true** square numbers, 64 from + to, not the mirrored ones — so the input geometry and the output indexing disagree. It costs nothing because both heads flatten the trunk and pass through a fully-connected layer, which absorbs any fixed permutation of its input. A convolutional policy head emitting one logit per board square, as some AlphaZero implementations use, would have had to be told about the mirror explicitly.

The score is not an input. The planes carry no score at all. The sum of the two scores is recoverable — it is the number of holes — but the split is not. The network is expected to learn what it needs from the piece and hole configuration.

Moves use a flat AlphaZero-style action space of $64 \times 64 = 4096$ logits:

$$\text{action}(m) = 64 \cdot \text{from}(m) + \text{to}(m)$$

Roughly three quarters of that space is never legal. Illegal actions are not masked during training — they are simply never the target, so the softmax starves them — and the search applies the legal

mask itself: the evaluator returns the logits of *exactly the legal moves*, in `Yolah::moves` order, and the search softmaxes over those. A pass maps to A1→A1, index 0, which is never a real move.

4 The network

4.1 Architecture

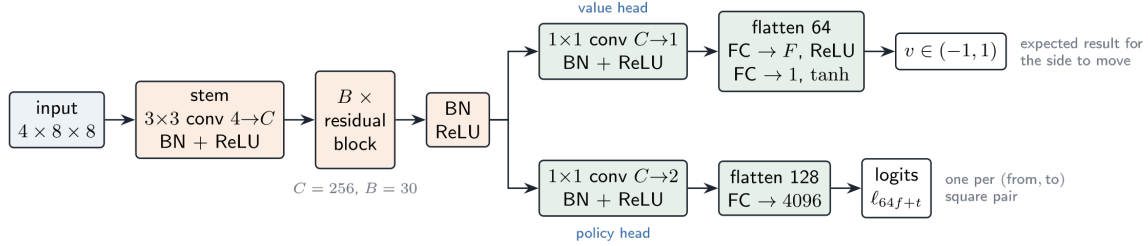


Figure 3: Net in `nnue/cnn_resnet_value_policy_chunked.py`. The shipped model is $C = 256$, $B = 30$, $F = 256$: about 35 M parameters and ≈ 4.5 GFLOP per position.

The trunk is a stem convolution followed by B pre-activation residual blocks and a final BatchNorm + ReLU. Both heads read the same $(C, 8, 8)$ feature map.

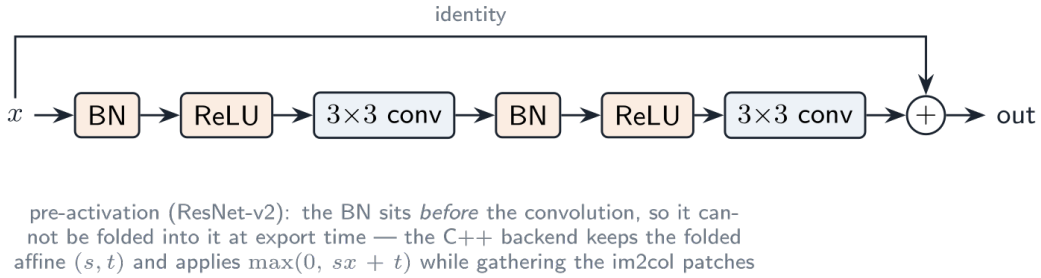


Figure 4: The residual block is ResNet-v2 style: BN and ReLU come *before* each convolution.

```
class ResBlock(nn.Module):
    def forward(self, x):
        residual = x
        x = self.conv1(torch.relu(self.bn1(x)))
        x = self.conv2(torch.relu(self.bn2(x)))
        return x + residual
```

The **value head** squeezes C channels to one with a 1×1 convolution, flattens the resulting 8×8 map, and runs two fully-connected layers into a tanh. The output is the expected result for the side to move; a winning probability is $(v + 1)/2$.

The **policy head** squeezes to two channels, flattens to 128 features, and projects to 4096 logits with a single fully-connected layer. That last layer holds $4096 \times 128 \approx 524$ k weights and is, per position, cheaper than one trunk block.

4.2 Training

Two scripts define the same network and differ only in how they feed it. `cnn_resnet_value_policy_chunked.py` is the one the shipped weights come from and the one `resnet_export.py` imports; `cnn_resnet_value_policy.py` is its sibling with a stock `DataLoader`, which is the right choice when the position cache fits in page cache. Everything in this section applies to both unless stated otherwise.

Both train the two heads at once on a supervised corpus of positions:

$$\mathcal{L} = \lambda_v (v - z)^2 + \lambda_p \text{CE}(\ell, a^*)$$

with $\lambda_v = \lambda_p = 1$, as in AlphaZero. The value target z is the game result from the point of view of the player to move at s ; the policy target a^* is the single move actually played, so the policy loss is a plain cross-entropy over the 4096 logits. Terminal positions have no next move and carry `POLICY_IGNORE_INDEX = -1`, which `nn.CrossEntropyLoss(ignore_index=...)` skips.

Note the asymmetry with a true AlphaZero loop: here the policy target is a *one-hot move*, not a visit distribution. When the self-play loop of Section 6 is running, the target becomes the search's π and the loss term becomes $-\sum_a \pi(a) \log p(a)$ — the same cross-entropy, with a soft target.

The training scripts are built around one constraint: the dataset does not fit in memory. Positions live in three `np.memmap` files (`positions` as `uint8`, `values` as `int8`, `policies` as `int16`) written by `preprocess.py`, roughly 253 GB in total, and most of what is unusual in the code exists only to make that workable.

Why the chunked loader. Sampling that file uniformly at random — which is what a stock `DataLoader` does — becomes millions of scattered ~256-byte reads. On a spinning disk that is seek-bound, and the only rescue is pre-loading the whole 253 GB into page cache. `ChunkedShuffleLoader` instead reads ~1 GB **contiguous** chunks sequentially, shuffles *within* a chunk and reshuffles the chunk order every epoch. That is an approximate shuffle, but a chunk holds around 4 M positions, which is more decorrelation than a 2048-position batch can use. One background thread per rank does the reading and the batch construction while the main thread drives the GPU; the overlap is real because numpy's `memcpy` and CUDA's pin-memory both release the GIL, and the bounded queue between them is the ping-pong buffer.

The DDP trap. Every rank must yield *exactly* the same number of batches. Each backward pass is a collective all-reduce, so a rank that runs one step fewer leaves the others hanging forever. Hence the floor division when the chunks are sharded across ranks — up to `world_size - 1` chunks are dropped — and the dropped trailing partial batch of each chunk.

Two more decisions that are easy to get wrong:

- The train/val split is **contiguous**, not random: positions are stored game by game, so a contiguous cut keeps whole games on one side of the split. A per-position random split would leak the same game into both halves.
- Mixed precision is selected on the compute capability, not on `torch.cuda.is_bf16_supported()` — that returns `True` on Turing, where bf16 is emulated and *slower* than fp32. Ampere and later get bf16 with the gradient scaler disabled; Turing gets fp16 with a live `GradScaler`.

The sibling script solves the same problem differently, and its two tricks are worth knowing if you ever go back to a `DataLoader`: `GameDataset.__getstate__` serialises only (`cache_dir`, `start`, `end`), because a raw `np.memmap` pickles *by value* and a spawned worker would copy the whole backing file; and `LeanDistributedSampler` replaces the stock `DistributedSampler`, whose

`torch.randperm(N).tolist()` turns a 4 GB index tensor into a ~30 GB list of Python ints **per rank**.

4.3 Export: folding BatchNorm

`resnet_export.py` turns one `.pt` checkpoint into three files. The interesting part is what it does to the BatchNorms.

In eval mode a BatchNorm is a per-channel affine map:

$$y = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta = s x + t, \quad s = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}}, \quad t = \beta - \mu s$$

Where a BN **follows** a convolution — the stem, and the two 1×1 head convolutions — the scale folds straight into the convolution weights and the shift becomes the bias the convolution did not have:

$$W' = W \odot s, \quad b' = t$$

Where a BN **precedes** a ReLU and a convolution — every pre-activation block, and the trunk's output BN — it cannot be merged into anything, so the export keeps (s, t) and the C++ backend applies $\max(0, sx + t)$ as it gathers the im2col patches, without an extra pass over the activations.

The three artefacts:

file	consumer	contents
<code>*.bin</code>	<code>nn::ResNetCPU</code>	flat little-endian float32, BN folded, convolutions laid out (out, tap, in) with tap $= 3dr + dc$
<code>*.ts</code>	<code>nn::ResNetTorch</code>	TorchScript module, traced in eval mode on a dummy $(1, 4, 8, 8)$ batch
<code>*.test.bin</code>	<code>alphazero_check</code>	a few positions with PyTorch's exact fp32 value and 4096 logits

The **(out, tap, in)** layout is not cosmetic — it is exactly the operand order the CPU backend's GEMM wants (Section 4.4).

The header is "YOLAHRSN", a version, and then (in, C, B, F, A) , so the C++ side never has to be told the architecture; it reads it.

4.4 The CPU backend

`nnue/resnet.cpp` is a from-scratch inference of the same network with Eigen and OpenMP, with no PyTorch at run time. It exists so the player always works, and as the reference the GPU backend is checked against.

Activations are kept **pixel-major** — `[position][pixel 0..63][channel]`, i.e. NHWC — which turns each 3×3 convolution into an im2col gather followed by one large GEMM:

$$\begin{matrix} \underline{\text{Out}} & = & \underline{\text{Col}} & \cdot & \underline{W^T} \\ 64n \times C & & 64n \times 9C & & 9C \times C \end{matrix}$$

With $C = 256$ that is a 512×2304 by 2304×256 product per convolution — a shape Eigen’s GEMM runs close to peak on with `-march=native -mfma`. The im2col copy itself is under 5 % of the layer cost, and the pre-activation BN+ReLU is applied *during* the gather, so the activations are read once.

Batching is done by splitting the batch into chunks of `MAX_CHUNK = 8` positions and giving the chunks to an OpenMP team, each thread with its own scratch buffers and Eigen running single-threaded inside. The backend is therefore happiest with **one** search thread handing it 32–128 positions at a time; several search threads must go through `nn::BatchingEvaluator` so their requests are merged rather than nested.

Denormals

The CMake LTO link line does not pass `-ffast-math`, so FTZ/DAZ are not set and the trunk’s denormal activations make inference roughly 30× slower. `nnue/resnet.cpp` sets FTZ/DAZ explicitly, per thread. Any new thread that calls into the network must do the same.

4.5 The libtorch backend

`nnue/resnet_torch.cpp` loads the TorchScript module and runs the whole batch as one `(n,4,8,8)` forward pass. It is compiled only with `-DENABLE_TORCH=ON` and a libtorch install (the one inside a pip torch package works: `-DCMAKE_PREFIX_PATH=<site-packages>/torch`). In half precision on an RTX-class GPU it is 20–50× the CPU backend at batch 256, which is why the search’s `preferred_batch_size()` is backend-specific.

A mutex serialises the forwards: callers are batched upstream anyway, and libtorch does not love concurrent `forward` on the same module.

4.6 The seam: `nn::Evaluator`

```
struct Request { const Yoloh* state; const Move* moves; uint16_t nb_moves; };
struct Result  { float value; float logits[Yoloh::MAX_NB_MOVES]; };

class Evaluator {
    virtual void evaluate(std::span<const Request>, std::span<Result>) = 0;
    virtual size_t preferred_batch_size() const = 0;
    virtual void reload(const std::string& weights_filename) = 0;
};
```

The search never touches a tensor. It hands over positions with their legal moves and gets back a value and the logits of *those* moves, in that order. The softmax, the temperature, and everything else are the search’s business. Three implementations sit behind the interface:

- **ResNetCPU** and **ResNetTorch**, the two backends above.
- **BatchingEvaluator**, a wrapper. Each search thread enqueues its span and blocks; a worker thread fires as soon as the queue reaches `max_batch`, or every registered client is waiting, or `max_wait` has elapsed. With T search threads submitting B leaves each, the backend sees up to $T \cdot B$ positions per call. An *empty* request is a barrier join — a thread that has nothing to evaluate but wants to wait for the next batch, which is what a thread whose descents all collided does instead of spinning.

4.7 The network cache

`player/nn_cache.h` is a direct-mapped, lock-stripped table of network outputs keyed by the Zobrist hash of (`board`, `side to move`) — exactly the network’s input, so the cached value is a pure function of the key. It stores the value and the priors of the legal moves as 16-bit fixed point, and verifies the number of legal moves on lookup so a 64-bit collision cannot silently hand back someone else’s priors.

It survives across moves and games (it depends only on the weights, so `reload_weights` clears it) and it is the second line of defence behind the node table of Section 5.6: it also catches positions that have fallen out of the search graph entirely.

5 Monte-Carlo tree search with a network

5.1 One simulation

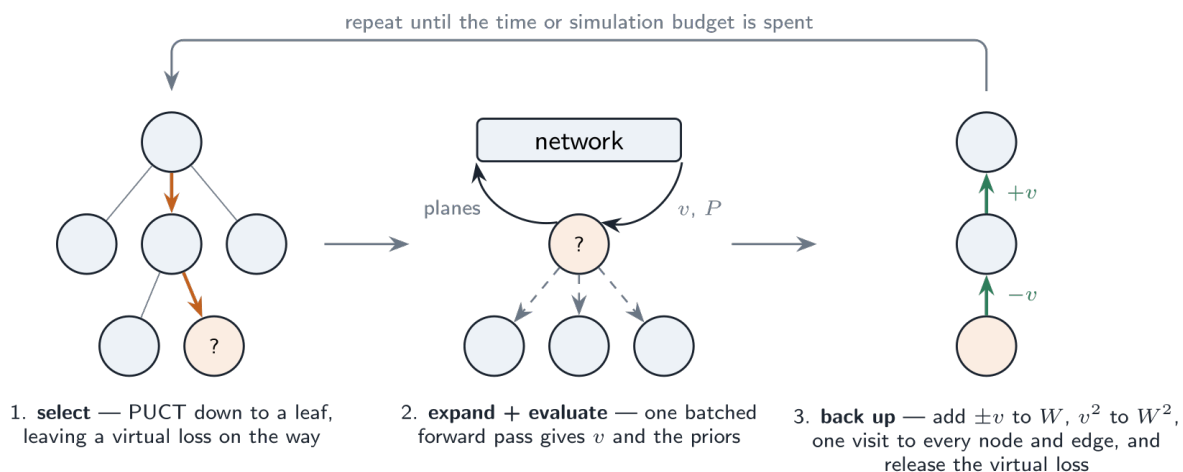


Figure 5: One simulation, in `Search::run_worker`. Steps 1 and 3 are per descent; step 2 is per batch.

A simulation walks down from the root picking children with the PUCT rule until it reaches a position the search has not expanded yet, asks the network what it is worth, and pushes that answer back up the path it came from. Classical MCTS would play a random game from the leaf instead; the network replaces both the rollout (the value head) and the move ordering heuristic (the policy head).

The four node states are **UNEXPANDED**, **EXPANDING** (a thread has claimed it and the network call is in flight), **EXPANDED**, and **TERMINAL**. The transitions are a **compare_exchange** on a single atomic byte, and **EXPANDED** is published with release semantics so that a thread observing it is guaranteed to see the fully built edge vector.

5.2 PUCT, in KataGo’s formulation

The child to descend into is

$$a^* = \arg \max_a \left[Q(s, a) + c(s) \hat{\sigma}(s) P(s, a) \frac{\sqrt{W(s) + \varepsilon}}{1 + W(s, a)} \right]$$

with $\varepsilon = 0.01$ (KataGo’s `TOTALCHILDWEIGHT_PUCT_OFFSET`, which keeps the exploration term positive when nothing has been visited yet) and

$$W(s) = \sum_a W(s, a)$$

the total weight of the **edges leaving** s — note: not the node’s own visit count, which is one larger. Every visit weighs 1 and every in-flight virtual loss also weighs 1.

The first term exploits, the second explores. $P(s, a)$ makes the exploration term follow the network’s opinion, $\sqrt{W(s)}$ makes it grow as the parent is studied, and $1/(1 + W(s, a))$ makes it collapse for a move already tried many times. In the limit, visit counts follow the policy tempered by the values.

5.2.1 The exploration coefficient

$$c(s) = c_{\text{puct}} + c_{\text{log}} \log \left(\frac{W(s) + c_{\text{base}}}{c_{\text{base}}} \right)$$

Defaults are KataGo’s `basicDecentParams`: $c_{\text{puct}} = 1.0$, $c_{\text{log}} = 0.45$, $c_{\text{base}} = 500$. Setting $c_{\text{log}} = 0$ recovers a constant c ; AlphaZero’s published variant is $c_{\text{log}} = 1$ with $c_{\text{base}} = 19652$, which with any realistic budget is numerically a constant — at $W = 10^4$ the log term contributes about 0.4 over the whole search. With base 500 it is live from the first few hundred visits, which is the point: a node with a lot of visits should widen, because the top move’s lead is by then well established and the remaining doubt is elsewhere.

5.2.2 The utility-stdev factor $\hat{\sigma}$

This is the piece that has no AlphaZero analogue. The exploration coefficient is scaled by how much the values coming back through this node *disagree*:

$$\hat{\sigma}(s) = 1 + \lambda \left(\frac{\sigma(s)}{\sigma_0} - 1 \right)$$

with $\lambda = 0.85$ (`cpuct_utility_stdev_scale`) and $\sigma_0 = 0.40$ (`cpuct_utility_stdev_prior`). $\sigma(s)$ is the standard deviation of the values backed up through s , regularised towards σ_0 with a weight of $w_p = 2$ visits:

$$\sigma(s)^2 = \frac{(\bar{u}^2 + \sigma_0^2) w_p + \bar{u}^2 N(s)}{w_p + N(s) - 1} - \bar{u}^2, \quad \bar{u} = \frac{W(s)}{N(s)}, \quad \bar{u}^2 = \frac{W^2(s)}{N(s)}$$

Every node therefore carries a second accumulator $W^2 = \sum v^2$ alongside $W = \sum v$. The negamax sign flip disappears in the square, so the same v^2 is added at every ply of a backup — one extra `fetch_add`, no extra bookkeeping.

The effect is sharp and worth internalising. A node whose children all evaluate around $+0.99$ has $\sigma \approx 0.02$, so $\hat{\sigma} \approx 0.2$ and exploration is cut fivefold: the position is decided, stop spending on it. A node with one child at $+0.6$ and another at -0.4 has a large σ and gets *more* exploration than plain PUCT would give it.

i σ_0 is not calibrated for this game

KataGo's $\sigma_0 = 0.40$ is tuned for its utility scale, which is win/loss (± 1) **plus** score utility terms, for a radius of about 1.4. Yolah's values are pure win/loss in $[-1, 1]$, so observed stdevs run smaller and $\hat{\sigma}$ sits below 1 more often than KataGo intends. Sweeping σ_0 downwards (0.25–0.30) before drawing conclusions from A/B games is the obvious first experiment.

5.3 First play urgency

$Q(s, a)$ is undefined for a move that has never been tried. Assuming 0 is a disaster in a game where values are mostly far from 0: in a won position every unvisited move looks like a draw, i.e. terrible, and the search never widens; in a lost position every unvisited move looks like an improvement, and the search sprays.

The FPU value answers with the parent's own value, minus a penalty that grows as more of the policy mass has already been tried:

$$Q_{\text{fpu}}(s) = U(s) - r \sqrt{m(s)}, \quad m(s) = \sum_{a: W(s,a) > 0} P(s, a)$$

$m(s)$ is the **visited policy mass**. When nothing has been tried, the penalty is zero and an unvisited move is worth exactly what the parent is worth. As the search works through the plausible moves, the bar for trying yet another one rises.

The baseline $U(s)$ is a blend of what the search has learned about s and what the network thought of it:

$$U(s) = \alpha \bar{q}(s) + (1 - \alpha) v_{\text{net}}(s), \quad \alpha = \min(1, m(s)^p)$$

so a barely-explored node trusts the network and a well-explored one trusts its own statistics. Finally `fpu_loss_prop` can drag the estimate a proportion of the way towards a loss (-1); it defaults to 0.

The root uses `root_fpu_reduction = 0.1` rather than `0.2`: at the root, unlike everywhere else, being wrong about a move is expensive and there are visits to spare.

5.4 Backup and the sign convention

The one thing to keep straight in a negamax MCTS. A node's W accumulates values **from the point of view of the player who moved into it** — which is precisely the quantity the *parent* needs, because $Q(s, a) = W(s \cdot a) / N(s \cdot a)$ is read by the player to move at s . The network's v is for the player to move at the leaf, so the leaf itself receives $-v$, and the sign flips once per ply going up:

```
float v = -leaf_value;
const float v2 = leaf_value * leaf_value; // same at every ply
for (size_t i = length; i-- > 0;) {
    path[i].node->w.fetch_add(v, relaxed);
    path[i].node->w2.fetch_add(v2, relaxed);
    path[i].node->n.fetch_add(1, relaxed);
    path[i].node->pending.fetch_sub(1, relaxed);
    if (Edge* e = path[i].edge) { e->n.fetch_add(1, relaxed); e->pending.fetch_sub(1, relaxed); }
    v = -v;
}
```

Terminal positions skip the network entirely and back up the exact result $\{-1, 0, +1\}$ from `terminal_value`, which compares the two scores from the point of view of the player to move.

5.5 Virtual loss, batching and collisions

A single leaf per network call would leave a GPU idle 99 % of the time. Each thread therefore gathers a whole batch of leaves before evaluating, which requires the descents to end up at *different* leaves — otherwise they would all follow the same argmax.

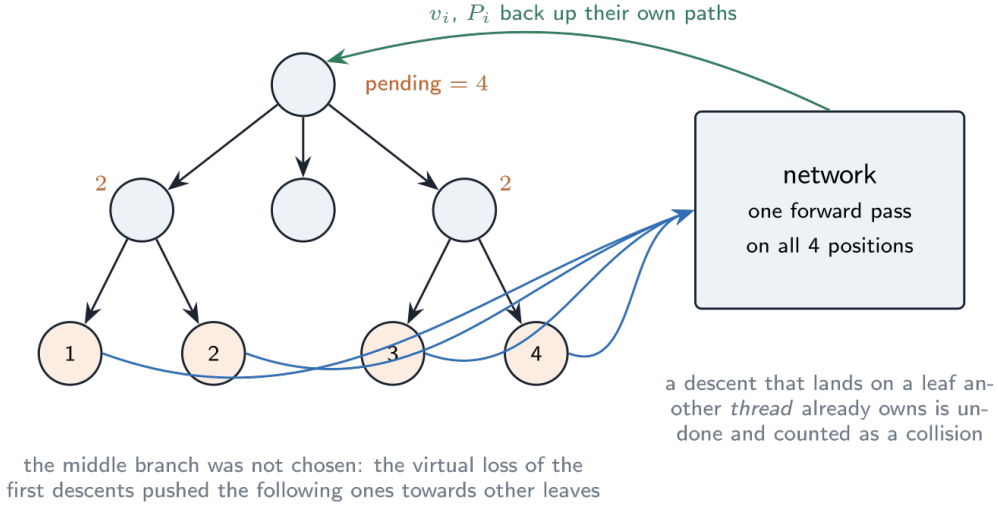


Figure 6: Gathering a batch. The **pending** counters are the virtual loss.

Every descent increments a **pending** counter on each node and edge it passes, and PUCT treats a pending visit as a **loss**:

$$Q(s, a) = \frac{W(s \cdot a) - \text{pending}(s \cdot a)}{N(s \cdot a) + \text{pending}(s \cdot a)}$$

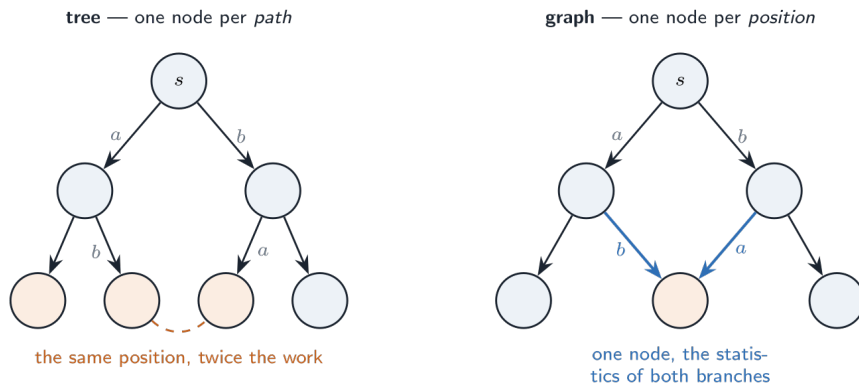
so the next descent sees the branch as temporarily worse and goes elsewhere. When the batch comes back, **backup** converts each pending visit into a real one; a descent that is abandoned calls **undo_pending** instead.

Three things can happen when a descent reaches a node in the **EXPANDING** state:

- **it is one of our own batch's leaves** — the descent piggybacks on the same network call. It records *its own path*, because two descents can reach the same leaf along different routes (and under graph search, through different parents entirely); the same value is then backed up along each path separately;
- **it belongs to another thread** — the descent is undone and counted as a collision. After **max_collisions** of those the batch is closed early;
- the state changed under us, in which case the loop simply re-reads it.

5.6 Graph search

Since move orders commute (Section 3), a tree stores the same position many times over and splits its statistics across those copies. The search instead keeps **one node per position**, in a sharded hash table, and lets several parents point at it.



The PUCT denominator uses the **edge** visits $W(s, a)$ — this parent's own share of the exploration — while $Q(s, a)$ is read from the **node**, which holds everything the search knows about that position, whichever parent paid for it.

Figure 7: A tree duplicates transposed positions; a graph shares them.

The data structures split in two:

```
struct Node { // one per position
    atomic<uint32_t> n, pending;
    atomic<float> w, w2;
    atomic<uint8_t> state;
    float net_v;
    pmr::vector<Edge> children;
};

struct Edge { // one per (position, move)
    Move action; float prior;
    atomic<Node*> child; // created on first descent
    atomic<uint32_t> n, pending; // THIS parent's share
};
```

and PUCT reads from both:

- $W(s, a)$, the denominator, is the **edge** count — this parent’s own share of the exploration, so a node that other parents have hammered does not look exhausted from here;
- $Q(s, a)$ is the **node**’s average — everything the search knows about that position, whoever paid for it.

That split is exactly KataGo’s `childWeight = rawChildWeight · edgeVisits / childVisits`, which for a search weighting every visit by 1 collapses to “the edge visits”.

Child nodes are created **lazily**, on the first descent through their edge, not when the parent is expanded. Expansion allocates `| moves | edges` (16 bytes each) instead of `| moves | full nodes`, which is the difference between ~3 200 nodes and ~20 000 nodes for a 3 200-simulation search.

5.6.1 The node key

The Zobrist hash covers the board and the side to move, and nothing else — which is right for the network cache, because that is exactly the network’s input. It is *not* enough for a node, which holds statistics and whose terminal value depends on the score split that passes can skew (Section 3). The node key therefore mixes the black score in:

```
uint64_t node_key(uint64_t zobrist_hash, const Yolah& state) {
    return zobrist_hash ^ mix(uint64_t(state.score().first) + 1);
}
```

The white score is implied: the two sum to the number of holes, which the board already determines.

The hash is maintained incrementally during a descent with `zobrist::update`, so linking an edge costs a multiply and a table probe, not a 64-square scan.

5.6.2 Tree reuse and garbage collection

With a node table, re-rooting stops being a search. `Search::reuse` looks the new position up by key; if it is there, it becomes the root with every statistic it has accumulated — at whatever depth it sat in the old graph, reached through whatever move order. The two-ply scan the tree version needed (root, children, grandchildren) is gone.

What re-rooting does need is a sweep, or memory grows without bound over a game. `collect()` marks everything reachable from the new root — a plain recursion, safe because the graph is acyclic — and destroys the rest. This runs once per move, and the mark is a generation counter rather than a per-node flag reset.

Setting `graph search: false` keeps the tree behaviour (every edge gets a fresh node under a unique key, and `reuse` falls back to the two-ply scan), which is how the two are A/B-compared.

5.7 At the root

Three things happen at the root and nowhere else.

5.7.1 Dirichlet noise

For self-play only, the root priors are mixed with Dirichlet noise so that repeated self-play games do not all follow the same line:

$$P'(s_0, a) = (1 - \epsilon) P(s_0, a) + \epsilon \eta_a, \quad \eta \sim \text{Dir}(\alpha)$$

with $\epsilon = 0.25$ and $\alpha = 0.3$ typical. The noisy priors live in `root_priors` and are used by PUCT *and* reported as the priors — the tree’s stored priors are untouched, so a re-rooted subtree is not contaminated.

5.7.2 Forced playouts

Noise only helps if the move it promotes actually gets tried more than once. A single bad evaluation is enough for PUCT to drop a move for the rest of the search, which wastes the noise. So, at the root, any child that has been visited **at least once** is forced up to

$$n_{\text{forced}}(a) = \sqrt{k P(s_0, a) W(s_0)}$$

visits ($k = 2$ in the KataGo paper) by giving it an overwhelming selection score while it is below that bar. Off by default (`forced_playouts_k = 0`); it belongs to self-play, and it is also automatically disabled on the fast searches of Section 5.8.

5.7.3 Policy target pruning

Forced playouts are deliberate distortions of the visit distribution — training on them would teach the network the noise. They are taken back out afterwards: for every child except the most-explored one, ask what visit count PUCT would *retrospectively* have wanted, by inverting the selection formula at the best child’s selection value

$$V^* = Q(s_0, a^*) + c \hat{\sigma} \sqrt{W} + \epsilon \frac{P(s_0, a^*)}{1 + W(s_0, a^*)}$$

$$n'(a) = \min \left(n(a), \left\lceil \frac{c \hat{\sigma} \sqrt{W} + \epsilon P(s_0, a)}{V^* - Q(s_0, a)} - 1 \right\rceil \right)$$

A child whose value alone already beats V^* keeps all its visits; one that only got there because it was forced collapses towards zero. These *play values* — not the raw visit counts — are what becomes π and what the move is chosen from. KataGo applies this at the root unconditionally, not only in self-play, and so does this implementation.

5.7.4 LCB move selection

The most-visited move is not always the best one to play: a move can accumulate visits early and then stop improving, while a less-visited sibling has a genuinely better and tighter value. The move played is instead the one with the best **lower confidence bound**

$$\text{lcb}(a) = Q(s_0, a) - \lambda_{\text{lcb}} \hat{\sigma}_{\bar{Q}}(a), \quad \hat{\sigma}_{\bar{Q}}(a) = \sqrt{\frac{\text{Var}(a)}{\text{ess}(a)}}$$

with $\lambda_{\text{lcb}} = 5$, restricted to children holding at least `min_visit_prop_for_lcb = 20 %` of the leader’s play value — below that the bound is meaningless. The variance comes from the child node’s W and W^2 , regularised by a prior saying the variance could be as large as the whole utility range, with a weight that vanishes as $1/n^2$:

$$\overline{u^2} \leftarrow \frac{\overline{u^2} n + (\overline{u^2} + R^2) w_p}{n + w_p}, \quad w_p = \frac{1}{n^2}, \quad R = 1$$

The winner does not simply override the choice: following KataGo, it is given just enough play value to beat every other child, which keeps π , the sort order and the move choice consistent with one another.

5.8 Playout cap randomization

A self-play move needs a large budget only if its policy target is going to be recorded. Most of the value of a self-play game is in the *game* — the sequence of positions and its result — and that only needs decent play.

So, with probability `playout_cap_fast_prob`, a search runs `nb_simulations_fast` simulations with **no root noise**, **no forced playouts**, and reports `full_search = false`; the driver records the position and the result but no policy target. The remaining moves get the full budget, the noise, the forced playouts, and a recorded π . The paper’s numbers are roughly 75 % fast at a quarter of the budget, which buys most of a full-budget game for a fraction of the network calls.

Off by default: it needs both `nb_simulations` and `nb_simulations_fast` set.

5.9 Choosing the move, and the policy target

```
play values → normalise → π    (training target)
               ↳ argmax        (competitive play)
               ↳ sample ∝ π^(1/τ) (self-play opening, ply < temperature_cutoff)
```

τ is the sampling temperature and `temperature_cutoff` the ply after which the search stops sampling and starts playing the best move. If the search could not complete a single simulation — a time budget too small for one network call — the priors stand in for π and the highest-prior move is played.

6 Learning by self-play

The network of the previous chapters was trained by *supervised* learning: it imitated the moves and the results of an existing corpus of games. It can therefore never become much stronger than whoever played those games. This chapter describes the loop that removes that ceiling: the engine plays against itself, learns from those games, plays better games with what it learned, and so on — AlphaZero’s recipe, with the refinements of KataGo.

The chapter follows the data. First the idea (Section 6.1) and the processes (Section 6.2); then self-play, which produces the data (Section 6.3); the trainer, which consumes it (Section 6.4); the evaluation matches, which measure the result (Section 6.5); how it all runs on the cluster and how the resources were sized (Section 6.6, Section 6.7); and finally KataGo’s auxiliary heads, the most promising next step (Section 6.9).

6.1 The idea

Look at what one search does to the network’s opinion. The network gives a prior $p(\cdot | s)$ and a value $v(s)$; 800 simulations later the search returns a distribution $\pi(\cdot | s)$ over the moves and a value. Both are **better** than what went in — that is the whole point of searching. So the search is a *policy improvement operator*: it maps a network to a stronger player. If we now train the network to predict π (instead of p) and the final result z of the game (instead of v), the next network starts where the last search ended. Iterate:

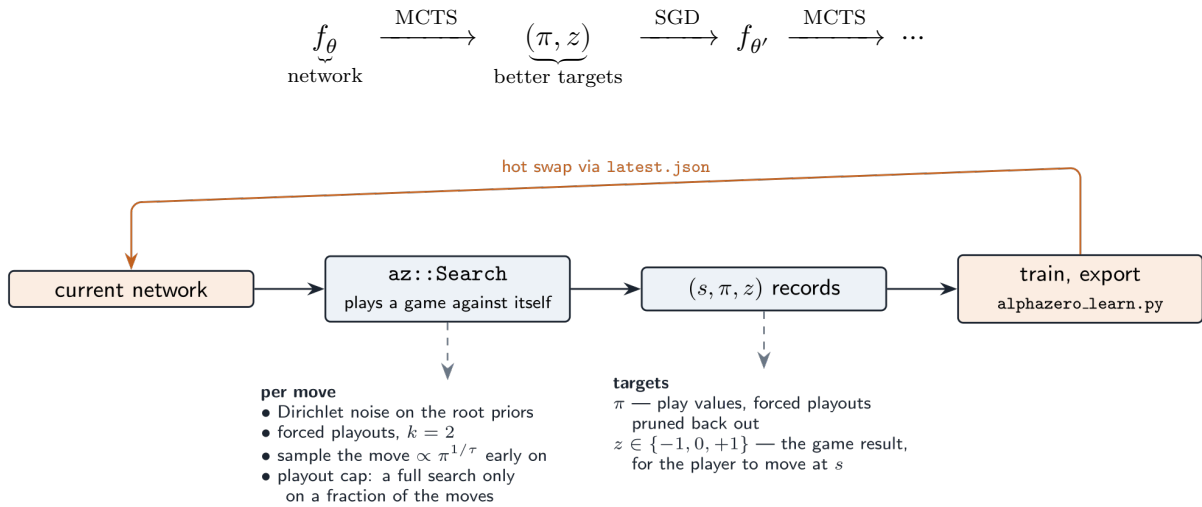


Figure 8: One turn of the loop: the network plays, the games become targets, the targets train the network.

This is *generalised policy iteration*: the search does the policy improvement step and the gradient descent does the policy evaluation step. No human games, no hand-made evaluation: the only knowledge injected is the rules, through the legal moves and the final score.

Why does it not just go round in circles? Because the two targets pull the network towards something strictly better than itself. π is the result of looking ahead with the current network — a network that predicts π directly has “compiled” 800 simulations of look-ahead into one evaluation. z is the *actual* result of the game, which no amount of self-deception can change: a network that believes a position is won while it keeps losing from there gets corrected. Each iteration the searches become stronger because the network guiding them is better, and the targets become better because the searches are stronger.

Our starting point is not a random network but the supervised `cnn_resnet_256x30_value_policy.pt`. That changes the tuning (small learning rate, no need to explore wildly at first) but not the algorithm.

6.2 The pieces and how they talk

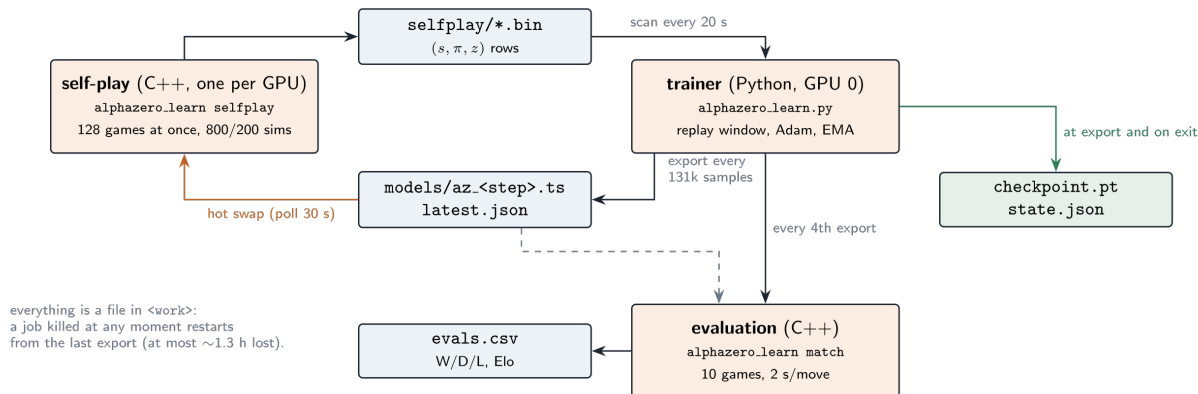


Figure 9: The learning loop. Three kinds of process, and every exchange between them is a file in the work directory.

Three programs run at the same time on one node (Figure 9):

1. **Self-play** — `alphazero_learn selfplay`, C++ (`player/alphazero_selfplay.cpp`), one process per GPU. It plays games with the newest network and writes training rows to `selfplay/*.bin`.
2. **The trainer** — `nnue/alphazero_learn.py`, Python. It is also the orchestrator: it launches and supervises the other two. It keeps a *replay window* of recent rows, takes gradient steps, and every `--export-every` training samples exports a new network (`models/az_<step>.ts`) and announces it in `latest.json`. Self-play picks the new network up at its next poll.
3. **Evaluation** — `alphazero_learn match`, C++ (`test/alphazero_learn_main.cpp`), started by the trainer every 4 hours. It plays 20 games of 2 s per move between the newest network and the previously evaluated one and appends the result to `evals.csv`.

Why files rather than sockets or shared memory? Because the run must survive being killed. A cluster job ends after two weeks and the loop must continue in the next one. With every exchange being a file that is renamed into place once it is complete (a POSIX rename is atomic), a process that dies at any moment leaves only complete files behind. The next job re-reads them and continues (Section 6.6).

There is no *gating*. The original AlphaGo Zero only promoted a new network after it beat the current one by 55 %; AlphaZero and KataGo dropped this and always self-play with the latest network, which is simpler and was found to be at least as good. The evaluation matches here are a **thermometer**, not a gate: they tell you whether learning progresses, they do not steer it.

6.3 Self-play: many games, one GPU

6.3.1 Why many games at once

A 30-block network is only efficient on the GPU with large batches: about 7 600 positions/s at batch 256 in fp16 on an RTX 3080, a small fraction of that at batch 8. A single MCTS, on the

other hand, degrades when it evaluates many leaves at once: with a batch of B leaves it must choose B descents *before* knowing any of their results (virtual loss), and beyond a few dozen leaves the descents collide and wander.

The solution is to decouple the two: run **many independent games**, each with its own single-threaded `az::Search` asking for only 8 leaves at a time, and let an `nn::BatchingEvaluator` merge the requests of all the games into one GPU call (Figure 10). With 128 games the GPU sees batches of up to 1024 positions, while each search behaves almost like a sequential one.

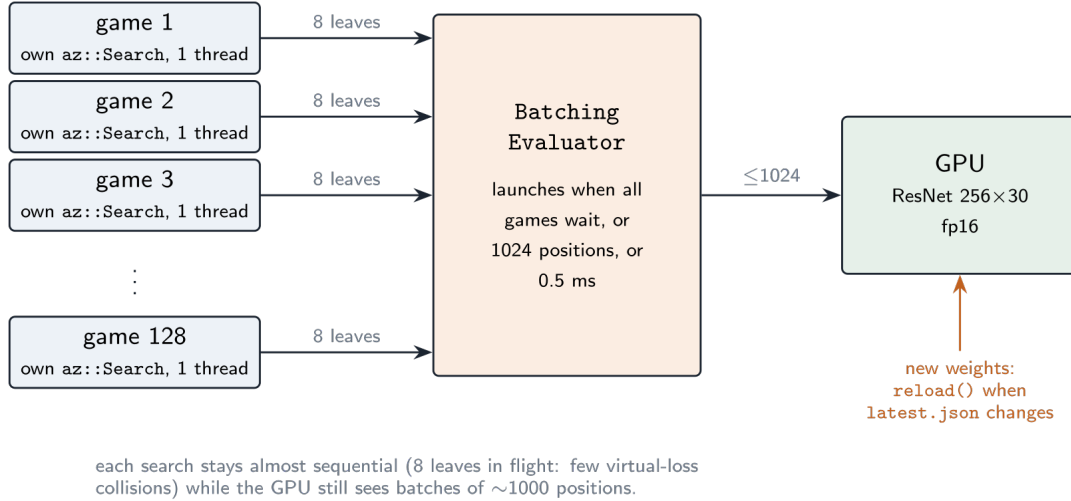


Figure 10: Many games share one network. Each search is almost sequential; the batcher gives the GPU large batches.

The batcher launches a GPU call as soon as *every* game is waiting, or the maximum batch ($8 \times$ number of games) is queued, or 0.5 ms have passed. When a game ends, its thread immediately starts the next one, so the number of games in flight stays constant; `set_nb_clients` is lowered only when a thread really exits (at the end of the run), otherwise the batcher would wait 0.5 ms for a client that will never come.

How many games? Enough to saturate the GPU, no more. On the RTX 3080, 128 games already keep it at 95 % and reach ~8 500 evaluations/s; 256 games do not do better (the batches get larger but the GPU was already busy). Two processes of 64 games on the same GPU do *worse*, ~6 000/s in total: each batch is half as large and the two processes contend for the device. Hence one process per GPU, with as many games as the GPU can absorb: 256 on the 48 GB cluster GPUs, which (A40 and RTX 8000) should have 2–2.5 times the fp16 throughput of a 3080 (Section 6.7).

The code of one game is short enough to read in full:

```
Yolah y;
std::vector<TrainingSample> samples;
search.reset(); // new game: forget the graph
while (!y.game_over() && !stop.load()) {
    const uint64_t g = generation.load(std::memory_order_acquire);
    if (g != seen) { // the network changed:
        search.reset(); // the graph's priors and values
        search.clear_cache(); // and the cached evaluations are stale
        seen = g;
    }
    const SearchResult r = search.search(y); // 800 or 200 simulations
    if (r.full_search && !r.children.empty()) // only full searches are recorded
        samples.push_back(make_sample(y, r, model_step.load()));
}
```

```

    y.play(r.best_move);                                // sampled  $\propto \pi^{(1/\tau)}$  early, best later
}
// ... then z is filled in for every sample, and the game goes to the writer

```

6.3.2 The self-play settings, and why

config/alphazero_mcts_selfplay_player.cfg:

key	value	why
nb simulations	800	the budget of a <i>full</i> search (AlphaZero)
playout cap fast prob / nb simulations fast	0.75 / 200	playout cap randomization, Section 5.8
dirichlet epsilon / alpha	0.25 / 0.3	root noise: tries moves the prior would never try
temperature / temperature cutoff	1.0 / 20	the first 20 plies are <i>sampled</i> $\propto \pi$: diverse openings
forced playouts k	2	every root move tried at least $\sqrt{kPN}$ times
policy target pruning	true	... and those forced visits are removed from π again
batch size	8	leaves per search per network call (see above)
nn cache	16 MB	per game; the cluster job raises it to 32 MB (768 games $\times$ 32 MB = 25 GB)

Playout cap randomization deserves a word, since it shapes the data. Value targets are cheap: every position of a game gets the game result z . Policy targets are expensive: a good π needs a big search. KataGo’s observation is that one can play *most* moves with a small search (200 simulations, no noise) — the game advances four times faster — and do a full 800-simulation search, and record a training row, only on a random 25 % of the moves. More games per hour means more value targets and more diverse positions; the policy targets that are recorded all come from full searches. The fast moves are **not recorded at all**: their π would be a poor target, and recording only their z would over-weight the value head.

Per recorded row this costs, on average, one full search plus three fast ones: $800 + 3 \times 200 = 1\,400$ simulations, a little less in network calls since tree reuse and the network cache save some. Measured: $\sim 1\,200$ network evaluations per training row. That single number converts any GPU’s evaluation rate into a data rate (Section 6.7).

6.3.3 What a training row contains

Each recorded position becomes a fixed-size, 336-byte record (`az::TrainingSample` in `player/alphazero_selfplay.h`, mirrored field by field by `SAMPLE_DTYPE` in `alphazero_learn.py`):

field	type	meaning
black, white, empty	$3 \times \text{uint64}$	the bitboards — exactly the network’s input
turn	uint8	0 = black to move, 1 = white
z	int8	final result for the side to move: $-1, 0, +1$
nb_moves	uint8	number of legal moves stored
root_q	float	the search’s value of the position (monitoring)
model_step	uint32	which network played this move
action[75]	uint16	policy index from*64+to of each legal move
prob[75]	uint16	$\pi(a) \cdot 65535$, rounded

π is stored sparsely, over the legal moves only (at most 75 in Yolah), quantised to 16 bits — a resolution of 1.5×10^{-5} , far below the noise of an 800-visit estimate. A dense 4096-float target would take 16 KB per row instead of 336 bytes.

The value target is filled in when the game ends. In Yolah every move scores one point for the player who makes it, so the final scores are the numbers of moves each side managed to make. A position where colour c is to move gets

$$z = \text{sign}(\text{score}_c - \text{score}_{\bar{c}}) \in \{-1, 0, +1\},$$

the same convention as the network’s value head and the search’s `terminal_value`.

Rows are grouped in files of about 2 000 (or every 10 minutes), named `<milliseconds since 1970>_<host>_gpu<k>_<seq>.bin`. Sorting the names sorts the files by age, across GPUs, hosts and jobs, which is all the trainer needs.

6.3.4 Hot-swapping the network

The supervisor thread of `run_selfplay` reads `latest.json` every 30 s. When the step changes it calls `evaluator.reload(path)`. That is safe while hundreds of games are searching: the libtorch backend takes its own mutex both in `evaluate` and in `load`, so a reload simply waits for the batch in flight and the next batch runs on the new weights. The supervisor then bumps an atomic `generation`; each game notices it before its next move and throws away its graph and its network cache, whose values came from the old weights. The rest of the game is played by the new network — a game may thus mix two consecutive networks, which KataGo also accepts: consecutive networks differ by a few hundred gradient steps.

6.4 The trainer

6.4.1 The replay window

Which rows should the network learn from? Only the most recent ones are made by a network close to the current one; old rows describe the play of a weaker player and their π is a worse target. But too small a window over-fits the last few thousand games. KataGo’s answer (`python/shuffle.py`) is a window that grows **sub-linearly** with the total number of rows N generated so far:

$$W(N) = N_0 \left(1 + \beta \frac{(N/N_0)^\alpha - 1}{\alpha} \right), \quad \alpha = 0.75, \beta = 0.4.$$

For $N \leq N_0$ everything is used. After that the window keeps growing — a stronger network deserves more data to fit — but slower and slower, so the fraction of the history in use shrinks: with KataGo’s $N_0 = 2.5 \cdot 10^5$ (the cluster setting), W is 1.1 M rows (31 %) at $N = 3.5$ M and 12 M (12 %) at $N = 100$ M (Figure 11). The derivative tells the same story: $W'(N) = \beta (N/N_0)^{\alpha-1}$ is the fraction of each new batch of rows that *enlarges* the window rather than replacing its oldest rows — 40 % at the start, 9 % after 100 M rows. W is also capped by `--max-window` (16 M rows, 5.4 GB of RAM on the cluster).

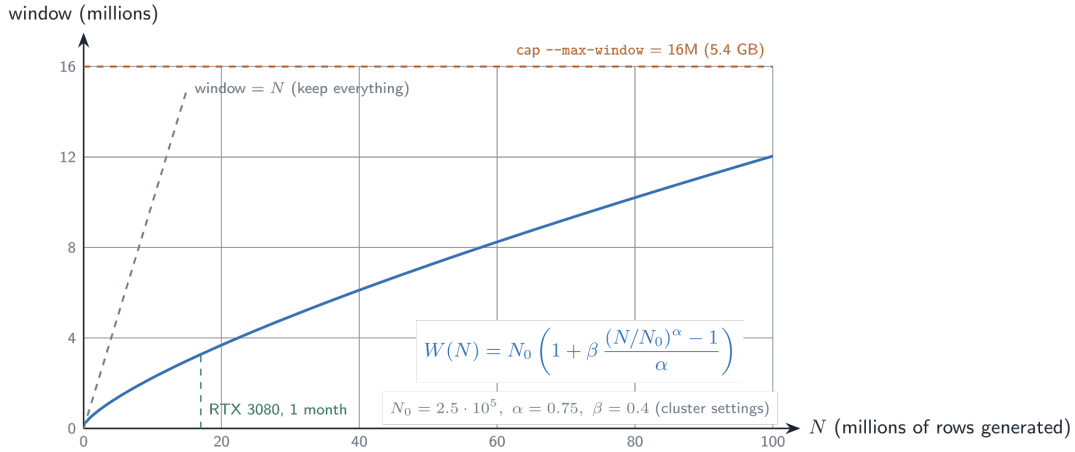


Figure 11: Replay window size as a function of the amount of data generated, with the cluster settings.

The implementation is a **ring buffer** of `--max-window` rows. New files are appended at the head; a batch draws uniform indices among the last $W(N)$ rows:

```
def sample(self, batch_size, rng):
    w = self.window_size()
    back = rng.integers(1, w + 1, size=batch_size) # 1 = newest row
    return self.buf[(self.head - back) % self.capacity]
```

On a restart, only the files that can still be in the buffer are read (the tail of the history, newest first until the capacity is reached); the headers of the others are enough to know N .

6.4.2 How much to train: the reuse ratio

The trainer and self-play run concurrently, so something must keep them in step. If the trainer ran freely it would take thousands of steps on the first rows and over-fit them; if it were too slow, self-play would keep playing with an outdated network. The rule is a **fixed number of training samples per generated row** (`--reuse`, default 4):

$$\text{samples trained} \leq r \cdot N.$$

When the trainer reaches the bound it sleeps until more rows arrive. With $r = 4$ each row is drawn about 4 times while it is in the window (in expectation; the draws are random). Training starts when $N \geq N_0$.

The trainer is never the bottleneck: one GPU trains the 256×30 network at $\sim 3\,400$ samples/s (RTX 3080, batch 1024, 4.9 GB), while even three cluster GPUs of self-play generate a few tens of rows per second, i.e. a hundred or two training samples per second at $r = 4$.

6.4.3 The loss

For a row (s, π, z) and the network output (v, ℓ) — value and the 4096 logits — the loss is the AlphaZero one with a **soft** policy target:

$$\mathcal{L}(\theta) = \lambda_v (v_\theta(s) - z)^2 - \lambda_p \sum_{a \in \text{legal}(s)} \pi(a) \log p_\theta(a | s), \quad p_\theta(a | s) = \frac{e^{\ell_a}}{\sum_{b=1}^{4096} e^{\ell_b}}.$$

With $\lambda_v = \lambda_p = 1$. Three remarks:

- The softmax runs over all 4096 logits, like in supervised training; the illegal moves are simply never targets, so their logits are pushed down. The search itself only ever reads the legal ones.
- The policy term is a cross-entropy, which decomposes as $H(\pi, p) = H(\pi) + \text{KL}(\pi \| p)$. The entropy $H(\pi)$ of the targets does not depend on θ , so what the gradient reduces is the KL divergence. The trainer logs both **p** (the cross-entropy) and **kl** — the latter is the interpretable one: 0 means the network already predicts what the search finds.
- The gradient of the policy term with respect to the logits is simply $\partial \mathcal{L} / \partial \ell_a = \lambda_p (p_\theta(a | s) - \pi(a))$: every logit is pushed up or down by how much the network under- or over-estimates what the search found.

In code, with the sparse targets gathered from the log-softmax:

```
v, logits = net(x)
logp = F.log_softmax(logits.float(), dim=1).gather(1, actions) # (B, 75)
p_loss = -(probs * logp).sum(1).mean() # padding has prob 0
v_loss = F.mse_loss(v.float(), z)
loss = args.value_weight * v_loss + args.policy_weight * p_loss
```

6.4.4 Optimiser and numerics

- **Adam**, learning rate 10^{-4} (the supervised run ended its cosine schedule at 10^{-5} from 10^{-3}), L2 weight decay 10^{-4} as in supervised training, a linear warm-up over the first 500 steps (the optimiser’s moment estimates start from zero after a fresh start), gradient norm clipped at 5.
- **Mixed precision** chosen as in the supervised trainer: bf16 on Ampere and newer (the cluster’s A40), fp16 with a gradient scaler on Turing (its RTX 8000, whose bf16 is emulated and slow). Consecutive jobs of one run may land on either, so the checkpoint must survive the switch: a bf16 job saves the state of a *disabled* gradient scaler, which is empty and which an enabled scaler refuses to load. The trainer then simply starts the scaler afresh — its scale adapts within a few steps. (`--amp` forces one or the other; that is how both switches were tested.)
- **Batch 1024**, channels-last memory layout (faster convolutions on tensor cores), `cudaNN.benchmark` on — without it, cuDNN’s default algorithms for this layout were an order of magnitude slower in our measurements.
- BatchNorm is in training mode: its running statistics follow the self-play data distribution.

6.4.5 Data augmentation: the 8 symmetries

The rules of Yolah do not change if the board is rotated by a quarter turn or mirrored: pieces move like queens and the initial position is symmetric. The position s and its image $g(s)$ under any of the 8 symmetries g of the square (the dihedral group D_4) therefore have the same value, and the policy of $g(s)$ is the policy of s with every move transformed by g :

$$v(gs) = v(s), \quad \pi(ga \mid gs) = \pi(a \mid s).$$

Each training row is shown to the network under a random one of the 8 transformations — eight times more data for free, and a network pushed towards the symmetry it should respect (Figure 12).

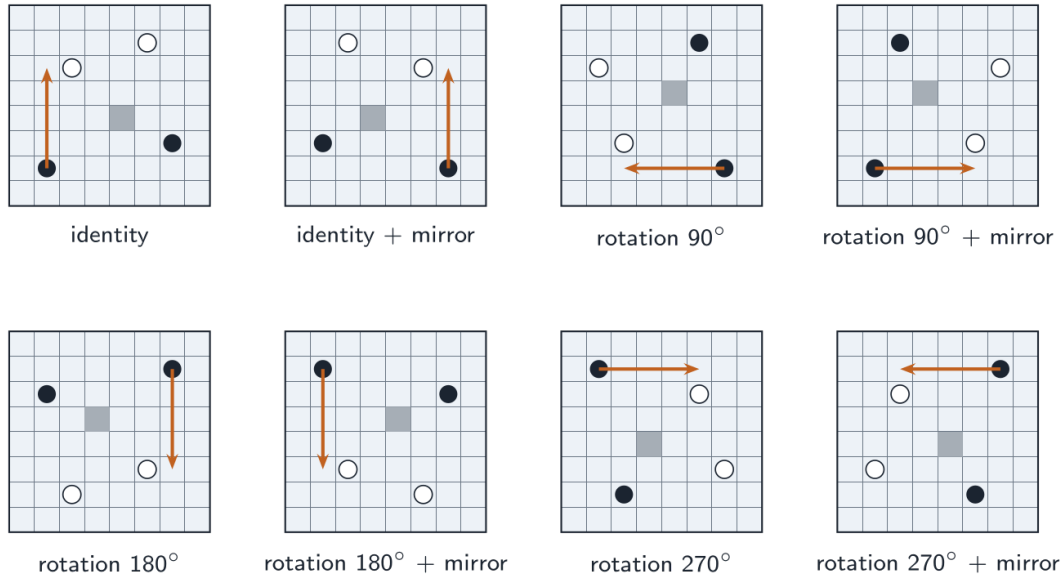


Figure 12: One position and one move under the 8 symmetries of the board. The value is the same for all eight; the move turns with the board.

The only subtlety is bookkeeping. The network input stores square q in plane cell $c = 63 - q$ (Section 3), and moves are indexed by *squares*. The tables are therefore computed on cells and converted:

```
cells = torch.arange(64).view(8, 8)
src = [f(rot90(cells, k)) for k in 0..3 for f in (identity, flip)] # (8, 64)
# new plane cell j takes the value of old cell src[g, j]
inv = argsort(src) # inv[g, old cell] = new cell
sq = 63 - inv[:, 63 - arange(64)] # old square -> new square
act = sq[:, :, None] * 64 + sq[:, None, :] # (8, 4096): move index -> move index
act[:, 0] = 0 # index 0 is the pass, and stays the pass
```

The planes are permuted with one **gather**, the move indices with one table lookup, both on the GPU. The tables were checked against the rules: for 200 self-play positions and all 8 symmetries, the transformed legal moves of s are exactly the legal moves of $g(s)$ computed by `server/yolah.py`.

6.4.6 Averaging the weights before export

The weights after the last gradient step are a noisy point: SGD bounces around the minimum. KataGo exports a *stochastic weight average* of recent snapshots; here we keep an **exponential moving average** of the weights, updated after every step:

$$\bar{\theta} \leftarrow \mu \bar{\theta} + (1 - \mu) \theta, \quad \mu = 0.995.$$

Unrolled, $\bar{\theta}_t = (1 - \mu) \sum_{k \geq 0} \mu^k \theta_{t-k}$: step $t-k$ weighs μ^k , the weights have mean age $\mu/(1-\mu) \approx 200$ steps. It is $\bar{\theta}$ that is exported and played by self-play, θ that keeps training. The BatchNorm running statistics are averaged the same way (`AveragedModel(..., use_buffers=True)`), so that the exported network is consistent.

6.4.7 Export and publication

Every `--export-every` training samples ($524\,288 = 512$ steps of 1024 on the cluster; with 4 samples per row, one network every 131 072 new rows):

1. `az_<step>.pt` — the plain `state_dict` of $\bar{\theta}$ (it can be fed to `resnet_export.py` for the CPU `.bin` backend);
2. `az_<step>.ts` — the network traced with `torch.jit.trace` in eval mode, exactly like `resnet_export.py` does, for the libtorch backend;
3. `latest.json` rewritten — self-play switches within 30 s;
4. `checkpoint.pt` (weights, EMA, optimiser, gradient scaler) and `state.json` (counters, evaluation state) rewritten.

Every write goes to a temporary name and is `os.replaced` into place.

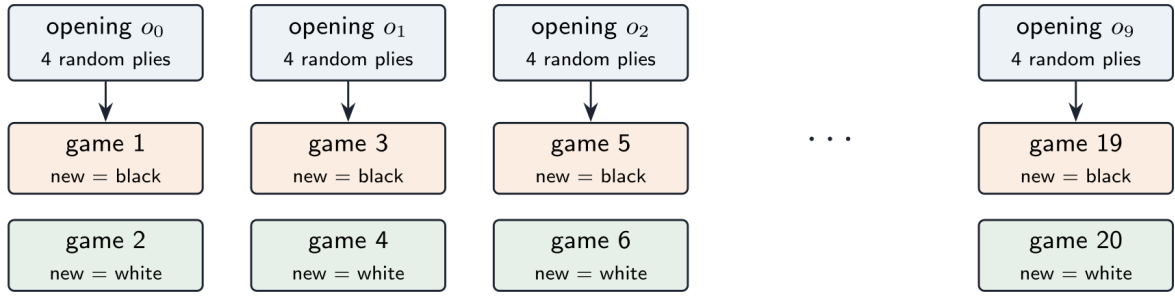
A 256×30 network is 141 MB per file and a month produces hundreds of exports, so old files are pruned (`cleanup_models`): an export that was never evaluated disappears at the next export; an evaluated one keeps its `.pt` — the history of the run, at 141 MB per evaluation — but loses its `.ts` once it can no longer be an opponent. If it is needed again (`--eval-opponents lag:K`), `Evaluator.ensure_ts` re-traces it from the `.pt`; the re-traced module gives bit-identical outputs. `--keep-all` disables pruning.

6.5 Measuring progress

6.5.1 The match protocol

Every 4 hours (`--eval-hours`; the first export of a run is evaluated at once), the newest network plays **20 games** against an older one, **2 seconds per move**, both sides using the competitive settings of `config/alphazero_mcts_eval_player.cfg` (no noise, no sampling, LCB). The opponent is by default the **previously evaluated network** (`--eval-opponents previous`; `initial` and `lag:K` are also available and can be combined). The schedule is in wall-clock time rather than in exports because the export rate depends on the GPUs, which differ between the development machine and the cluster; `--eval-every K` switches to one evaluation every K exports.

A detail matters here. Two deterministic players repeat the same game from the same position: 10 games with the new network as black would be *one* game played ten times. So each pair of games starts from a fresh random opening of 4 plies, and the new network plays it once with each



Both players search 2 s per move, no noise, no sampling: from a given position they are (almost) deterministic. Without the random openings the 10 games with the same colours would be the same game ten times. The pair of games sharing an opening cancels its bias (one colour may be favoured by it). Score $s = (W + D/2)/20$.

Figure 13: The 20 evaluation games: 10 random openings, each played once with each colour.

colour (Figure 13). The pair cancels the bias of the opening, whichever side it favours — and the first player’s advantage in Yolah is large (Section 6.7).

A match takes $20 \times 55 \times 2 \text{ s} \approx 37$ minutes. It runs on the last GPU while that GPU also plays self-play games: both players suffer the same contention, so the comparison is fair, but “2 s per move” buys fewer simulations than on an idle GPU.

6.5.2 From a score to Elo

With W wins, D draws and L losses out of $n = 20$ games, the score is $s = (W + D/2)/n$. Under the Elo model the expected score of a player Δ points stronger is $1/(1 + 10^{-\Delta/400})$; inverting,

$$\Delta = 400 \log_{10} \frac{s}{1-s}.$$

A 20–0 result would give $\Delta = \infty$, so the estimate uses a half-game prior: $\tilde{s} = (W + D/2 + 1/2)/(n + 1)$, which caps $|\Delta|$ at 645. Each evaluation adds its Δ to the opponent’s rating, giving a **chained Elo** for every evaluated network relative to the initial one (`model_elo` in `evals.csv`).

⚠ Twenty games is a thermometer, not a microscope

The standard error of a score over n games is at most $\sqrt{s(1-s)/n}$, i.e. 0.11 for $n = 20$ near $s = 0.5$ (less if there are many draws). Through the slope of the Elo curve at $s = 0.5$, $d\Delta/ds = 1600/\ln 10 \approx 695$, that is ± 78 Elo for one evaluation. The chained Elo accumulates the errors of every link like a random walk: $\pm 78 \cdot \sqrt{k}$ after k links. Read the **trend** over many evaluations, not any single one; a run that learns shows a chained Elo that climbs steadily. For a precise comparison of two networks at the end, play a few hundred games (400 games: ± 17 Elo):

```
./alphazero_learn match --config ../config/alphazero_mcts_eval_player.cfg \
--a work/models/az_A.ts --b work/models/az_B.ts --games 400 --time 500000
```

Adding `initial` to `--eval-opponents` gives an unchained measurement against a fixed reference, at the cost of another 20 games per evaluation.

6.6 Running it on the cluster

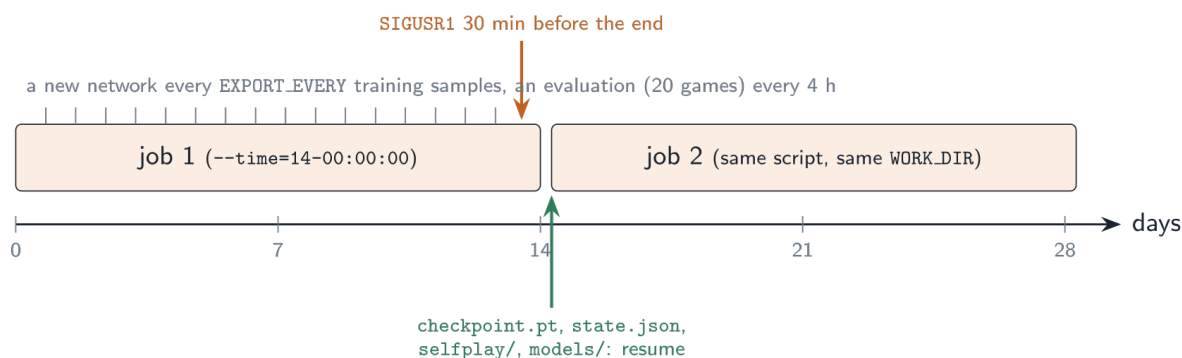


Figure 14: Two 2-week jobs make one month-long run.

6.6.1 The cluster

resource	what there is	what the run uses
GPU nodes	5 nodes, 13 cards of 48 GB: 7 A40, 6 RTX 8000	3 cards (the per-user maximum)
CPU nodes	18 nodes, 592 cores, 8 TB of RAM	nothing (see Section 6.7)
/data	44 TB, redundant, visible from every node	the work directory (~60 GB)
/calcul	17 TB, distributed	nothing

The run lives on `/data`: it must survive for a month and be reachable from whichever node the next job lands on, and its I/O is modest (about 1.2 GB of self-play rows a day, a 141 MB network per export, small reads by the trainer), so the redundant storage is the right trade.

6.6.2 The image

`nnue/alphazero_learn.def` builds a *toolchain* image: CUDA 12.4 with cuDNN, g++-13 (the code is C++23), Boost, oneTBB, Google benchmark, CMake, Python 3 and torch 2.6.0 — whose wheel also contains libtorch, the C++ library the TorchScript backend links against. Unlike the other `.def` files it does **not** contain the Yoloh sources: the job bind-mounts the repository and compiles `alphazero_learn` on the compute node, at the start of every job (about a minute). Two reasons: the CMake flags use `-march=native`, and a binary compiled on the machine that builds the image may use instructions the cluster CPUs lack (a crash with `SIGILL`); and changing the code then requires a `git pull`, not a new image. The build is cached in `WORK_DIR/build/<hash of the CPU model>`.

```
# once, on a machine where you are root (or with --fakeroot)
cd nnue && sudo singularity build alphazero_learn.sif alphazero_learn.def
# copy the .sif to the cluster, clone the repo there, copy the initial network
scp alphazero_learn.sif cluster:/data/$USER/AlphaZeroLearn/
# on the cluster: git clone the repository into ~/Yoloh, then
scp cnn_resnet_256x30_value_policy.pt cluster:Yoloh/nnue/
```

The network is stored in the repository with Git LFS, which the cluster does not have: a clone there holds a 134-byte *pointer* in place of the 144 MB file, hence the `scp`. Both the job script and the trainer check for a pointer and stop with an explicit message rather than a cryptic `torch.load` error.

6.6.3 The job script

`nnue/alphazero_learn.sh` asks for **3 GPUs, 32 CPUs, 128 GB of RAM and 14 days** (Figure 15). Its header lists every tunable as an environment variable (`SELFPLAY_GAMES`, `LR`, `REUSE`, `EVAL_HOURS`, ...); the paths (`YOLAH_DIR`, default `~/Yolah`; `SIF` and `WORK_DIR`, default under `/data/$USER/AlphaZeroLearn`) and the shell functions shared with the next script live in `nnue/alphazero_common.sh`. It does two things: compile, then run `alphazero_learn.py` in the container. The first lines of its output name the GPUs it got (`nvidia-smi`).

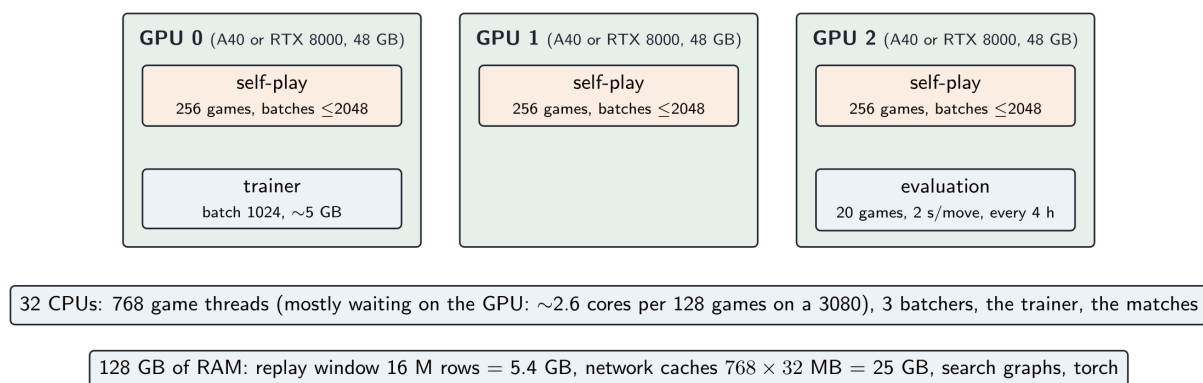


Figure 15: How the job uses the node.

6.6.4 Splitting the GPUs over several nodes

The GPU nodes have two or three cards each, and a job asking for three GPUs on *one* node may wait a long time for one to be completely free. Because the loop only communicates through files on `/data`, the three GPUs do not have to be on the same node (Figure 16):

```

sbatch --gres=gpu:1 alphazero_learn.sh      # trainer + self-play + evaluations
sbatch alphazero_selfplay.sh                # one more self-play GPU, any node
sbatch alphazero_selfplay.sh                # and another

```

`nnue/alphazero_selfplay.sh` (1 GPU, 12 CPUs, 48 GB) runs only `alphazero_learn selfplay` on the same work directory. It waits until the main job has published a network, compiles for its own node (a lock serialises two jobs compiling for the same CPU model), hot-swaps every new network like the main job's own self-play, and writes its games under a name that carries its host and job id — two jobs on one node both see their GPU as number 0, so the GPU number alone would not be unique. It stops by itself when no new network has appeared for 6 hours (`--max-stale-hours`): when the main job is over, it does not keep playing with an outdated network until its own time limit.

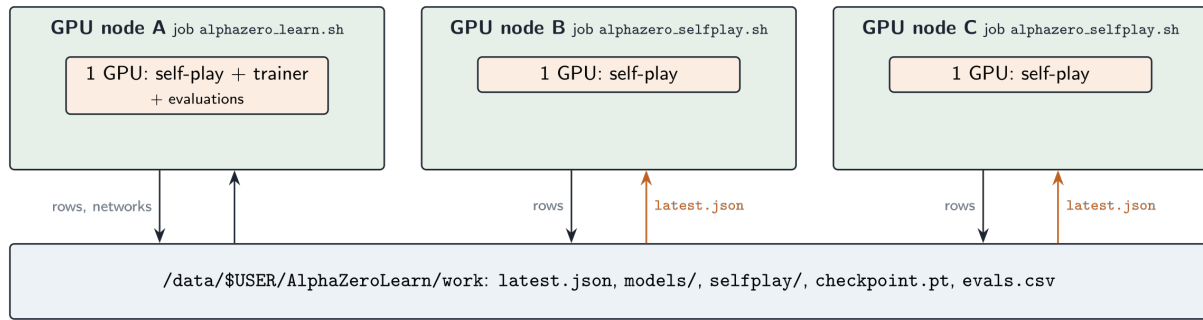


Figure 16: Three 1-GPU jobs on three nodes, one run: every exchange is a file on /data.

6.6.5 Stopping cleanly

Stopping cleanly matters, because the last minutes of a job are the ones that would otherwise be lost:

- `#SBATCH --signal=B:USR1@1800` makes SLURM send `SIGUSR1` to the batch script 30 minutes before the time limit. The script's `trap` forwards it as `SIGTERM` to the trainer, which stops at its next step, writes `checkpoint.pt`, sends `SIGTERM` to the self-play processes — they finish their current searches, write the finished games and exit — and stops the evaluation (which stays in `pending_evals` and is played first by the next job).
- In case the signal is lost, the script also reads the job's remaining time (`squeue -o %L`) and passes `--max-hours` = that minus 20 minutes.
- The job script starts the container with `exec`, so the signal reaches Singularity itself, which forwards it into the container. Sending it to a bash subshell around Singularity would kill the subshell and leave the trainer and its self-play processes running as orphans. This was caught in testing.
- The trainer refuses to start if it cannot see a GPU. The usual cause is a container whose CUDA driver is not bound, and without the check self-play would fall back to the CPU and run about 50 times slower for two weeks. The self-play log's second line names the backend and device actually used.
- Even a `kill -9` loses at most the work since the last export and the games in progress: self-play rows are on disk as soon as a file is complete.

To continue, submit **the same script with the same `WORK_DIR`**:

```
sbatch alphazero_learn.sh      # days 0-14
sbatch alphazero_learn.sh      # days 14-28: resumes from WORK_DIR
```

`RESUBMIT=1` makes the job resubmit itself after a clean stop (never after a crash, to avoid a resubmission loop).

6.6.6 What to watch

file	what it tells you
<code>alphazero_learn_<job>.out</code>	the trainer's log: losses every 2 min, exports, evaluations
<code>WORK_DIR/evals.csv</code>	one line per match: W/D/L, score, Elo difference, chained Elo

file	what it tells you
WORK_DIR/train_log.csv	step, rows, window, value loss, policy loss, KL, samples/s
WORK_DIR/logs/selfplay_gpu<k>.log	games/s, rows/s, evals/s, game length, colour balance (every 5 min)
WORK_DIR/logs/eval.log	every evaluation game, move by move result
WORK_DIR/evals/*.json	the complete record of every evaluation game

Healthy signs: the value loss slowly decreasing, the KL staying small (the network keeps up with the search, typically a few tenths), the chained Elo climbing, the colour balance of self-play evolving slowly. Warning signs: a value loss that collapses towards 0 (the window is too small: it memorises), a KL that keeps growing (the learning rate is too small or the reuse too low), games getting systematically shorter (a degenerate strategy — look at `evals/*.json`).

6.7 Sizing for the cluster

6.7.1 What was measured

On the development machine (one RTX 3080, 10 GB, WSL2), with the initial network and 800/200 simulations:

quantity	value
self-play, 128 games	~8 500 evaluations/s, GPU at 95 %
self-play, 256 games	no better (the GPU is saturated)
self-play, 2 processes $\times$ 64 games	~6 000 evaluations/s in total (worse)
games	~0.51 games/s — ~44 000 per day, 55 plies on average
training rows (full searches only)	~6.9 rows/s — ~600 000 per day
evaluations per row	~1 200
trainer, batch 1024	~3 400 samples/s, 4.9 GB (alone on the GPU)
colour balance	black wins 61 %, white 8 %, draws 31 %

The last line is a property of Yolah, not of the loop: the first player has a large advantage, which is why the evaluation plays every opening with both colours.

6.7.2 From the measurements to the settings

Self-play is the bottleneck, as in every AlphaZero-like system; the rest is arithmetic.

Data rate. With ~1 200 evaluations per row, a GPU doing E evaluations per second produces $E/1200$ rows/s, i.e. $72 E$ rows per day. How fast are the cluster's cards? The network is convolutions on 8×8 boards with 256 channels: compute-bound, in fp16 with fp32 accumulation. The A40 (Ampere, ~150 TFLOPS of fp16 tensor work) and the RTX 8000 (Turing, ~130 TFLOPS) do that at full rate, whereas a GeForce RTX 3080 does it at half its nominal rate (~60 TFLOPS). From the specifications, then, a cluster card should be 2 to 2.5 times a 3080. Taking a conservative $2 \times$: $3 \times 17\,000$ evaluations/s ≈ 3.6 M rows per day, **~100 M rows in 28 days** (≈ 7 M games). Each self-play log prints `evals/s`, so the real number is known after five minutes; everything below scales linearly with it.

Games per GPU: 256. Enough to saturate a GPU 2–2.5 times faster than the 3080, which saturates at 128. More games cost only RAM and threads, both plentiful. One process per GPU (measured above: two per GPU is worse).

Replay window: $N_0 = 250\,000$, **cap 16 M.** KataGo’s own N_0 . At 3.6 M rows/day the first 250 000 rows arrive within two hours, and after a month $W(N) \approx 12$ M rows (Figure 11) — 4 GB, well under the 16 M cap (5.4 GB) chosen to leave headroom if the GPUs are faster than assumed.

Export every 524 288 samples. At $r = 4$ that is a new network every 131 072 rows, about every 50 minutes at 3.6 M rows/day — frequent enough for self-play to play with a fresh network, rare enough that the export (a trace and two 141 MB files) is negligible.

Evaluation every 4 hours, 20 games. Six per day, 37 minutes each on the last GPU: about 15 % of that GPU’s self-play. About 170 evaluations over the month, i.e. a 24 GB history of evaluated networks.

RAM and CPUs. $768 \text{ games} \times 32 \text{ MB of network cache} = 25 \text{ GB}$, the replay buffer 5.4 GB, plus the search graphs and torch: well under the 128 GB requested. The game threads spend most of their time waiting for the GPU: on the 3080, 128 games used ~ 2.6 cores. The CPU work (descents, backups, encoding the positions) is proportional to the number of *evaluations*, not to the number of games, so a GPU twice as fast needs about twice that: $3 \times 2 \times 2.6 \approx 16$ cores for self-play, plus the trainer’s data pipeline and the 4 search threads of each evaluation player. 32 CPUs leave a comfortable margin, and asking for less than a whole node makes the job easier to schedule.

Heterogeneous GPUs. A job may get A40s, RTX 8000s or, in principle, a mix. Self-play does not care. The trainer switches between bf16 and fp16 (see the optimiser notes above). The evaluation compares two networks on the *same* GPU, so the match is fair whatever the card; but “2 s per move” buys more simulations on an A40 than on an RTX 8000, so the absolute level of play of two evaluations made in different jobs may differ slightly — another reason to read the Elo trend rather than single values. To pin a card type, use the cluster’s gres type (`--gres=gpu:a40:3; sinfo -o "%N %G"` lists them).

Why not the CPU nodes? Self-play could in principle run on the 592 CPU cores with the pure C++ backend (`nn::ResNetCPU`). Measured on the development machine it evaluates 130–160 positions/s on about ten cores; a 33-core node would do perhaps 450/s, and all 18 nodes $\sim 8\,000$ /s — about half of one A40, for a gain of roughly 15 % on three GPUs. That would not pay for the complexity (a second network format to export at every update, jobs spread over 18 nodes) nor for occupying a shared resource. The GPUs are where the network is fast.

Disk. ~ 34 GB of self-play rows for 100 M rows, ~ 24 GB of evaluated networks: plan for ~ 60 GB in `WORK_DIR`.

6.8 Parameters of the learning loop

All are command-line options of `alphazero_learn.py`; the `.sh` exposes the important ones as environment variables and sets the cluster values:

option	default	cluster (<code>.sh</code>)	meaning
<code>--selfplay-games</code>	128	256	concurrent games per self-play process (one per GPU)

option	default	cluster (.sh)	meaning
--selfplay-set 'k=v'		nn cache=32	override a key of the self-play config
--batch-size	1024	1024	training batch
--lr	1e-4	1e-4	Adam learning rate (after warm-up)
--warmup-steps	500	500	linear warm-up
--weight-decay	1e-4	1e-4	L2
--ema-decay	0.995	0.995	weight averaging for export; 0 = raw weights
--reuse	4	4	training samples per generated row
--min-rows	100 000	250 000	N_0 : rows before training starts, window base
--max-window	4 M	16 M	ring buffer capacity
--export-every	131 072	524 288	training samples between two networks
--eval-hours	4	4	time between two evaluations
--eval-every	0		evaluate every K exports instead (0 = use --eval-hours)
--eval-games / --eval-seconds	20 / 2	20 / 2	the match
--eval-opening-plies	4	4	random plies before each pair of games
--eval-opponents	previous	previous	previous , initial , lag:K , comma separated
--train-gpu / --eval-gpu	0 / -1		trainer on the first GPU, matches on the last
--keep-all	off		keep every exported network
--max-hours	0	time limit — 20 min	stop cleanly after this long
--no-selfplay	off		train only (self-play running elsewhere)
--allow-cpu	off		run without CUDA (tests only)
--amp	auto		trainer precision: auto (bf16 on Ampere+, else fp16), bf16 , fp16

6.9 KataGo's auxiliary heads

Everything so far trains the network on two targets per position: one scalar z and one distribution π . KataGo's most effective idea on top of AlphaZero is to ask the network for **more predictions than the search needs** — extra *heads* that exist only during training (Figure 17). This section explains what they are, why they help, what they would be in Yolah, and how they would fit into this code. They are not implemented yet.

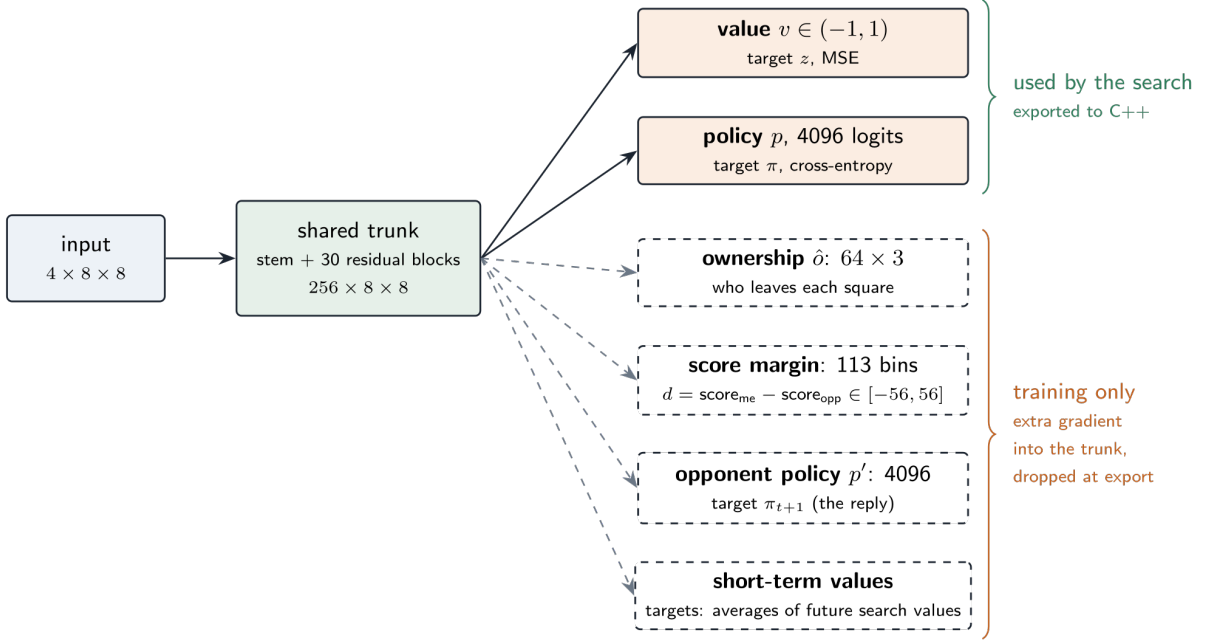


Figure 17: The two heads the search uses, and the auxiliary heads that exist only to train the trunk.

6.9.1 Why predicting more helps

Think of the information a game gives to the value head: **one bit-and-a-half** — win, draw or loss — shared by 55 positions. A position early in the game gets the credit or blame for 50 later moves it had nothing to do with. The value target is extremely *noisy*, and that noise is what makes value learning the slow part of AlphaZero.

An auxiliary target is any quantity that is (a) known at the end of the game for free, (b) much richer than z , and (c) related to what the network must understand. Its head is small — a 1×1 convolution and a linear layer on top of the shared trunk — and is thrown away at export time. What remains is its **gradient into the trunk**: the trunk must now compute features that also predict the auxiliary target, and features that explain *where* and *by how much* a game is won are exactly the features that make the value head accurate. It is regularisation by multi-task learning, and in the KataGo paper's ablations removing these targets slows learning down markedly, most of all early in a run.

The total loss becomes a weighted sum; the weights are small because the auxiliary heads must shape the trunk without dominating the two heads that matter:

$$\mathcal{L} = \mathcal{L}_v + \mathcal{L}_p + w_{\text{own}} \mathcal{L}_{\text{own}} + w_{\text{score}} \mathcal{L}_{\text{score}} + w_{\text{opp}} \mathcal{L}_{\text{opp}} + w_{\text{stv}} \mathcal{L}_{\text{stv}}.$$

6.9.2 Ownership

In Go, KataGo predicts for each of the 361 points who owns it at the end of the game — 361 targets per position instead of one. The loss is a per-point cross-entropy, scaled by the board area:

$$\mathcal{L}_{\text{own}} = -\frac{1}{64} \sum_{q=1}^{64} \sum_c o_q(c) \log \hat{o}_q(c), \quad w_{\text{own}} \approx 1.5,$$

where o_q is the one-hot final owner of square q and $\hat{o}_q$ the network’s softmax (KataGo writes the same weight as $1.5/b^2$ on the *sum* over the $b \times b$ points, which is this mean times 1.5).

Yolah has an exact equivalent, and a particularly clean one. Every move leaves a hole on the square the piece came from, and scores one point for its player (`Yolah::play`). So at the end of a game **each square is either a hole left by black, a hole left by white, or neither**, and the number of holes each player left *is* their score (Figure 18, a real game of the initial network: 29–26). The ownership map is the final score decomposed square by square: it tells the trunk not only *that* black wins but *which regions* black managed to use.

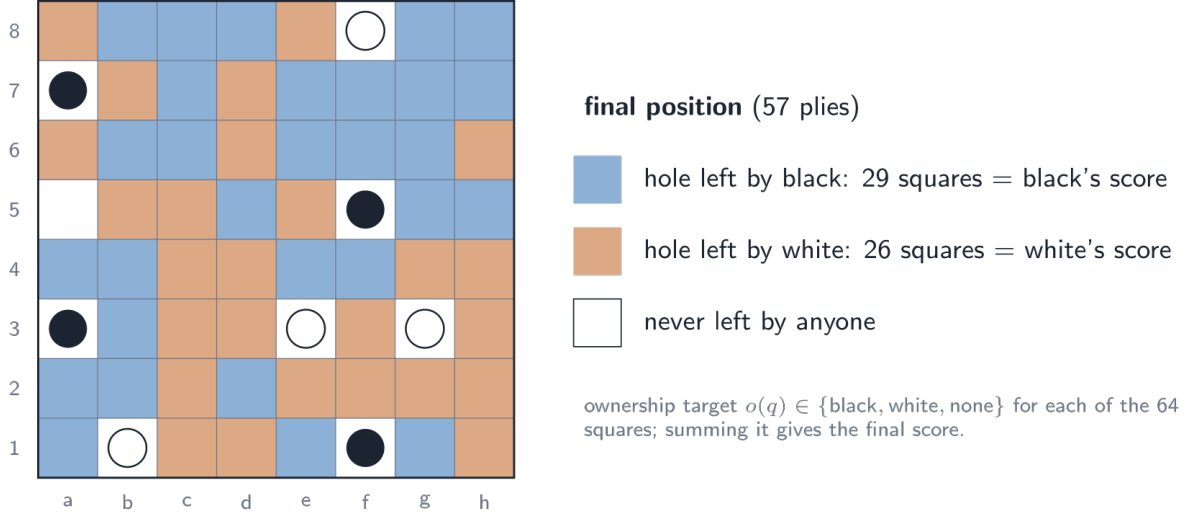


Figure 18: The ownership target in Yolah, on a real game: every square is labelled by the player who left it. Counting the labels gives the final score.

The head: a 1×1 convolution from the 256 trunk channels to 3 channels (black, white, none) — an $8 \times 8 \times 3$ output, one softmax per square. Relative to the side to move (“mine / the opponent’s / none”), as KataGo does, so that the target does not depend on the colour. Under the 8 symmetries the ownership map transforms exactly like the input planes (the same **gather**).

6.9.3 Score margin

z only says who won. The margin $d = \text{score}_{\text{me}} - \text{score}_{\text{opp}}$ says by how much, and distinguishes a crushing win from a lucky one. In Yolah $|d| \leq 56$ (64 squares minus 8 pieces), so the head can predict the whole **distribution** of the margin — 113 logits, one per possible value — trained with a cross-entropy against the one-hot final margin:

$$\mathcal{L}_{\text{score}} = -\log \hat{P}(d_{\text{final}}),$$

plus, as in KataGo, a term on the cumulative distribution so that predicting +11 when the truth is +12 costs less than predicting -30: $\sum_x (\hat{F}(x) - \mathbb{1}[x \geq d_{\text{final}}])^2$ with $\hat{F}$ the predicted CDF. KataGo also uses the predicted score in the search (a small utility for winning by more); here it would be training-only.

6.9.4 Opponent's reply

A second policy head predicts π_{t+1} , the search's distribution at the *next* position — the opponent's reply. It forces the trunk to represent the opponent's threats, not only one's own options. KataGo's weight is $w_{\text{opp}} \approx 0.15$. It needs no new data: the target is the π of the next row of the same game, when that move was a full search (with playout cap randomization, 25 % of the time; otherwise the term is masked out).

6.9.5 Short-term value targets

z is noisy because it is far in the future. Later versions of KataGo add value targets that look only a few moves ahead: exponentially weighted averages of the *search values* of the next positions,

$$v_t^{(\lambda)} = (1 - \lambda) \sum_{k \geq 0} \lambda^k q_{t+k} \quad (\text{sign-corrected for the side to move}),$$

with horizons $1/(1 - \lambda)$ of a few moves to a few tens of moves, each trained by its own small head with a squared error. They are the temporal-difference idea of reinforcement learning: a target that is less exact than z (it inherits the search's errors) but far less noisy. We already record the search value (`root_q`) of the full-search positions; the fast searches' values would have to be recorded too.

6.9.6 How it would fit into this code

The key point is that **nothing in C++ has to change**:

1. **The network** gets the extra heads as extra modules; `forward` returns all outputs during training.
2. **The export** traces a small wrapper whose `forward` returns only `(v, p)`; the `.ts` file has the same interface as today, and `resnet_export.py` writes the `.bin` from the value and policy weights only, so both C++ backends load the network unchanged.
3. **The data** needs, per row, the final ownership map (2 bits $\times$ 64 squares = 16 bytes), the final margin (1 byte), and a game identifier and ply so that the trainer can pair a row with the next one of the same game (the opponent-policy and short-term targets). That is a version 2 of `TrainingSample` — the file header already carries a version and the row size precisely for this.
4. **The starting point**: the trunk and the two existing heads load from the current network; the new heads start random, with their weights ramped up over the first few thousand steps so that their initially large, random gradients do not disturb a trunk that already plays well.

The ownership and score heads are the ones with the best ratio of expected gain to effort: the targets are free, exact and — in Yolah — sum to the very quantity the game is decided on.

7 Code map

file	what lives there
nnue/cnn_resnet_value_policy_chunked.py	Net, ResBlock, encode_cnn, ChunkedShuffleLoader, the DDP trainer — the production script
nnue/cnn_resnet_value_policy.py	the same network with a stock DataLoader + LeanDistributedSampler
nnue/resnet_export.py	BatchNorm folding, the .bin writer, the TorchScript trace, the test vectors
nnue/resnet.{h,cpp}	nn::ResNetCPU — im2col + Eigen GEMM, OpenMP chunking, FTZ/DAZ
nnue/resnet_torch.{h,cpp}	nn::ResNetTorch — TorchScript on the GPU
nnue/nn_evaluator.{h,cpp}	the Request/Result/Evaluator interface, encode_planes, action_index, make_evaluator
nnue/batched_evaluator.{h,cpp}	nn::BatchingEvaluator — merges several search threads into one backend batch
player/nn_cache.h	az::NNCache — direct-mapped, lock-stripped cache of network outputs
player/alphazero_mcts.h	SearchParams, ChildStat, SearchResult, Search — and the algorithm overview
player/alphazero_mcts.cpp	everything below: the node table, selection, backup, the worker loop, the root logic
player/alphazero_mcts_player.{h,cpp}	the Player wrapper and the JSON schema
test/alphazero_mcts_check.cpp	search / reuse self-checks and a head-to-head harness
test/alphazero_check_main.cpp	the alphazero_check CLI
player/alphazero_selfplay.{h,cpp}	TrainingSample (the row format), run_selfplay — many games on one batched network, hot swap
test/alphazero_learn_main.cpp	the alphazero_learn CLI: selfplay and match
nnue/alphazero_learn.py	the trainer and orchestrator: replay window, loss, symmetries, EMA, export, evaluations
nnue/alphazero_learn.{def,sh}	the cluster image and the SLURM job
config/alphazero_mcts_selfplay_player.cfg	the search settings of self-play
config/alphazero_mcts_eval_player.cfg	the search settings of the evaluation matches

The functions worth knowing in `alphazero_mcts.cpp`:

function	role
<code>select_child</code>	Section 5.2 — one PUCT argmax, plus forced playouts at the root
<code>utility_stddev_factor</code>	$\hat{\sigma}(s)$ from the node's W and W^2
<code>fpu_value</code>	Section 5.3 — what an unvisited child is assumed to be worth
<code>priors_from_logits</code>	softmax over the legal moves, with <code>policy_temperature</code>

function	role
expand	build the edge vector, store <code>net_v</code> , publish EXPANDED
backup / undo_pending run_worker	negamax accumulation / virtual-loss rollback gather → evaluate → expand → back up, per thread
get_or_create, lookup, collect, mark reuse, new_root, expand_root	the node table and its garbage collection re-rooting between moves
root_play_values	policy target pruning and the LCB adjustment
make_result	the statistics, π , and the move choice

8 Parameter reference

All of these are JSON keys of `AlphaZeroMCTSPlayer` (spaces, not underscores) and fields of `az::SearchParams` (underscores).

8.1 Budget and parallelism

key	default	meaning
microseconds	1 000 000	wall-clock budget; 0 = use the simulation budget only
nb simulations	0	simulation budget; 0 = time only
nb threads	1	search threads sharing the graph ("hardware concurrency" allowed)
batch size	backend hint	leaves gathered per thread per network call
max collisions	= batch size	cross-thread collisions that close a batch early
nn cache	64	MB of network-output cache; 0 disables it

8.2 PUCT

key	default	meaning
cpuct exploration	1.0	c_{puct}
cpuct exploration log	0.45	c_{log} ; 0 gives a constant coefficient
cpuct exploration base	500	c_{base}
cpuct utility stdev scale	0.85	λ ; 0 disables $\hat{\sigma}$ (plain AlphaZero)
cpuct utility stdev prior	0.40	σ_0 — see the calibration note in Section 5.2
cpuct utility stdev prior weight	2.0	w_p , in visits

key	default	meaning
policy temperature	1.0	softmax temperature on the priors; >1 flattens

8.3 First play urgency

key	default	meaning
fpu reduction	0.2	r away from the root
root fpu reduction	0.1	r at the root
fpu loss prop / root fpu loss prop	0.0	drag the estimate towards a loss
fpu parent weight by visited policy	true	blend $\bar{q}$ and v_{net} by the visited mass
fpu parent weight by visited policy pow	1.0	the exponent p
fpu parent weight	0.0	constant blend, used only when the above is false

8.4 Graph and root

key	default	meaning
graph search	true	one node per position; false = a plain tree
use lcb	true	play the best lower confidence bound
lcb stdevs	5.0	λ_{lcb}
min visit prop for lcb	0.20	eligibility threshold, as a fraction of the leader
policy target pruning	true	invert PUCT to undo the forced playouts
reuse tree	true	re-root instead of starting from scratch

8.5 Self-play

key	default	meaning
dirichlet alpha / dirichlet epsilon	0.3 / 0.0	root noise; $\epsilon = 0$ disables it
temperature / temperature cutoff	0.0 / 0	sample the move $\propto \pi^{1/\tau}$ below the cutoff ply
forced playouts k	0.0	k ; the paper uses 2.0, self-play only
playout cap fast prob	0.0	fraction of moves that get the small budget
nb simulations fast	0	that small budget

9 Measurements

All figures below: ResNetCPU, 256×30 weights, one search thread, WSL2 on a desktop CPU. Reproduce with

```
./alphazero_check --no-bench --sims 3200 --set 'graph search=false'
```

9.1 What graph search actually buys

Four positions, 3 200 simulations each:

position		graph	tree
0 (ply 0)	network calls	3 195	2 552 + 646 cache hits
	distinct positions	3 195	2 552
	sims/s	156	181
1 (ply 17)	network calls	3 141	2 632 + 515 cache hits
	distinct positions	3 141	2 632
	sims/s	159	175
2 (ply 19)	network calls	3 175	2 947 + 231 cache hits
	distinct positions	3 175	2 947
	sims/s	156	162
3 (ply 10)	network calls	3 195	2 630 + 570 cache hits
	distinct positions	3 195	2 630
	sims/s	155	178

Read this carefully, because the naive reading is backwards. Graph search is **not** a speedup — it is 5–15 % *slower* per simulation. What it changes is what a simulation buys.

In tree mode, 7–20 % of the leaves are transpositions. The search allocates a duplicate node for a position it has already evaluated, the network cache serves the value for free, and the simulation backs that value up along its own path. Those are cheap and they are genuine MCTS updates — the path they refresh is a real one — they are simply *redundant*: no new information entered the search. In graph mode the descent links to the existing node and *keeps going*, ending at a position nobody has evaluated, which costs a real network call.

Same simulation count, same number of nodes, but **20–25 % more distinct positions examined**, and every shared node pools the statistics of all its parents instead of splitting them across copies. Correcting for the lower simulation rate, the advantage is **4–8 % more distinct positions per wall-clock second** — consistently across all four positions — plus the pooling, which no eval count captures.

So the trade is: cheap redundant simulations against fewer, more informative ones. Which of those wins games is not something an eval counter can answer.

! This is not yet a strength measurement

More distinct positions per second is a proxy, not a result. The question “does graph search win games” needs a head-to-head:

```
./alphazero_check --games 200 --time 1000000 --opponent <tree-mode.cfg>
```

and a 4–8 % node advantage is worth, on the experience of other engines, some single-digit

number of Elo — which a few hundred games cannot separate from noise (200 games carry a standard error of roughly ± 25 Elo). Resolving it means thousands of games on the libtorch backend.

The default is `graph search: true` on the strength of the proxy and of the statistic pooling, not on a measured Elo gain. Nothing in this document should be read as one. The flag exists so the decision can be re-tested cheaply.

9.2 Node allocation

Children are now created lazily, one per descent, instead of eagerly at expansion. At 400 simulations the eager version allocated **20 335** node objects; the lazy one allocates **396** — one per position actually reached. The unreached children cost a 16-byte **Edge** instead of a 56-byte **Node** with its own vector. This one is unambiguous: fewer allocations, less memory, better locality, and it is what makes a node table cheap enough to maintain.

9.3 What LCB and pruning change

At 400 simulations, position 2 (ply 19) with the raw visit counts:

```
f2:d2  N: 194  Q: +0.639  lcb: +0.479
f2:e3  N:  87  Q: +0.759  lcb: +0.533  ← played
```

The most-visited move is not the one with the best lower bound, and LCB plays the second. That is the intended behaviour, but at 400 simulations with $\lambda_{\text{lcb}} = 5$ it is an aggressive override — the eligibility threshold ($87 - 0.2 \times 194$) is doing a lot of work. Worth re-checking at the budgets the engine will actually play at.

Policy target pruning is similarly consequential: at position 0 it drives several children with a dozen visits each to $\pi = 0$, because their priors are so small that PUCT would never have visited them — they were reached through FPU, not through their policy. That is exactly what the mechanism is for, but it makes π markedly peakier than the visit distribution, and it should be eyed before it is trained on.

10 References

- Silver et al., *Mastering the game of Go without human knowledge*, Nature 2017 — AlphaGo Zero: PUCT, Dirichlet noise, the two-headed network.
- Silver et al., *A General Reinforcement Learning Algorithm that Masters Chess, Shogi and Go through Self-Play*, 2018 — AlphaZero, and the $c_{\text{init}} + \log((N + c_{\text{base}} + 1)/c_{\text{base}})$ coefficient.
- D. J. Wu, *Accelerating Self-Play Learning in Go*, arXiv:1902.10565, 2019 — the KataGo paper: playout cap randomization, forced playouts, policy target pruning.
- KataGo source, `cpp/search/` — `searchexplorehelpers.cpp` for the selection formula, `getFpuValueForChildrenAssumeVisited` for FPU and $\hat{\sigma}$, `searchresults.cpp` for the play-selection values and LCB, `searchhelpers.cpp` for `getSelfUtilityLCBAndRadius`, `searchparams.cpp` for the defaults quoted here.
- Leela Chess Zero — the FPU-reduction formulation this and KataGo share.

11 Annotated source

Every file the document describes, in full, generated from the working tree by `doc/make_appendix.py`. Each one carries an orientation note, a table of the symbols it defines, and its complete source with line numbers — so the sections above can point at `alphazero_mcts.cpp:212` and mean it.

The running commentary lives in the sources themselves rather than being duplicated here; what the tables add is the one-line answer to “what is this for” for every symbol, which a comment sitting next to a definition cannot give you at a glance.

11.1 The search

These five files are the search proper. They depend on `game.h`, `misc.h`, `zobrist.h` and the `nn::Evaluator` interface of the next part, and on nothing else.

11.1.1 `player/alphazero_mcts.h`

317 lines.

The public surface: the parameter block, what a search reports, and the class. The header comment is the algorithm overview — the same one Section 5.2 through Section 5.8 expand on.

`SearchParams` is a plain aggregate with the KataGo defaults baked in, so `az::SearchParams{}` is a usable configuration once a budget is set. Everything private is forward-declared: `Node`, `Edge`, `PathStep` and `Worker` are defined in the `.cpp`, so nothing that includes this header sees the graph representation.

symbol	role
<code>SearchParams</code>	every knob, grouped: budget, parallelism, PUCT, FPU, graph, root, self-play
<code>ChildStat</code>	one root move as reported: move, prior, raw edge visits, <code>q</code> , <code>q2</code> , play value, LCB
<code>SearchResult</code>	the move, the root value, the counters, <code>full_search</code> , the children and π
<code>SearchResult::to_string</code>	the one-line-plus-children summary printed by <code>"verbose": true</code>
<code>Search</code>	the search object: owns the node table, the network cache and the parameters
<code>Search::search</code>	run one search from a position and pick a move
<code>Search::reset</code>	drop the graph (new game); the network cache survives
<code>Search::clear_cache</code>	drop the cached network outputs (after new weights)
<code>Search::params</code>	read/write access to the live parameters
<code>Search::Shard</code>	one stripe of the node table: a mutex and a <code>hash → Node*</code> map

```

1  #ifndef ALPHAZERO_MCTS_H
2  #define ALPHAZERO_MCTS_H
3  // alphazero_mcts.h – AlphaZero-style Monte-Carlo tree search for Yohai, with
4  // the refinements KataGo added on top of it.
5  //
6  // Inspired by MCTS_mem_player.h (same pool-allocated storage, same lock-free
7  // multi-threading style) but the leaf is evaluated by the two-headed ResNet
8  // instead of a random playout, and children are selected with PUCT, in the
9  // KataGo formulation (cpp/search/searchexplorehelpers.cpp):
10 //
11 //     a* = argmax_a Q(s,a) + c(s) · P(s,a) · √(W(s) + 0.01) / (1 + W(s,a))
12 //     c(s) = [cpuct + cpuct_log · log((W(s) + cpuct_base) / cpuct_base)] · σ(s)
13 //     σ(s) = 1 + stdev_scale · (stdev(s) / stdev_prior - 1)
14 //
15 // W(s) = Σ_a W(s,a) is the total weight of the *edges* leaving s (one per
16 // visit, plus one per in-flight virtual loss) and stdev(s) is the running
17 // standard deviation of the values backed up through s, blended with a prior.
18 // The σ^factor explores more where the children disagree and less where the
19 // node's value has settled. AlphaZero is the special case
20 // cpuct_log = stdev_scale = 0.
21 //
22 // GRAPH SEARCH —————
23 // The search space is a DAG, not a tree. Every Yohai move turns the square it
24 // leaves into a hole (`empty ^= from`), so the number of holes increases by
25 // exactly one per move and a position can never repeat: no cycles, ever. At
26 // the same time the four stones of a side move independently, so move orders
27 // commute and the same position is reached along very many paths.
28 //
29 // The search therefore stores ONE node per distinct position, in a shared
30 // hash table keyed by the Zobrist hash (plus the score split, which the hash
31 // does not cover – see node_key in the .cpp). A node reached through k
32 // different parents accumulates the statistics of all k of them: a line
33 // refuted in one branch is refuted in every branch that transposes into it.
34 //
35 // • Node – one per position: visit count, ΣV, ΣV², the network's value,
36 //           and the edges to its successors.
37 // • Edge – one per (position, move): prior, pointer to the successor node
38 //           (created on first descent), and this parent's own visit count.
39 //
40 // PUCT uses the EDGE visits in the denominator (this parent's share of the
41 // exploration) and the CHILD NODE's average for Q (everything the search
42 // knows about that position, whoever explored it). That is exactly KataGo's
43 // childWeight = rawChildWeight · edgeVisits / childVisits, which for a search
44 // weighting every visit by 1 collapses to "edge visits".
45 //
46 // Search loop (per thread, repeated until the time / simulation budget is spent):
47 // 1. GATHER up to `batch_size` leaves. Each descent applies a *virtual loss*
48 //    (a pending visit that counts as a loss) to every node and edge on its
49 //    path so that the following descents – of this thread and of the others
50 //    – are steered towards different leaves. A descent that reaches a leaf
51 //    some other thread is already expanding is a "collision"; it is undone
52 //    and, after `max_collisions` of them, the batch is closed early.
53 //    Terminal leaves are scored exactly and backed up at once.
54 // 2. EVALUATE the batch with one call to nn::Evaluator (a GPU wants a few
55 //    hundred positions per call; the CPU backend a few dozen).
56 // 3. EXPAND every leaf with the network's priors (softmax over the legal
57 //    moves only, optional temperature) and BACK UP its value along every
58 //    path that reached it, flipping the sign at each ply (negamax
59 //    convention), converting the pending visits into real ones.
60 //
61 // Values are stored per node as W = Σ v from the point of view of the player
62 // who moved INTO the node, i.e. Q(s,a) = W(child)/N(child) is exactly the
63 // quantity PUCT needs at the parent. The network's value is for the player to

```

```

64 // move at the evaluated position, so the leaf itself receives -v.
65 //
66 // Network cache: before asking the network for a leaf, the search looks the
67 // position up in an NNCache keyed by its Zobrist hash (player/nn_cache.h). It
68 // is a second line of defence behind the node table – it also catches
69 // positions that have dropped out of the graph, and it survives across moves
70 // and games.
71 //
72 // Tree reuse: search(state) looks `state` up in the node table and, if it is
73 // there, re-roots on it – keeping every statistic gathered so far, however
74 // deep in the old graph the position was. The nodes that are no longer
75 // reachable from the new root are then collected (mark and sweep).
76 //
77 // ROOT REFINEMENTS —————
78 // Three things happen at the root that do not happen anywhere else:
79 // • FORCED PLAYOUTS (KataGo paper §5.1, self-play only): a child that has
80 //   been visited at least once is forced up to  $\sqrt{k \cdot P(s,a) \cdot W(s)}$  visits, so
81 //   that a move the noise made interesting is actually tried a few times
82 //   instead of being dropped after one bad evaluation.
83 // • POLICY TARGET PRUNING (same section): those forced visits are taken
84 //   back out afterwards – each non-best child is reduced to the visit count
85 //   that PUCT would retrospectively have given it – so the training target
86 //   is not polluted by playouts the search was forced to make.
87 // • LCB MOVE SELECTION: the move played is the one with the best lower
88 //   confidence bound  $Q - \lambda \cdot \sigma(Q)$ , not simply the most visited, among the
89 //   children with enough visits for the bound to mean anything.
90 //
91 // Self-play: everything the driver needs is on SearchResult – the play-value
92 // distribution  $\pi$  over the root's moves (policy target), the root value, and
93 // move sampling with a temperature. Dirichlet noise on the root priors and
94 // PLAYOUT CAP RANDOMIZATION (a large budget with noise and a recorded policy
95 // target on a fraction of the moves, a small one without on the rest) are
96 // parameters. A new network is exported with nnue/resnet_export.py and
97 // swapped in through nn::Evaluator::reload(). See
98 // player/alphazero_mcts_player.h for the JSON keys.
99 #include "game.h"
100 #include "misc.h"
101 #include "nn_evaluator.h"
102 #include "nn_cache.h"
103 #include <atomic>
104 #include <chrono>
105 #include <cstdint>
106 #include <memory_resource>
107 #include <mutex>
108 #include <string>
109 #include <unordered_map>
110 #include <vector>
111
112 namespace az {
113
114 struct SearchParams {
115     // — budget: the search stops when either bound is hit (0 = no bound) —
116     uint64_t microseconds = 1'000'000; // wall-clock budget
117     uint64_t nb_simulations = 0; // e.g. 800 for AlphaZero-style self-play
118     // — parallelism —
119     size_t nb_threads = 1; // search threads sharing the graph
120     size_t batch_size = 32; // leaves gathered per thread per network call
121     size_t max_collisions = 0; // cross-thread collisions closing a batch (0 =
122     //   ⇨ batch_size)
123     size_t nn_cache_mb = 64; // transposition cache of network outputs (0 = off)
124
125     // — PUCT (KataGo defaults, cpp/search/searchparams.cpp) —
126     float cpuct_exploration = 1.0f; // c

```

```

126 float    cpuct_exploration_log = 0.45f;    // c_log (0 = AlphaZero: no log term)
127 float    cpuct_exploration_base = 500.0f; // c_base
128 // Exploration is scaled by  $\sigma^2 = 1 + \text{scale} \cdot (\text{stdev} / \text{prior} - 1)$ , where stdev
129 // is the standard deviation of the values backed up through the node,
130 // regularised towards `prior` with weight `prior_weight` visits. Scale 0
131 // disables it ( $\sigma^2 = 1$ ) and gives the plain AlphaZero/LC0 behaviour.
132 float    cpuct_utility_stdev_scale      = 0.85f;
133 float    cpuct_utility_stdev_prior      = 0.40f;
134 float    cpuct_utility_stdev_prior_weight = 2.0f;
135
136 // — First play urgency —
137 // An unvisited child is worth
138 //     baseline - fpu_reduction ·  $\sqrt{(\text{prior mass of the visited siblings})}$ 
139 // instead of 0, so that the search does not spread over every child of a
140 // clearly bad position. `baseline` is the value of the parent for the
141 // player to move there: its running average Q blended with its raw network
142 // value, the average weighing (visited prior mass)pow - an unexplored
143 // node trusts the network, a well explored one its own statistics.
144 // fpu_loss_prop finally drags the estimate a proportion of the way towards
145 // a loss (-1). The root uses its own, gentler, constants.
146 float    fpu_reduction      = 0.2f;
147 float    root_fpu_reduction = 0.1f;
148 float    fpu_loss_prop      = 0.0f;
149 float    root_fpu_loss_prop = 0.0f;
150 bool     fpu_parent_weight_by_visited_policy = true;
151 float    fpu_parent_weight_by_visited_policy_pow = 1.0f;
152 // Used only when the above is false: a constant weight for the network
153 // value in the blend (0 = use the running average alone).
154 float    fpu_parent_weight = 0.0f;
155
156 // Softmax temperature applied to the policy logits at expansion (>1
157 // flattens the priors; LC0 uses ~1.36 for its nets).
158 float    policy_temperature = 1.0f;
159
160 // — graph search —
161 // Share one node between every path that reaches the same position. Off
162 // reverts to a plain tree (each path gets its own node), which is only
163 // useful for measuring what the sharing is worth.
164 bool     graph_search      = true;
165
166 // — root: move choice —
167 // Play the move with the best lower confidence bound Q - lcb_stdevs· $\sigma(Q)$ 
168 // among the children holding at least min_visit_prop_for_lcb of the most
169 // visited child's visits, instead of the most visited move.
170 bool     use_lcb           = true;
171 float    lcb_stdevs        = 5.0f;
172 float    min_visit_prop_for_lcb = 0.20f;
173 // Take the forced playouts back out of the root's play values, so that the
174 // policy target reflects what PUCT would have wanted (KataGo always does
175 // this at the root; it is what makes forced playouts safe to train on).
176 bool     policy_target_pruning = true;
177
178 // — self-play exploration —
179 float    dirichlet_alpha    = 0.3f;    // Dir( $\alpha$ ) noise mixed into the root priors
180 float    dirichlet_epsilon  = 0.0f;    // 0 = no noise (competitive play)
181 float    temperature        = 0.0f;    // move sampling  $\propto \pi^{(1/\tau)}$ ; 0 = argmax
182 int      temperature_cutoff = 0;        // sample only while ply < cutoff (0 = never sample)
183 // Force every root child that has been visited at least once up to
184 //  $\sqrt{(k \cdot P \cdot W)}$  visits. 0 = off; the paper uses k = 2 during self-play only.
185 float    forced_playouts_k  = 0.0f;
186 // Playout cap randomization: with probability `fast_prob` the search runs
187 // `nb_simulations_fast` simulations with no root noise, no forced playouts
188 // and SearchResult::full_search = false - the driver must not record a

```

```

189 // policy target for those moves. Needs nb_simulations > 0. 0 = off.
190 float   playout_cap_fast_prob = 0.0f;
191 uint64_t nb_simulations_fast = 0;
192
193 // — misc —
194 bool     reuse_tree          = true;
195 uint64_t seed                = 0;          // 0 = random_device
196 };
197
198 struct ChildStat {
199     Move move;
200     float prior;          // network prior P(s,a) (after noise if any)
201     uint32_t visits;      // edge visits N(s,a), before any pruning
202     float q;              // Q(s,a) for the side to move at the root,  $\in [-1, 1]$ 
203     float q2;             // mean of  $Q^2$  over the child's visits (for the LCB variance)
204     float play_value;     // visits after policy target pruning / LCB adjustment
205     float lcb;            //  $Q(s,a) - lcb\_stdevs \cdot \sigma(Q)$ , or -2 if not computed
206 };
207
208 struct SearchResult {
209     Move best_move = Move::none();
210     float root_value = 0.0f;          // Q of the root for the side to move (from visits)
211     float net_value = 0.0f;          // raw network value of the root position
212     uint64_t nb_simulations = 0;      // simulations run in this call (reused ones excluded)
213     uint32_t root_visits = 0;        // total visits of the root (including reused)
214     uint64_t nb_collisions = 0;      // cross-thread collisions (undone descents)
215     uint64_t nb_cache_hits = 0;      // leaves expanded from the network cache
216     uint64_t nb_evaluations = 0;     // positions sent to the network
217     uint64_t nb_transpositions = 0;  // edges that linked to an existing node
218     bool full_search = true;         // false = a fast playout-cap search: no policy target
219     double seconds = 0.0;
220     size_t tree_nodes = 0;           // distinct positions currently in the graph
221     std::vector<ChildStat> children; // sorted by play value, descending
222     // Training target  $\pi(a)$ : the root's play values, normalised. With
223     // policy_target_pruning these are the visit counts with the forced
224     // playouts taken back out.
225     std::vector<float> policy;
226     std::string to_string(size_t max_children = 8) const;
227 };
228
229 class Search {
230 public:
231     Search(nn::Evaluator& evaluator, const SearchParams& params, std::pmr::memory_resource*
232     ↪ memory);
233     ~Search();
234     Search(const Search&) = delete;
235     Search& operator=(const Search&) = delete;
236
237     // Run a search from `state` (must not be game over) and pick a move.
238     SearchResult search(const Yolah& state);
239     // Forget the graph (call at the start of a new game). The network cache is
240     // kept: it only depends on the weights.
241     void reset();
242     // Drop the cached network outputs (after loading new weights).
243     void clear_cache();
244
245     SearchParams& params() { return prm; }
246     const SearchParams& params() const { return prm; }
247     size_t nb_nodes() const { return node_count.load(std::memory_order_relaxed); }
248 private:
249     struct Node;
250     struct Edge;

```

```

251 struct Worker; // per-thread state (buffers, PRNG)
252 struct PathStep; // one ply of a descent: the node reached and the edge taken
253
254 enum : uint8_t { UNEXPANDED = 0, EXPANDING = 1, EXPANDED = 2, TERMINAL = 3 };
255
256 // — node table (one entry per distinct position) —
257 static constexpr size_t NB_SHARDS = 64;
258 struct Shard {
259     std::mutex mutex;
260     std::unordered_map<uint64_t, Node*> nodes;
261 };
262 Node* get_or_create(uint64_t key, Worker* w);
263 Node* lookup(uint64_t key);
264 Node* new_node();
265 void destroy_all();
266 void collect(Node* keep); // mark and sweep from the new root
267 void mark(Node* node);
268
269 // graph management
270 bool reuse(const Yolah& state, uint64_t key);
271 void new_root(const Yolah& state, uint64_t key);
272 void expand_root(const Yolah& state);
273 void apply_root_noise(Worker& w);
274 // search
275 void run_worker(Worker& w, const Yolah& root_state);
276 uint32_t select_child(const Node& node, bool is_root) const;
277 float utility_stddev_factor(const Node& node, uint32_t visits) const;
278 float fpu_value(const Node& node, uint32_t visits, float visited_policy_mass, bool is_root)
    ↪ const;
279 void priors_from_logits(const nn::Result& result, size_t nb_moves, float* priors) const;
280 void expand(Node& node, const Yolah::MoveList& moves, const float* priors, float value);
281 void backup(const PathStep* path, size_t length, float leaf_value);
282 void undo_pending(const PathStep* path, size_t length);
283 static float terminal_value(const Yolah& state);
284 // result
285 SearchResult make_result(const Yolah& state, double seconds, uint64_t sims,
286     uint64_t collisions, Worker& w) const;
287 void root_play_values(std::vector<ChildStat&> children) const;
288
289 nn::Evaluator& evaluator;
290 SearchParams prm;
291 std::pmr::polymorphic_allocator<> alloc;
292 Shard shards[NB_SHARDS];
293 Node* root = nullptr;
294 Yolah root_state; // position at the root (valid when root != nullptr)
295 uint64_t root_key = 0; // its node-table key
296 std::vector<float> root_priors; // root priors after Dirichlet noise (empty = none)
297 uint32_t mark_counter = 0;
298 std::atomic<uint64_t> node_count{0};
299 std::atomic<uint64_t> simulations{0}; // simulations completed in the current search
300 std::atomic<uint64_t> reserved{0}; // simulations claimed by the workers (budget
    ↪ accounting)
301 std::atomic<uint64_t> collisions{0};
302 std::atomic<uint64_t> cache_hits{0};
303 std::atomic<uint64_t> evaluations{0};
304 std::atomic<uint64_t> transpositions{0};
305 std::unique_ptr<NNCache> cache;
306 std::atomic<bool> stop{false};
307 std::chrono::steady_clock::time_point deadline;
308 uint64_t sim_budget = 0; // simulations for the current search (playout cap)
309 bool full_search = true; // false = fast search: no noise, no forced playouts
310 PRNG rng; // search-level PRNG (playout cap randomization)
311 uint64_t base_seed;

```

```

312     uint64_t search_counter = 0;
313 };
314
315 } // namespace az
316
317 #endif

```

11.1.2 player/alphazero_mcts.cpp

857 lines.

The implementation, in the order the search uses it: the graph representation, the node table and its garbage collection, re-rooting, the selection rule, the per-thread worker loop, and finally the root post-processing that turns visit counts into a move and a training target.

Two invariants hold everywhere and explain most of the code:

- a node's w is the sum of values **for the player who moved into it**, so w/n read from the parent is already the parent's $Q(s, a)$ and the sign flips once per ply on the way up;
- **state** is the only atomic with ordering that matters. **EXPANDED** is stored with release and read with acquire, which is what makes the edge vector safe to read without a lock; every statistic is relaxed, because a stale read only changes which leaf a descent picks.

symbol	role
Search::Edge	one per (position, move): prior, successor pointer, this parent's visit and pending counts
Search::Node	one per position: n , pending , w , w2 , state , the GC mark, the network value, the edges
Search::PathStep	one ply of a descent: the node reached and the edge taken to reach it
Search::Worker	per-thread buffers: PRNG, the batch's leaves, its descents, the path arena, the request/result spans
same_position	full position equality, used by the tree-mode re-rooting scan
mix	splitmix64 finaliser, used to fold the score into the node key
node_key	Zobrist hash $\oplus$ mixed black score — see Section 5.6.1 for why the score is needed
Search::terminal_value	the exact result of a finished game, for the player to move
Search::new_node	allocate one node from the pool resource and count it
Search::lookup	find a position in the node table (used by re-rooting)
Search::destroy_all	destroy every node; the table is the ownership list
Search::mark / collect	mark and sweep from the new root; safe as a plain recursion because the graph is acyclic
Search::get_or_create	table lookup or insert, under the shard lock
Search::new_root	start a fresh graph at a position

symbol	role
<code>Search::reuse</code>	re-root by table lookup (graph mode) or by the two-ply scan (tree mode), then collect
<code>Search::expand_root</code>	the one unbatched network call, for the root itself
<code>Search::apply_root_noise</code>	mix Dirichlet noise into a <i>copy</i> of the root priors, leaving the graph's own priors alone
<code>Search::utility_stdev_factor</code>	$\hat{\sigma}(s)$ from the node's <code>w</code> and <code>w2</code> — Section 5.2
<code>Search::fpu_value</code>	what an unvisited child is assumed to be worth — Section 5.3
<code>Search::select_child</code>	one PUCT argmax, plus the forced-playout override at the root
<code>Search::priors_from_logits</code>	softmax over the legal moves only, with policy_temperature
<code>Search::expand</code>	build the edge vector, store the network value, publish EXPANDED
<code>Search::backup</code>	negamax accumulation into every node and edge of one path
<code>Search::undo_pending</code>	roll the virtual loss back on an abandoned descent
<code>Search::run_worker</code>	gather → evaluate → expand → back up, until the budget is spent
<code>Search::root_play_values</code>	policy target pruning and the LCB adjustment — Section 5.7
<code>Search::make_result</code>	assemble the statistics, π and the move choice

```

1 // alphazero_mcts.cpp – see alphazero_mcts.h for the algorithm overview.
2 #include "alphazero_mcts.h"
3 #include "misc.h"
4 #include "zobrist.h"
5 #include <algorithm>
6 #include <cmath>
7 #include <iomanip>
8 #include <limits>
9 #include <random>
10 #include <sstream>
11 #include <thread>
12
13 namespace az {
14
15 // — Graph node and edge —————
16 // One Node per distinct position, one Edge per (position, move). All
17 // statistics are atomics updated with relaxed ordering: they are heuristics, a
18 // slightly stale read only changes which leaf a descent picks, never the
19 // graph's integrity. The only synchronisation that matters is `state`
20 // (acquire/release) – a thread that observes EXPANDED is guaranteed to see the
21 // fully built edge vector – and Edge::child, published the same way.
22 struct Search::Edge {
23     Move action = Move::none(); // move leading to the successor
24     float prior = 0.0f; // P(s,a) from the policy head
25     std::atomic<Search::Node*> child{nullptr}; // successor, created on first descent
26     std::atomic<uint32_t> n{0}; // edge visits: this parent's share of the child
27     std::atomic<uint32_t> pending{0}; // in-flight visits through this edge
28
29     Edge() = default;
30     Edge(Move m, float p) : action(m), prior(p) {}

```

```

31 // Atomics are not movable; the vector needs a move constructor anyway.
32 Edge(const Edge& o)
33     : action(o.action), prior(o.prior), child(o.child.load()),
34       n(o.n.load()), pending(o.pending.load()) {}
35 Edge& operator=(const Edge& o) {
36     action = o.action; prior = o.prior;
37     child = o.child.load(); n = o.n.load(); pending = o.pending.load();
38     return *this;
39 }
40 };
41
42 struct Search::Node {
43     std::atomic<uint32_t> n{0}; // completed visits N(s), from every parent
44     std::atomic<uint32_t> pending{0}; // in-flight visits (virtual loss), each counted as a loss
45     std::atomic<float> w{0.0f}; //  $\Sigma$  backed-up values for the player who moved INTO this
46     ↪ node
47     std::atomic<float> w2{0.0f}; //  $\Sigma$  of their squares (sign-free: stdev of the subgraph's
48     ↪ values)
49     std::atomic<uint8_t> state{UNEXPANDED};
50     uint32_t mark = 0; // garbage collection: last mark_counter that reached it
51     float net_v = 0.0f; // network value at this node (player to move here)
52     std::pmr::vector<Edge> children;
53
54     using allocator_type = std::pmr::polymorphic_allocator<>;
55     explicit Node(allocator_type a = {}) : children(a) {}
56 };
57
58 // One ply of a descent: the node reached and the edge that was taken to reach
59 // it (null for the root, which no edge leads to).
60 struct Search::PathStep {
61     Node* node;
62     Edge* edge;
63 };
64
65 // — Per-thread state —————
66 struct Search::Worker {
67     static constexpr size_t MAX_PATH = Yolah::MAX_NB_PLIES + 8; // root → deepest leaf
68     struct Leaf {
69         Node* node;
70         uint64_t hash; // Zobrist key, for the network cache
71         Yolah state;
72         Yolah::MoveList moves;
73     };
74     // A descent that reached `leaf`: the value evaluated there is backed up
75     // along this path. Two descents of the same batch can reach the same leaf
76     // through different paths – under graph search even through different
77     // parents – so every descent keeps its own path.
78     struct Descent {
79         uint32_t leaf;
80         uint32_t path_len;
81     };
82     PRNG prng;
83     std::vector<Leaf> leaves; // distinct leaves to evaluate
84     std::vector<Descent> descents; // one per gathered visit
85     std::vector<PathStep> paths; // (batch_size + 1) × MAX_PATH steps
86     std::vector<nn::Request> requests;
87     std::vector<nn::Result> results;
88
89     Worker(uint64_t seed, size_t batch_size)
90         : prng(seed ? seed : 0x9E3779B97F4A7C15ull),
91           paths((batch_size + 1) * MAX_PATH, PathStep{nullptr, nullptr}),
92           results(batch_size) {
93         leaves.reserve(batch_size);
94     }
95 };

```

```

92     descents.reserve(batch_size);
93     requests.reserve(batch_size);
94 }
95 PathStep* path(size_t k) { return paths.data() + k * MAX_PATH; }
96 };
97
98 // — helpers —————
99 namespace {
100     bool same_position(const Yolah& a, const Yolah& b) {
101         return a.nb_plies() == b.nb_plies() &&
102             a.bitboard(Yolah::BLACK) == b.bitboard(Yolah::BLACK) &&
103             a.bitboard(Yolah::WHITE) == b.bitboard(Yolah::WHITE) &&
104             a.empty_bitboard() == b.empty_bitboard() &&
105             a.score() == b.score();
106     }
107
108     uint64_t mix(uint64_t x) {
109         x ^= x >> 30; x *= 0xBF58476D1CE4E5B9ull;
110         x ^= x >> 27; x *= 0x94D049BB133111EBull;
111         return x ^ (x >> 31);
112     }
113
114     // Key of a position in the node table. zobrist::hash covers the board and
115     // the side to move – exactly what the network sees, which is why the
116     // network cache can key on it alone. It does NOT cover how the points
117     // scored so far split between the two players: two positions with the same
118     // board can differ there (a pass scores nothing), and the terminal value
119     // depends on the split. Nodes carry statistics, not just network outputs,
120     // so their key has to separate them.
121     uint64_t node_key(uint64_t zobrist_hash, const Yolah& state) {
122         return zobrist_hash ^ mix(uint64_t(state.score().first) + 1);
123     }
124 }
125
126 float Search::terminal_value(const Yolah& state) {
127     const auto [black, white] = state.score();
128     const int diff = (state.current_player() == Yolah::BLACK ? 1 : -1) * (int(black) -
129 ↪ int(white));
130     return diff > 0 ? 1.0f : (diff < 0 ? -1.0f : 0.0f);
131 }
132
133 // — construction / node table —————
134 Search::Search(nn::Evaluator& evaluator, const SearchParams& params, std::pmr::memory_resource*
135 ↪ memory)
136     : evaluator(evaluator), prm(params), alloc(memory),
137     rng(params.seed ? params.seed : 0x2545F4914F6CDD1Dull),
138     base_seed(params.seed ? params.seed : std::random_device{}()) {
139     if (prm.microseconds == 0 && prm.nb_simulations == 0)
140         throw std::invalid_argument("az::Search: either microseconds or nb_simulations must be >
141 ↪ 0");
142     prm.nb_threads = std::max<size_t>(1, prm.nb_threads);
143     prm.batch_size = std::max<size_t>(1, prm.batch_size);
144     cache = std::make_unique<NNCache>(prm.nn_cache_mb << 20);
145 }
146
147 Search::~Search() {
148     destroy_all();
149 }
150
151 void Search::clear_cache() {
152     cache->clear();
153 }

```

```

152 Search::Node* Search::new_node() {
153     node_count.fetch_add(1, std::memory_order_relaxed);
154     return alloc.new_object<Node>();
155 }
156
157 Search::Node* Search::lookup(uint64_t key) {
158     Shard& sh = shards[key % NB_SHARDS];
159     std::lock_guard lock(sh.mutex);
160     auto it = sh.nodes.find(key);
161     return it == sh.nodes.end() ? nullptr : it->second;
162 }
163
164 void Search::destroy_all() {
165     for (Shard& sh : shards) {
166         for (auto& [key, node] : sh.nodes) alloc.delete_object(node);
167         sh.nodes.clear();
168     }
169     root = nullptr;
170     node_count = 0;
171 }
172
173 void Search::reset() {
174     destroy_all();
175     root_priors.clear();
176 }
177
178 // Depth-first marking of everything still reachable from the new root. The
179 // position graph is acyclic (every move fills a square, so the number of holes
180 // strictly increases), hence a plain recursion with a "already marked" test
181 // terminates and visits each node once.
182 void Search::mark(Node* node) {
183     if (!node || node->mark == mark_counter) return;
184     node->mark = mark_counter;
185     if (node->state.load(std::memory_order_relaxed) != EXPANDED) return;
186     for (Edge& e : node->children) mark(e.child.load(std::memory_order_relaxed));
187 }
188
189 void Search::collect(Node* keep) {
190     ++mark_counter;
191     mark(keep);
192     uint64_t freed = 0;
193     for (Shard& sh : shards) {
194         for (auto it = sh.nodes.begin(); it != sh.nodes.end(); ) {
195             if (it->second->mark != mark_counter) {
196                 alloc.delete_object(it->second);
197                 it = sh.nodes.erase(it);
198                 ++freed;
199             } else {
200                 ++it;
201             }
202         }
203     }
204     node_count.fetch_sub(freed, std::memory_order_relaxed);
205 }
206
207 // Find (or create) the node for `key`. Under graph search the key identifies
208 // the position, so every path reaching it gets the same node; in tree mode the
209 // caller passes a fresh unique key and every edge gets its own node.
210 Search::Node* Search::get_or_create(uint64_t key, Worker* w) {
211     Shard& sh = shards[key % NB_SHARDS];
212     std::lock_guard lock(sh.mutex);
213     auto [it, inserted] = sh.nodes.try_emplace(key, nullptr);
214     if (!inserted) {

```

```

215         transpositions.fetch_add(1, std::memory_order_relaxed);
216         return it->second;
217     }
218     (void)w;
219     it->second = new_node();
220     return it->second;
221 }
222
223 void Search::new_root(const Yolah& state, uint64_t key) {
224     destroy_all();
225     root = get_or_create(key, nullptr);
226     root_state = state;
227     root_key = key;
228 }
229
230 bool Search::reuse(const Yolah& state, uint64_t key) {
231     if (!root) return false;
232     if (prm.graph_search) {
233         // The node table is the whole tree-reuse mechanism: whatever depth the
234         // position sat at in the old graph, its statistics are right there.
235         Node* node = lookup(key);
236         if (!node) return false;
237         root = node;
238         root_state = state;
239         root_key = key;
240         collect(root); // drop everything the new root cannot reach
241         return true;
242     }
243     // Tree mode: the position can only be the root, one of its children or one
244     // of its grandchildren.
245     if (same_position(state, root_state)) return true;
246     if (root->state.load(std::memory_order_acquire) != EXPANDED) return false;
247     for (Edge& e : root->children) {
248         Node* child = e.child.load(std::memory_order_acquire);
249         if (!child) continue;
250         Yolah s1 = root_state;
251         s1.play(e.action);
252         if (same_position(s1, state)) {
253             root = child; root_state = state; root_key = key; collect(root);
254             return true;
255         }
256         if (child->state.load(std::memory_order_acquire) != EXPANDED) continue;
257         for (Edge& g : child->children) {
258             Node* grandchild = g.child.load(std::memory_order_acquire);
259             if (!grandchild) continue;
260             Yolah s2 = s1;
261             s2.play(g.action);
262             if (same_position(s2, state)) {
263                 root = grandchild; root_state = state; root_key = key; collect(root);
264                 return true;
265             }
266         }
267     }
268     return false;
269 }
270
271 void Search::expand_root(const Yolah& state) {
272     if (state.game_over()) {
273         root->state.store(TERMINAL, std::memory_order_release);
274         return;
275     }
276     Yolah::MoveList moves;
277     state.moves(moves);

```

```

278     nn::Request request{&state, moves.begin(), static_cast<uint16_t>(moves.size())};
279     nn::Result result;
280     evaluator.evaluate(std::span<const nn::Request>(&request, 1), std::span<nn::Result>(&result,
↪ 1));
281     root->state.store(EXPANDING, std::memory_order_relaxed);
282     float priors[Yolah::MAX_NB_MOVES];
283     priors_from_logits(result, moves.size(), priors);
284     expand(*root, moves, priors, result.value);
285 }
286
287 void Search::apply_root_noise(Worker& w) {
288     root_priors.clear();
289     if (!full_search || prm.dirichlet_epsilon <= 0.0f || root->children.empty()) return;
290     std::mt19937_64 gen(w.prng.rand<uint64_t>());
291     std::gamma_distribution<float> gamma(prm.dirichlet_alpha, 1.0f);
292     const size_t nb = root->children.size();
293     root_priors.resize(nb);
294     float sum = 0.0f;
295     for (size_t i = 0; i < nb; i++) { root_priors[i] = gamma(gen); sum += root_priors[i]; }
296     for (size_t i = 0; i < nb; i++) {
297         const float noise = sum > 0.0f ? root_priors[i] / sum : 1.0f / nb;
298         root_priors[i] = (1.0f - prm.dirichlet_epsilon) * root->children[i].prior +
↪ prm.dirichlet_epsilon * noise;
299     }
300 }
301
302 // — search primitives —————
303 // KataGo's σ-factor: the exploration constant is scaled by how much the values
304 // backed up through this node disagree, measured against a prior stdev.
305 float Search::utility_stdev_factor(const Node& node, uint32_t visits) const {
306     if (prm.cpuct_utility_stdev_scale <= 0.0f) return 1.0f;
307     const float prior = prm.cpuct_utility_stdev_prior;
308     float stdev = prior;
309     if (visits > 1) {
310         const float n = static_cast<float>(visits);
311         // The sign of the mean is irrelevant here, only its square is used.
312         const float mean = node.w.load(std::memory_order_relaxed) / n;
313         const float mean_sq = mean * mean;
314         // Relaxed reads of w and w2 can be slightly out of step; clamp so that
315         // the variance never comes out negative.
316         const float sq_avg = std::max(node.w2.load(std::memory_order_relaxed) / n, mean_sq);
317         const float pw = prm.cpuct_utility_stdev_prior_weight;
318         stdev = std::sqrt(std::max(0.0f,
319             ((mean_sq + prior * prior) * pw + sq_avg * n) / (pw + n - 1.0f) - mean_sq));
320     }
321     return 1.0f + prm.cpuct_utility_stdev_scale * (stdev / prior - 1.0f);
322 }
323
324 // Value assumed for a child that has not been visited yet (first play urgency).
325 float Search::fpu_value(const Node& node, uint32_t visits, float visited_policy_mass, bool
↪ is_root) const {
326     // Everything here is from the point of view of the player to move at
327     // `node`: node.w is for the player who moved into it, hence the sign flip,
328     // and node.net_v is the network's value for the player to move.
329     const float parent_q = visits > 0 ? -node.w.load(std::memory_order_relaxed) / visits :
↪ node.net_v;
330     float baseline = parent_q;
331     if (prm.fpu_parent_weight_by_visited_policy) {
332         const float p = prm.fpu_parent_weight_by_visited_policy_pow;
333         const float mass = p == 1.0f ? visited_policy_mass : std::pow(visited_policy_mass, p);
334         const float avg_weight = std::min(1.0f, mass);
335         baseline = avg_weight * parent_q + (1.0f - avg_weight) * node.net_v;
336     } else if (prm.fpu_parent_weight > 0.0f) {

```

```

337     baseline = prm.fpu_parent_weight * node.net_v + (1.0f - prm.fpu_parent_weight) * parent_q;
338 }
339 const float reduction = (is_root ? prm.root_fpu_reduction : prm.fpu_reduction) *
    ↪ std::sqrt(visited_policy_mass);
340 const float loss_prop = is_root ? prm.root_fpu_loss_prop : prm.fpu_loss_prop;
341 const float fpu = baseline - reduction;
342 return fpu + (-1.0f - fpu) * loss_prop;    // values live in [-1, 1]: a loss is -1
343 }
344
345 uint32_t Search::select_child(const Node& node, bool is_root) const {
346     const auto& children = node.children;
347     const uint32_t nb = static_cast<uint32_t>(children.size());
348     const uint32_t n_parent = node.n.load(std::memory_order_relaxed);
349     const float* priors = (is_root && !root_priors.empty()) ? root_priors.data() : nullptr;
350
351     // Total weight of the edges leaving this node (KataGo's totalChildWeight -
352     // the node's own visit is not part of it) and prior mass of the moves that
353     // have been tried at least once.
354     float total_child_weight = 0.0f, visited_mass = 0.0f;
355     for (uint32_t i = 0; i < nb; i++) {
356         const Edge& e = children[i];
357         const uint32_t w = e.n.load(std::memory_order_relaxed) +
    ↪ e.pending.load(std::memory_order_relaxed);
358         total_child_weight += static_cast<float>(w);
359         if (w > 0) visited_mass += priors ? priors[i] : e.prior;
360     }
361
362     const float cpuct = prm.cpuct_exploration + prm.cpuct_exploration_log *
363         std::log((total_child_weight + prm.cpuct_exploration_base) / prm.cpuct_exploration_base);
364     // The  $\sqrt{W}$  of the PUCT formula, with KataGo's offset keeping it positive at  $W = 0$ .
365     const float explore_scaling = cpuct * utility_stdev_factor(node, n_parent)
366         * std::sqrt(total_child_weight + 0.01f);
367     const float fpu = fpu_value(node, n_parent, visited_mass, is_root);
368     // Forced playouts (self-play only): a move that has been tried once is
369     // pushed up to  $\sqrt{(k \cdot P \cdot W)}$  visits before PUCT is allowed to drop it.
370     const bool forced = is_root && full_search && prm.forced_playouts_k > 0.0f;
371     const float forced_scale = forced ? prm.forced_playouts_k * total_child_weight : 0.0f;
372
373     uint32_t best = 0;
374     float best_score = -std::numeric_limits<float>::infinity();
375     for (uint32_t i = 0; i < nb; i++) {
376         const Edge& e = children[i];
377         const uint32_t n = e.n.load(std::memory_order_relaxed);
378         const uint32_t pending = e.pending.load(std::memory_order_relaxed);
379         const uint32_t n_eff = n + pending;
380         const float p = priors ? priors[i] : e.prior;
381         // Q comes from the child NODE - everything the search knows about that
382         // position, whichever parent paid for it - while the PUCT denominator
383         // is this EDGE's own visit count.
384         float q = fpu;
385         const Node* child = e.child.load(std::memory_order_acquire);
386         if (child) {
387             const uint32_t cn = child->n.load(std::memory_order_relaxed);
388             const uint32_t cp = child->pending.load(std::memory_order_relaxed);
389             // Every pending visit is counted as a loss (-1): the virtual loss.
390             if (cn + cp > 0)
391                 q = (child->w.load(std::memory_order_relaxed) - static_cast<float>(cp)) / (cn +
    ↪ cp);
392         }
393         float score = q + explore_scaling * p / (1.0f + n_eff);
394         if (forced && n_eff > 0 && static_cast<float>(n_eff) < std::sqrt(forced_scale * p))
395             score += 1e6f;
396         if (score > best_score) { best_score = score; best = i; }

```

```

397     }
398     return best;
399 }
400
401 void Search::priors_from_logits(const nn::Result& result, size_t nb_moves, float* priors) const {
402     // Softmax over the legal moves only, with the policy temperature.
403     float max_logit = -std::numeric_limits<float>::infinity();
404     for (size_t m = 0; m < nb_moves; m++) max_logit = std::max(max_logit, result.logits[m]);
405     float sum = 0.0f;
406     const float inv_t = 1.0f / prm.policy_temperature;
407     for (size_t m = 0; m < nb_moves; m++) {
408         priors[m] = std::exp((result.logits[m] - max_logit) * inv_t);
409         sum += priors[m];
410     }
411     for (size_t m = 0; m < nb_moves; m++) priors[m] /= sum;
412 }
413
414 void Search::expand(Node& node, const Yolah::MoveList& moves, const float* priors, float value) {
415     const size_t nb = moves.size();
416     node.children.reserve(nb);
417     for (size_t m = 0; m < nb; m++) node.children.emplace_back(moves[m], priors[m]);
418     node.net_v = value;
419     node.state.store(EXPANDED, std::memory_order_release); // publishes the edges
420 }
421
422 void Search::backup(const PathStep* path, size_t length, float leaf_value) {
423     // The leaf's value is for the player to move at the leaf; a node's W is
424     // for the player who moved into it, hence the initial sign flip, and one
425     // more flip per ply on the way up (negamax).
426     float v = -leaf_value;
427     //  $\sum v^2$  is the same at every ply: the negamax flip disappears in the square.
428     const float v2 = leaf_value * leaf_value;
429     for (size_t i = length; i-- > 0;) {
430         Node* node = path[i].node;
431         node->w.fetch_add(v, std::memory_order_relaxed);
432         node->w2.fetch_add(v2, std::memory_order_relaxed);
433         node->n.fetch_add(1, std::memory_order_relaxed);
434         node->pending.fetch_sub(1, std::memory_order_relaxed);
435         if (Edge* e = path[i].edge) {
436             e->n.fetch_add(1, std::memory_order_relaxed);
437             e->pending.fetch_sub(1, std::memory_order_relaxed);
438         }
439         v = -v;
440     }
441 }
442
443 void Search::undo_pending(const PathStep* path, size_t length) {
444     for (size_t i = 0; i < length; i++) {
445         path[i].node->pending.fetch_sub(1, std::memory_order_relaxed);
446         if (Edge* e = path[i].edge) e->pending.fetch_sub(1, std::memory_order_relaxed);
447     }
448 }
449
450 // — the per-thread search loop —————
451 void Search::run_worker(Worker& w, const Yolah& root_position) {
452     Node* const root_node = root;
453     const uint64_t root_hash = zobrist::hash(root_position);
454     const size_t max_collisions = prm.max_collisions ? prm.max_collisions : prm.batch_size;
455     uint64_t local_collisions = 0, local_hits = 0, local_evals = 0;
456     float priors[Yolah::MAX_NB_MOVES];
457     // Tree mode: every edge gets its own node, under a key nothing else uses.
458     static std::atomic<uint64_t> unique_key{0x8000000000000000ull};
459

```

```

460 while (!stop.load(std::memory_order_relaxed)) {
461     // — 1. gather —
462     w.leaves.clear();
463     w.descents.clear();
464     w.requests.clear();
465     size_t budget = prm.batch_size;
466     if (sim_budget) {
467         // Claim visits from the shared budget so that the threads together
468         // never exceed it (unused claims are returned below).
469         uint64_t claimed = reserved.load(std::memory_order_relaxed);
470         for (;;) {
471             if (claimed >= sim_budget) { budget = 0; break; }
472             const uint64_t want = std::min<uint64_t>(budget, sim_budget - claimed);
473             if (reserved.compare_exchange_weak(claimed, claimed + want,
474                 ↪ std::memory_order_relaxed)) {
475                 budget = want;
476                 break;
477             }
478         }
479         if (budget == 0) {
480             if (simulations.load(std::memory_order_relaxed) >= sim_budget) break;
481             // Other threads still hold claims: wait for their batch instead of spinning.
482             evaluator.evaluate({}, {});
483             std::this_thread::yield();
484             continue;
485         }
486     }
487     size_t batch_collisions = 0, gathered_visits = 0;
488     while (gathered_visits < budget && batch_collisions <= max_collisions) {
489         PathStep* path = w.path(w.descents.size());
490         size_t len = 0;
491         Node* node = root_node;
492         Yolah state = root_position;
493         uint64_t hash = root_hash;
494         node->pending.fetch_add(1, std::memory_order_relaxed);
495         path[len++] = {node, nullptr};
496         for (;;) {
497             uint8_t st = node->state.load(std::memory_order_acquire);
498             if (st == EXPANDED) {
499                 Edge& e = node->children[select_child(*node, node == root_node)];
500                 const uint8_t mover = state.current_player();
501                 state.play(e.action);
502                 hash = zobrist::update(hash, mover, e.action);
503                 e.pending.fetch_add(1, std::memory_order_relaxed);
504                 Node* child = e.child.load(std::memory_order_acquire);
505                 if (!child) {
506                     // Link the edge to its successor, sharing the node with
507                     // every other path that reaches the same position. The
508                     // shard lock serialises the linking of this edge: in
509                     // graph mode it is picked by the position key, in tree
510                     // mode by the edge itself.
511                     const uint64_t key = prm.graph_search ? node_key(hash, state)
512                                                         : unique_key.fetch_add(1,
513                                                         ↪ std::memory_order_relaxed);
514                     Shard& sh = shards[(prm.graph_search ? key : std::hash<const void*>{})(&e))
515 ↪ % NB_SHARDS];
516                     std::unique_lock lock(sh.mutex);
517                     child = e.child.load(std::memory_order_acquire);
518                     if (!child) {
519                         auto [it, inserted] = sh.nodes.try_emplace(key, nullptr);
520                         if (inserted) it->second = new_node();
521                         else
522                             transpositions.fetch_add(1, std::memory_order_relaxed);
523                     }
524                     child = it->second;

```

```

520         e.child.store(child, std::memory_order_release);
521     }
522 }
523 child->pending.fetch_add(1, std::memory_order_relaxed);
524 path[len++] = {child, &e};
525 node = child;
526 continue;
527 }
528 if (st == TERMINAL) {
529     backup(path, len, terminal_value(state));
530     simulations.fetch_add(1, std::memory_order_relaxed);
531     ++gathered_visits;
532     break;
533 }
534 if (st == UNEXPANDED) {
535     uint8_t expected = UNEXPANDED;
536     if (node->state.compare_exchange_strong(expected, EXPANDING,
537         ↪ std::memory_order_acq_rel,
538                                         std::memory_order_acquire)) {
539         ++gathered_visits;
540         if (state.game_over()) {
541             node->state.store(TERMINAL, std::memory_order_release);
542             backup(path, len, terminal_value(state));
543             simulations.fetch_add(1, std::memory_order_relaxed);
544             break;
545         }
546         Worker::Leaf& leaf = w.leaves.emplace_back();
547         leaf.node = node;
548         leaf.state = state;
549         state.moves(leaf.moves);
550         leaf.hash = hash;
551         // Transposition the node table missed (a position that
552         // dropped out of the graph, or tree mode): expand from
553         // the network cache, no network call.
554         float value;
555         if (cache->lookup(leaf.hash, leaf.moves.size(), value, priors)) {
556             expand(*node, leaf.moves, priors, value);
557             backup(path, len, value);
558             simulations.fetch_add(1, std::memory_order_relaxed);
559             ++local_hits;
560             w.leaves.pop_back();
561         } else {
562             w.descents.push_back({static_cast<uint32_t>(w.leaves.size() - 1),
563                 static_cast<uint32_t>(len)});
564         }
565         break;
566     }
567     if (expected != EXPANDING) continue; // became EXPANDED/TERMINAL meanwhile:
568     ↪ go on
569 }
570 // st == EXPANDING: the leaf is being evaluated. If it is ours
571 // (same batch), this descent rides on the same evaluation -
572 // its path is kept separately, it may well differ. Otherwise
573 // the descent is undone.
574 uint32_t mine = UINT32_MAX;
575 for (uint32_t i = 0; i < w.leaves.size(); i++)
576     if (w.leaves[i].node == node) { mine = i; break; }
577 if (mine != UINT32_MAX) {
578     w.descents.push_back({mine, static_cast<uint32_t>(len)});
579     ++gathered_visits;
580 } else {
581     undo_pending(path, len);
582     ++batch_collisions;

```

```

581         }
582         break;
583     }
584 }
585 local_collisions += batch_collisions;
586 if (sim_budget && gathered_visits < budget)
587     reserved.fetch_sub(budget - gathered_visits, std::memory_order_relaxed);
588
589 // — 2. evaluate —
590 const size_t nb = w.leaves.size();
591 if (nb == 0 && gathered_visits == 0) {
592     // Every descent collided with leaves another thread is evaluating:
593     // join the evaluator's barrier (an empty request) so we resume
594     // right after that batch has been expanded, instead of spinning.
595     evaluator.evaluate({}, {});
596     std::this_thread::yield();
597 }
598 if (nb > 0) {
599     for (Worker::Leaf& leaf : w.leaves)
600         w.requests.push_back({&leaf.state, leaf.moves.begin(),
↪ static_cast<uint16_t>(leaf.moves.size())});
601     evaluator.evaluate(w.requests, std::span<nn::Result>(w.results.data(), nb));
602     local_evals += nb;
603     // — 3. expand ... —
604     for (size_t i = 0; i < nb; i++) {
605         Worker::Leaf& leaf = w.leaves[i];
606         const nn::Result& r = w.results[i];
607         priors_from_logits(r, leaf.moves.size(), priors);
608         cache->store(leaf.hash, leaf.moves.size(), r.value, priors);
609         expand(*leaf.node, leaf.moves, priors, r.value);
610     }
611     // — ... and back up, once per descent, along its own path —
612     for (size_t d = 0; d < w.descents.size(); d++) {
613         const Worker::Descent& desc = w.descents[d];
614         backup(w.path(d), desc.path_len, w.results[desc.leaf].value);
615     }
616     simulations.fetch_add(w.descents.size(), std::memory_order_relaxed);
617 }
618
619 if (prm.microseconds && std::chrono::steady_clock::now() >= deadline) stop.store(true,
↪ std::memory_order_relaxed);
620 if (sim_budget && simulations.load(std::memory_order_relaxed) >= sim_budget)
621     stop.store(true, std::memory_order_relaxed);
622 }
623 collisions.fetch_add(local_collisions, std::memory_order_relaxed);
624 cache_hits.fetch_add(local_hits, std::memory_order_relaxed);
625 evaluations.fetch_add(local_evals, std::memory_order_relaxed);
626 }
627
628 // — top level —
629 SearchResult Search::search(const Yolah& state) {
630     using namespace std::chrono;
631     const auto start = steady_clock::now();
632
633     const uint64_t key = node_key(zobrist::hash(state), state);
634     if (!(prm.reuse_tree && reuse(state, key))) new_root(state, key);
635     if (root->state.load(std::memory_order_acquire) == UNEXPANDED) expand_root(state);
636
637     // Playout cap randomization: most self-play moves get a small budget, no
638     // root noise and no policy target; a minority get the full treatment.
639     full_search = true;
640     sim_budget = prm.nb_simulations;
641     if (prm.nb_simulations && prm.nb_simulations_fast && prm.playout_cap_fast_prob > 0.0f) {

```

```

642     const float u = (rng.rand<uint64_t>() >> 11) * 0x1.0p-53f;
643     if (u < prm.playout_cap_fast_prob) {
644         full_search = false;
645         sim_budget = prm.nb_simulations_fast;
646     }
647 }
648
649 std::vector<std::unique_ptr<Worker>> workers;
650 for (size_t t = 0; t < prm.nb_threads; t++) {
651     // Distinct, non-zero seeds per thread and per search.
652     const uint64_t seed = (base_seed + 0x9E3779B97F4A7C15ull * (search_counter *
        ↪ prm.nb_threads + t + 1)) | 1;
653     workers.push_back(std::make_unique<Worker>(seed, prm.batch_size));
654 }
655 ++search_counter;
656
657 simulations = 0;
658 reserved = 0;
659 collisions = 0;
660 cache_hits = 0;
661 evaluations = 0;
662 transpositions = 0;
663 stop = false;
664 if (root->state.load(std::memory_order_acquire) == EXPANDED) {
665     apply_root_noise(*workers[0]);
666     deadline = prm.microseconds ? start + microseconds(prm.microseconds) :
        ↪ steady_clock::time_point::max();
667     if (prm.nb_threads == 1) {
668         run_worker(*workers[0], state);
669     } else {
670         std::vector<std::jthread> threads;
671         for (size_t t = 0; t < prm.nb_threads; t++)
672             threads.emplace_back([this, &workers, t, &state] { run_worker(*workers[t], state);
        ↪ });
673     }
674 }
675 const double seconds = duration<double>(steady_clock::now() - start).count();
676 SearchResult res = make_result(state, seconds, simulations.load(), collisions.load(),
        ↪ *workers[0]);
677 res.root_visits = root->n.load();
678 return res;
679 }
680
681 // Play values of the root's moves: the edge visits, with the forced playouts
682 // taken back out (KataGo's getReducedPlaySelectionWeight) and the LCB bonus
683 // applied. They serve both as the training target  $\pi$  and as the move choice.
684 void Search::root_play_values(std::vector<ChildStat>& children) const {
685     const size_t nb = children.size();
686     if (nb == 0) return;
687
688     float total = 0.0f;
689     for (const ChildStat& c : children) total += static_cast<float>(c.visits);
690     for (ChildStat& c : children) c.play_value = static_cast<float>(c.visits);
691
692     // — 1. the most stably explored child (KataGo's "non LCB best") —
693     size_t best = 0;
694     {
695         float best_goodness = -1e30f;
696         for (size_t i = 0; i < nb; i++) {
697             const float v = static_cast<float>(children[i].visits);
698             // Discount one visit's worth: the most recent one may be an outlier.
699             const float g = v * std::max(0.0f, v - 1.0f) / std::max(1.0f, v) + 2.0f *
                ↪ children[i].prior;

```

```

700         if (g > best_goodness) { best_goodness = g; best = i; }
701     }
702 }
703
704 // — 2. policy target pruning —
705 // How many visits would PUCT have given each child, in retrospect, if it
706 // had never been forced? Invert the selection formula at the best child's
707 // selection value and keep the smaller of the two counts.
708 if (prm.policy_target_pruning && total > 0.0f) {
709     const float cpuct = prm.cpuct_exploration + prm.cpuct_exploration_log *
710         std::log((total + prm.cpuct_exploration_base) / prm.cpuct_exploration_base);
711     const float explore_scaling = cpuct * utility_stdev_factor(*root,
712         ↳ root->n.load(std::memory_order_relaxed))
713         * std::sqrt(total + 0.01f);
714     const ChildStat& b = children[best];
715     const float best_value = b.q + explore_scaling * b.prior / (1.0f +
716         ↳ static_cast<float>(b.visits));
717     for (size_t i = 0; i < nb; i++) {
718         if (i == best) continue;
719         ChildStat& c = children[i];
720         if (c.visits == 0) { c.play_value = 0.0f; continue; }
721         const float excess = best_value - c.q; // the exploration term it would need
722         if (excess <= 0.0f) continue; // still the best move on value alone: keep
723         ↳ it
724         const float wanted = explore_scaling * c.prior / excess - 1.0f;
725         c.play_value = std::min(c.play_value, std::ceil(std::max(0.0f, wanted)));
726     }
727 }
728
729 // — 3. LCB: give the best lower bound enough play value to be chosen —
730 if (!prm.use_lcb || total <= 0.0f) return;
731 std::vector<float> radius(nb, 0.0f);
732 const float best_play = children[best].play_value;
733 float best_lcb = -1e10f;
734 size_t best_lcb_index = SIZE_MAX;
735 for (size_t i = 0; i < nb; i++) {
736     ChildStat& c = children[i];
737     if (c.visits == 0) { c.lcb = -2.0f; radius[i] = 2.0f * prm.lcb_stdevs; continue; }
738     // Weighted-sample machinery from KataGo, specialised to unit weights:
739     // every visit weighs 1, so weightSum = weightSqSum = edge visits and
740     // the effective sample size is the edge visit count itself.
741     const float n = static_cast<float>(c.visits);
742     const float mean_sq = c.q * c.q;
743     float sq_avg = std::max(c.q2, mean_sq + 1e-8f);
744     // A prior saying the variance could be as large as the whole utility
745     // range (radius 1), with a weight that vanishes as the visits grow.
746     const float prior_w = 1.0f / (n * n);
747     sq_avg = (sq_avg * n + (sq_avg + 1.0f) * prior_w) / (n + prior_w);
748     const float weight_sum = n + prior_w;
749     const float ess = weight_sum * weight_sum / (n + prior_w * prior_w);
750     const float var = std::max(0.0f, sq_avg - mean_sq);
751     radius[i] = std::sqrt(var / ess) * prm.lcb_stdevs;
752     c.lcb = c.q - radius[i];
753     if (c.play_value > 0.0f && c.play_value >= prm.min_visit_prop_for_lcb * best_play) {
754         if (c.lcb > best_lcb) { best_lcb = c.lcb; best_lcb_index = i; }
755     }
756 }
757 if (best_lcb_index == SIZE_MAX) return;
758 // The winner is given just enough play value to beat every other child:
759 // a child whose bound is `excess` worse would need its radius to grow by
760 // `factor` to catch up, and the radius shrinks like 1/√weight.
761 float adjusted = children[best_lcb_index].play_value;
762 for (size_t i = 0; i < nb; i++) {

```

```

760     if (i == best_lcb_index) continue;
761     const float excess = best_lcb - children[i].lcb;
762     if (excess < 0.0f) continue; // that child only lost on the visit threshold
763     const float factor = (radius[i] + excess) / (radius[i] + 0.20f * excess);
764     adjusted = std::max(adjusted, factor * factor * children[i].play_value);
765 }
766 children[best_lcb_index].play_value = adjusted;
767 }
768
769 SearchResult Search::make_result(const Yoloh& state, double seconds, uint64_t sims,
770                                uint64_t nb_collisions, Worker& w) const {
771     SearchResult res;
772     res.seconds = seconds;
773     res.nb_simulations = sims;
774     res.nb_collisions = nb_collisions;
775     res.nb_cache_hits = cache_hits.load();
776     res.nb_evaluations = evaluations.load();
777     res.nb_transpositions = transpositions.load();
778     res.full_search = full_search;
779     res.tree_nodes = node_count.load();
780     res.net_value = root->net_v;
781     const uint32_t root_n = root->n.load();
782     res.root_value = root_n > 0 ? -root->w.load() / root_n : root->net_v;
783
784     const auto& children = root->children;
785     res.children.reserve(children.size());
786     for (size_t i = 0; i < children.size(); i++) {
787         const Edge& e = children[i];
788         const Node* c = e.child.load(std::memory_order_acquire);
789         const uint32_t cn = c ? c->n.load() : 0;
790         ChildStat st{};
791         st.move = e.action;
792         st.prior = root_priors.empty() ? e.prior : root_priors[i];
793         st.visits = e.n.load();
794         st.q = cn > 0 ? c->w.load() / cn : 0.0f;
795         st.q2 = cn > 0 ? c->w2.load() / cn : 0.0f;
796         st.lcb = -2.0f;
797         res.children.push_back(st);
798     }
799     root_play_values(res.children);
800     std::stable_sort(res.children.begin(), res.children.end(), [](const ChildStat& a, const
801     ↪ ChildStat& b) {
802         return a.play_value != b.play_value ? a.play_value > b.play_value : a.q > b.q;
803     });
804     if (res.children.empty()) return res; // terminal root
805
806     // Training target  $\pi$ : the play values, normalised. If the search could not
807     // complete a single simulation, fall back to the priors.
808     float total = 0.0f;
809     for (const ChildStat& c : res.children) total += c.play_value;
810     res.policy.resize(res.children.size());
811     for (size_t i = 0; i < res.children.size(); i++)
812         res.policy[i] = total > 0 ? res.children[i].play_value / total : res.children[i].prior;
813
814     // Move choice: the best play value or, in the self-play opening phase, a
815     // sample  $\propto \pi^{(1/\tau)}$ .
816     size_t pick = 0;
817     if (total > 0.0f && prm.temperature > 0.0f && state.nb_plies() < prm.temperature_cutoff) {
818         std::vector<double> weights(res.children.size());
819         double sum = 0.0;
820         for (size_t i = 0; i < weights.size(); i++) {
821             weights[i] = std::pow(static_cast<double>(res.children[i].play_value), 1.0 /
822     ↪ prm.temperature);

```

```

821         sum += weights[i];
822     }
823     double r = (w.prng.rand<uint64_t>() >> 11) * 0x1.0p-53 * sum;
824     for (pick = 0; pick + 1 < weights.size(); pick++) {
825         r -= weights[pick];
826         if (r <= 0.0) break;
827     }
828     } else if (total == 0.0f) {
829         for (size_t i = 1; i < res.children.size(); i++)
830             if (res.children[i].prior > res.children[pick].prior) pick = i;
831     }
832     res.best_move = res.children[pick].move;
833     return res;
834 }
835
836 std::string SearchResult::to_string(size_t max_children) const {
837     std::ostringstream os;
838     os << std::fixed << std::setprecision(3);
839     os << "[ move ]: " << best_move << " Q: " << std::showpos << root_value << std::noshowpos
840         << " net: " << std::showpos << net_value << std::noshowpos
841         << " sims: " << nb_simulations << (full_search ? "" : " (fast)")
842         << " (" << root_visits << " root visits, " << nb_evaluations
843         << " net evals, " << nb_cache_hits << " cache hits, " << nb_transpositions
844         << " transpositions, " << nb_collisions << " collisions) "
845         << std::setprecision(1) << seconds << "s, "
846         << std::setprecision(0) << (seconds > 0 ? nb_simulations / seconds : 0) << " sims/s, nodes:
847         ↪ " << tree_nodes << "\n";
848     os << std::setprecision(3);
849     for (size_t i = 0; i < std::min(max_children, children.size()); i++) {
850         const ChildStat& c = children[i];
851         os << " " << c.move << " N: " << std::setw(6) << c.visits << " Q: " << std::showpos <<
852         ↪ c.q
853         << " lcb: " << c.lcb << std::noshowpos << " P: " << c.prior
854         << " pi: " << (policy.empty() ? 0.0f : policy[i]) << "\n";
855     }
856     return os.str();
857 } // namespace az

```

11.1.3 player/alphazero_mcts_player.h

92 lines.

The **Player** wrapper. The header comment is the authoritative JSON schema — the parameter reference in Section 8 is the same list with the defaults spelled out.

symbol	role
AlphaZeroMCTSPlayer	Player implementation: owns the pool resource, the evaluator and the az::Search
AlphaZeroMCTSPlayer::play	one search, one move; the result is kept for last_result()
AlphaZeroMCTSPlayer::game_over	reset the graph at the end of a game
AlphaZeroMCTSPlayer::last_result	the full root statistics of the last search — the self-play hook
AlphaZeroMCTSPlayer::reload_weights	hot-swap the network: resets the graph and the network cache
AlphaZeroMCTSPlayer::config	round-trip the live configuration back to JSON

symbol

role

```
1  #ifndef ALPHAZERO_MCTS_PLAYER_H
2  #define ALPHAZERO_MCTS_PLAYER_H
3  // alphazero_mcts_player.h – Player wrapper around az::Search + a network backend.
4  //
5  // JSON configuration (keys follow the other players' conventions):
6  // {
7  //   "name": "AlphaZeroMCTSPlayer",
8  //   "microseconds": 1000000,          // thinking time per move (0 = use only "nb simulations")
9  //   "nb simulations": 0,              // simulation budget (0 = time only); 800 for self-play
10 //   "nb threads": 1,                  // search threads ("hardware concurrency" allowed)
11 //   "batch size": 32,                 // leaves gathered per search thread per network call
12 //   "weights": "../nnue/cnn_resnet_256x30_value_policy.bin", // .bin (cpu) or .ts (torch)
13 //   "backend": "cpu",                // "cpu" (Eigen/OpenMP) | "torch" (libtorch,
14 //   ↪ -DENABLE_TORCH=ON)
15 //   "device": "cuda",                // torch only
16 //   "fp16": true,                    // torch only
17 //   "nb eval threads": 0,             // cpu only: OpenMP threads inside the network (0 = all)
18 //   // PUCT, KataGo style (see alphazero_mcts.h); the defaults are KataGo's
19 //   "cpuct exploration": 1.0, "cpuct exploration log": 0.45, "cpuct exploration base": 500,
20 //   "cpuct utility stdev scale": 0.85, // 0 = plain AlphaZero exploration
21 //   "cpuct utility stdev prior": 0.40, "cpuct utility stdev prior weight": 2.0,
22 //   "fpu reduction": 0.2, "root fpu reduction": 0.1,
23 //   "fpu loss prop": 0.0, "root fpu loss prop": 0.0,
24 //   "fpu parent weight by visited policy": true,
25 //   "fpu parent weight by visited policy pow": 1.0, "fpu parent weight": 0.0,
26 //   "policy temperature": 1.0,
27 //   "graph search": true,             // one node per position (DAG) instead of a tree
28 //   // root move choice
29 //   "use lcb": true, "lcb stdevs": 5.0, "min visit prop for lcb": 0.2,
30 //   "policy target pruning": true,    // take the forced playouts back out of pi
31 //   "dirichlet alpha": 0.3, "dirichlet epsilon": 0.0, // root noise (self-play)
32 //   "temperature": 0.0, "temperature cutoff": 0, // move sampling (self-play)
33 //   "forced playouts k": 0.0,        // 2.0 during self-play, 0 for competitive play
34 //   "playout cap fast prob": 0.0,    // self-play: fraction of moves with a small budget
35 //   "nb simulations fast": 0,        // ... and how small (needs "nb simulations")
36 //   "reuse tree": true,
37 //   "nn cache": 64,                  // MB of transposition cache for network outputs (0 = off)
38 //   "seed": 0,
39 //   "verbose": false                 // print the root statistics after every move
40 // }
41 //
42 // Only "microseconds" (or "nb simulations") and "weights" are mandatory.
43 //
44 // Threading model: the search threads share one tree and one evaluator. With
45 // more than one search thread the evaluator is wrapped in nn::BatchingEvaluator
46 // so that the backend sees merged batches (essential for a GPU, and it keeps
47 // the OpenMP CPU backend from being entered concurrently).
48 //
49 // Self-play: after play() the full root statistics of the last search are
50 // available through last_result() – its play values are the policy target  $\pi$ 
51 // and the game outcome gives  $z$ . The learning loop itself (player/
52 // alphazero_selfplay.h, nnue/alphazero_learn.py) does not use this class: it
53 // runs az::Search directly, many games on one shared evaluator, with the
54 // parameters built by search_params(). Self-play settings: "dirichlet
55 // epsilon" 0.25, "temperature" 1.0, "temperature cutoff" 20, "nb simulations"
56 // 800 – see config/alphazero_mcts_selfplay_player.cfg.
57 #include "player.h"
58 #include "alphazero_mcts.h"
59 #include "nn_evaluator.h"
60 #include <memory>
61 #include <memory_resource>
```

```

60 #include <string>
61
62 class AlphaZeroMCTSPlayer : public Player {
63 public:
64     explicit AlphaZeroMCTSPlayer(const json& config);
65     ~AlphaZeroMCTSPlayer() override;
66
67     Move play(Yolah) override;
68     void game_over(Yolah) override;
69     std::string info() override;
70     json config() override;
71
72     const az::SearchResult& last_result() const { return last; }
73     az::Search& search() { return *searcher; }
74     nn::Evaluator& evaluator() { return *net; }
75     // Load new weights (same architecture) into the running backend.
76     void reload_weights(const std::string& weights_filename);
77     // The search parameters described by a JSON config (the keys above);
78     // missing keys keep az::SearchParams' defaults. Used by the self-play
79     // driver, which runs az::Search directly on a shared evaluator.
80     static az::SearchParams search_params(const json& config);
81
82 private:
83     json cfg;
84     std::string weights_filename;
85     bool verbose;
86     std::pmr::synchronized_pool_resource memory;
87     std::unique_ptr<nn::Evaluator> net;
88     std::unique_ptr<az::Search> searcher;
89     az::SearchResult last;
90 };
91
92 #endif

```

11.1.4 player/alphazero_mcts_player.cpp

149 lines.

JSON in, SearchParams out, plus the decision of whether to wrap the backend in a `BatchingEvaluator`. The only subtlety is the ordering in the constructor: the backend has to exist before `batch_size` can default to `preferred_batch_size()`, and the search has to be destroyed before the pool resource it allocated the graph from — hence the explicit destructor.

symbol	role
<code>read_nb_threads</code>	accepts a number or the string "hardware concurrency"
<code>AlphaZeroMCTSPlayer::AlphaZeroMCTSPlayer</code>	parse the config, build the backend, wrap it if multi-threaded, build the search
<code>AlphaZeroMCTSPlayer::~AlphaZeroMCTSPlayer</code>	destroy the search first: the graph lives in memory

```

1 #include "alphazero_mcts_player.h"
2 #include "batched_evaluator.h"
3 #include <iostream>
4 #include <stdexcept>
5 #include <thread>
6

```

```

7 using std::string, std::invalid_argument;
8
9 namespace {
10     size_t read_nb_threads(const json& j, const char* key, size_t default_value) {
11         if (!j.contains(key)) return default_value;
12         if (j[key].is_number()) return j[key].get<size_t>();
13         if (j[key].is_string() && j[key].get<string>() == "hardware concurrency")
14             return std::thread::hardware_concurrency();
15         throw invalid_argument(string("number or \"hardware concurrency\" expected for ") + key);
16     }
17 }
18
19 az::SearchParams AlphaZeroMCTSPlayer::search_params(const json& j) {
20     az::SearchParams p;
21     p.microseconds = j.value("microseconds", uint64_t(0));
22     p.nb_simulations = j.value("nb simulations", uint64_t(0));
23     p.nb_threads = read_nb_threads(j, "nb threads", 1);
24     p.batch_size = j.value("batch size", size_t(0));
25     p.max_collisions = j.value("max collisions", size_t(0));
26     p.nn_cache_mb = j.value("nn cache", size_t(64));
27     p.cpuct_exploration = j.value("cpuct exploration", p.cpuct_exploration);
28     p.cpuct_exploration_log = j.value("cpuct exploration log", p.cpuct_exploration_log);
29     p.cpuct_exploration_base = j.value("cpuct exploration base", p.cpuct_exploration_base);
30     p.cpuct_utility_stdev_scale = j.value("cpuct utility stdev scale",
↪ p.cpuct_utility_stdev_scale);
31     p.cpuct_utility_stdev_prior = j.value("cpuct utility stdev prior",
↪ p.cpuct_utility_stdev_prior);
32     p.cpuct_utility_stdev_prior_weight = j.value("cpuct utility stdev prior weight",
↪ p.cpuct_utility_stdev_prior_weight);
33     p.fpu_reduction = j.value("fpu reduction", p.fpu_reduction);
34     p.root_fpu_reduction = j.value("root fpu reduction", p.root_fpu_reduction);
35     p.fpu_loss_prop = j.value("fpu loss prop", p.fpu_loss_prop);
36     p.root_fpu_loss_prop = j.value("root fpu loss prop", p.root_fpu_loss_prop);
37     p.fpu_parent_weight_by_visited_policy =
38         j.value("fpu parent weight by visited policy", p.fpu_parent_weight_by_visited_policy);
39     p.fpu_parent_weight_by_visited_policy_pow =
40         j.value("fpu parent weight by visited policy pow",
↪ p.fpu_parent_weight_by_visited_policy_pow);
41     p.fpu_parent_weight = j.value("fpu parent weight", p.fpu_parent_weight);
42     p.policy_temperature = j.value("policy temperature", p.policy_temperature);
43     p.graph_search = j.value("graph search", p.graph_search);
44     p.use_lcb = j.value("use lcb", p.use_lcb);
45     p.lcb_stdevs = j.value("lcb stdevs", p.lcb_stdevs);
46     p.min_visit_prop_for_lcb = j.value("min visit prop for lcb", p.min_visit_prop_for_lcb);
47     p.policy_target_pruning = j.value("policy target pruning", p.policy_target_pruning);
48     p.forced_playouts_k = j.value("forced playouts k", p.forced_playouts_k);
49     p.playout_cap_fast_prob = j.value("playout cap fast prob", p.playout_cap_fast_prob);
50     p.nb_simulations_fast = j.value("nb simulations fast", p.nb_simulations_fast);
51     p.dirichlet_alpha = j.value("dirichlet alpha", p.dirichlet_alpha);
52     p.dirichlet_epsilon = j.value("dirichlet epsilon", p.dirichlet_epsilon);
53     p.temperature = j.value("temperature", p.temperature);
54     p.temperature_cutoff = j.value("temperature cutoff", p.temperature_cutoff);
55     p.reuse_tree = j.value("reuse tree", true);
56     p.seed = j.value("seed", uint64_t(0));
57     return p;
58 }
59
60 AlphaZeroMCTSPlayer::AlphaZeroMCTSPlayer(const json& j) : cfg(j) {
61     if (!j.contains("weights")) throw invalid_argument("weights key expected");
62     if (!j.contains("microseconds") && !j.contains("nb simulations"))
63         throw invalid_argument("microseconds or nb simulations key expected");
64     weights_filename = j["weights"].get<string>();
65     verbose = j.value("verbose", false);

```

```

66
67     az::SearchParams p = search_params(j);
68
69     // Backend, then the batching wrapper when several search threads share it.
70     std::unique_ptr<nn::Evaluator> backend = nn::make_evaluator(j);
71     if (p.batch_size == 0) p.batch_size = backend->preferred_batch_size();
72     if (p.nb_threads > 1) {
73         const size_t merged = j.value("merged batch size", p.batch_size * p.nb_threads);
74         net = std::make_unique<nn::BatchingEvaluator>(std::move(backend), p.nb_threads, merged);
75     } else {
76         net = std::move(backend);
77     }
78     searcher = std::make_unique<az::Search>(*net, p, &memory);
79 }
80
81 AlphaZeroMCTSPlayer::~AlphaZeroMCTSPlayer() {
82     // The tree lives in `memory`; destroy it before the pool goes away.
83     searcher.reset();
84 }
85
86 Move AlphaZeroMCTSPlayer::play(Yolah yolah) {
87     last = searcher->search(yolah);
88     if (verbose) std::cout << last.to_string() << std::flush;
89     return last.best_move;
90 }
91
92 void AlphaZeroMCTSPlayer::game_over(Yolah) {
93     searcher->reset();
94 }
95
96 void AlphaZeroMCTSPlayer::reload_weights(const std::string& filename) {
97     net->reload(filename);
98     weights_filename = filename;
99     cfg["weights"] = filename;
100     searcher->reset(); // the old tree's priors/values came from the old network
101     searcher->clear_cache(); // so did the cached evaluations
102 }
103
104 std::string AlphaZeroMCTSPlayer::info() {
105     return "alphazero mcts player (PUCT + value/policy resnet, " + net->info() + ")";
106 }
107
108 json AlphaZeroMCTSPlayer::config() {
109     json j = cfg;
110     j["name"] = "AlphaZeroMCTSPlayer";
111     const az::SearchParams& p = searcher->params();
112     j["microseconds"] = p.microseconds;
113     j["nb simulations"] = p.nb_simulations;
114     if (p.nb_threads == std::thread::hardware_concurrency()) j["nb threads"] = "hardware
    ↪ concurrency";
115     else j["nb threads"] = p.nb_threads;
116     j["batch size"] = p.batch_size;
117     j["weights"] = weights_filename;
118     j["backend"] = cfg.value("backend", string("cpu"));
119     j["cpuct exploration"] = p.cpuct_exploration;
120     j["cpuct exploration log"] = p.cpuct_exploration_log;
121     j["cpuct exploration base"] = p.cpuct_exploration_base;
122     j["cpuct utility stdev scale"] = p.cpuct_utility_stdev_scale;
123     j["cpuct utility stdev prior"] = p.cpuct_utility_stdev_prior;
124     j["cpuct utility stdev prior weight"] = p.cpuct_utility_stdev_prior_weight;
125     j["fpu reduction"] = p.fpu_reduction;
126     j["root fpu reduction"] = p.root_fpu_reduction;
127     j["fpu loss prop"] = p.fpu_loss_prop;

```

```

128     j["root fpu loss prop"] = p.root_fpu_loss_prop;
129     j["fpu parent weight by visited policy"] = p.fpu_parent_weight_by_visited_policy;
130     j["fpu parent weight by visited policy pow"] = p.fpu_parent_weight_by_visited_policy_pow;
131     j["fpu parent weight"] = p.fpu_parent_weight;
132     j["policy temperature"] = p.policy_temperature;
133     j["graph search"] = p.graph_search;
134     j["use lcb"] = p.use_lcb;
135     j["lcb stdevs"] = p.lcb_stdevs;
136     j["min visit prop for lcb"] = p.min_visit_prop_for_lcb;
137     j["policy target pruning"] = p.policy_target_pruning;
138     j["forced playouts k"] = p.forced_playouts_k;
139     j["playout cap fast prob"] = p.playout_cap_fast_prob;
140     j["nb simulations fast"] = p.nb_simulations_fast;
141     j["dirichlet alpha"] = p.dirichlet_alpha;
142     j["dirichlet epsilon"] = p.dirichlet_epsilon;
143     j["temperature"] = p.temperature;
144     j["temperature cutoff"] = p.temperature_cutoff;
145     j["reuse tree"] = p.reuse_tree;
146     j["nn cache"] = p.nn_cache_mb;
147     j["verbose"] = verbose;
148     return j;
149 }

```

11.1.5 player/nn_cache.h

105 lines.

A direct-mapped cache of network outputs, keyed by the Zobrist hash — which is exactly the network's input, so a hit is always valid. It is the second line of defence behind the node table: it also catches positions swept out of the graph, and it survives across moves and games.

Two details worth noting: the stored move count is verified on lookup, so a 64-bit collision cannot hand back another position's priors; and priors are 16-bit fixed point, which costs 1/65535 of resolution and halves the entry size.

symbol	role
NNCache::Entry	key, value, move count, and the priors as <code>uint16</code> fixed point
NNCache::NNCache	size in bytes, rounded down to a power-of-two entry count; 0 disables it
NNCache::lookup	hit only if both the key and the move count match
NNCache::store	always overwrite — the table is direct-mapped
NNCache::clear	take every stripe lock, then wipe (used by <code>reload_weights</code>)

```

1  #ifndef NN_CACHE_H
2  #define NN_CACHE_H
3  // nn_cache.h – transposition cache of network evaluations for the AlphaZero
4  // MCTS (az::Search).
5  //
6  // In Yolah the four stones of a side move independently, so the same position
7  // is reached through many move orders: a large share of the leaves the search
8  // wants to expand are transpositions of positions already evaluated. The
9  // network only sees (board, side to move), so its output is a pure function of
10 // the Zobrist hash of that pair – we cache it: value + the priors of the legal

```

```

11 // moves (in Yolah::moves order, which is deterministic for a position). A hit
12 // expands the node without a network call, which is where >99 % of the time
13 // goes.
14 //
15 // Direct-mapped table (index = hash & mask, always overwrite), lock-stripped so
16 // several search threads can use it concurrently; the cost of a lookup is
17 // negligible next to a network evaluation. The hash also verifies the number of
18 // legal moves, so a 64-bit collision cannot silently produce garbage priors.
19 // Priors are stored as 16-bit fixed point (1/65535 resolution).
20 //
21 // Statistics are shared by everything that evaluates the same network, so the
22 // cache survives across moves and games; it must be cleared when the weights
23 // change (az::Search::clear_cache / AlphaZeroMCTSPlayer::reload_weights).
24 #include "game.h"
25 #include <array>
26 #include <atomic>
27 #include <cstdint>
28 #include <cstring>
29 #include <mutex>
30 #include <vector>
31
32 namespace az {
33
34 class NNCache {
35 public:
36     struct Entry {
37         uint64_t key = 0;
38         float value = 0.0f;
39         uint16_t nb_moves = 0;
40         uint16_t priors[Yolah::MAX_NB_MOVES] = {};
41     };
42
43     // size_bytes = 0 disables the cache. The entry count is rounded down to a
44     // power of two.
45     explicit NNCache(size_t size_bytes) {
46         size_t n = size_bytes / sizeof(Entry);
47         size_t pow2 = 1;
48         while (pow2 * 2 <= n) pow2 *= 2;
49         if (n >= 2) {
50             table.resize(pow2);
51             mask = pow2 - 1;
52         }
53     }
54
55     bool enabled() const { return !table.empty(); }
56     size_t capacity() const { return table.size(); }
57     size_t size_bytes() const { return table.size() * sizeof(Entry); }
58
59     // Returns true and fills value / priors[0..nb_moves) on a hit.
60     bool lookup(uint64_t key, size_t nb_moves, float& value, float* priors) {
61         if (table.empty()) return false;
62         const Entry& e = table[key & mask];
63         std::lock_guard lock(locks[key & (locks.size() - 1)]);
64         if (e.key != key || e.nb_moves != nb_moves) {
65             misses.fetch_add(1, std::memory_order_relaxed);
66             return false;
67         }
68         value = e.value;
69         for (size_t m = 0; m < nb_moves; m++) priors[m] = e.priors[m] * (1.0f / 65535.0f);
70         hits.fetch_add(1, std::memory_order_relaxed);
71         return true;
72     }
73

```

```

74     void store(uint64_t key, size_t nb_moves, float value, const float* priors) {
75         if (table.empty()) return;
76         Entry& e = table[key & mask];
77         std::lock_guard lock(locks[key & (locks.size() - 1)]);
78         e.key = key;
79         e.value = value;
80         e.nb_moves = static_cast<uint16_t>(nb_moves);
81         for (size_t m = 0; m < nb_moves; m++)
82             e.priors[m] = static_cast<uint16_t>(priors[m] * 65535.0f + 0.5f);
83     }
84
85     void clear() {
86         for (size_t i = 0; i < locks.size(); i++) locks[i].lock();
87         std::fill(table.begin(), table.end(), Entry{});
88         hits = 0;
89         misses = 0;
90         for (size_t i = locks.size(); i-- > 0;) locks[i].unlock();
91     }
92
93     uint64_t nb_hits() const { return hits.load(std::memory_order_relaxed); }
94     uint64_t nb_misses() const { return misses.load(std::memory_order_relaxed); }
95
96 private:
97     std::vector<Entry> table;
98     size_t mask = 0;
99     std::array<std::mutex, 256> locks;
100     std::atomic<uint64_t> hits{0}, misses{0};
101 };
102
103 } // namespace az
104
105 #endif

```

11.2 The network interface and its backends

Everything below `nn::Evaluator` is interchangeable: the search never learns which backend it is talking to, and `make_evaluator` picks one from the same JSON the player was given.

11.2.1 `nnue/nn_evaluator.h`

121 lines.

The interface, plus the two encodings that *must* agree with the training code: `encode_planes` (the four input planes, in the reversed cell order of Section 3) and `action_index` (`from*64 + to`). If either drifts from the Python side, the network silently reads a permuted board.

symbol	role
PLANES, PLANE_CELLS, INPUT_FLOATS, NUM_ACTIONS	the shapes, shared with the training script
<code>encode_planes</code>	position $\rightarrow$ 256 floats, NCHW, cell $i = \text{bit } 63 - i$
<code>action_index</code>	move $\rightarrow$ policy index; a pass maps to 0 (A1 $\rightarrow$ A1)
Request / Result	a position with its legal moves / a value and the logits of exactly those moves

symbol	role
Evaluator	the abstract backend: <code>evaluate</code> , <code>preferred_batch_size</code> , <code>info</code> , <code>reload</code>
<code>make_evaluator</code>	factory over the "backend" key

```

1  #ifndef NN_EVALUATOR_H
2  #define NN_EVALUATOR_H
3  // nn_evaluator.h – backend-agnostic interface between the AlphaZero-style MCTS
4  // (player/alphazero_mcts.h) and the two-headed value/policy network.
5  //
6  // The search never touches tensors: it hands the evaluator a batch of
7  // positions together with their legal moves and receives, per position,
8  //   • a scalar value  $v \in (-1, +1)$  from the point of view of the player to move
9  //   (the network's tanh output, trained on  $z \in \{-1, 0, +1\}$ );
10 //   • the raw policy logits of *exactly the legal moves*, in the order they
11 //     were given. Softmax (with an optional temperature) is done by the search.
12 //
13 // Two backends implement this interface:
14 //   nnue/resnet.h – pure C++ (Eigen + OpenMP), no external dependency,
15 //                 loads the .bin written by nnue/resnet_export.py;
16 //   nnue/resnet_torch.h – libtorch/TorchScript, GPU capable, built only with
17 //                         -DENABLE_TORCH=ON, loads the .ts from the same script.
18 // plus a wrapper:
19 //   nnue/batched_evaluator.h – merges the requests of several search threads
20 //                             into one large batch before calling a backend.
21 //
22 // Learning loop (nnue/alphazero_learn.py): a freshly trained network is
23 // exported as TorchScript and swapped in with reload() while the self-play
24 // games keep running – no search object needs to be rebuilt.
25 #include "game.h"
26 #include <stdint>
27 #include <memory>
28 #include <span>
29 #include <string>
30
31 namespace nn {
32
33 // — Board encoding (MUST match preprocess.py / encode_cnn in
34 //   cnn_resnet_value_policy_chunked.py) —————
35 //
36 // The network sees 4 planes of 8x8:
37 //   plane 0 black stones, plane 1 white stones, plane 2 "empty" (vacated)
38 //   squares, plane 3 the side to move (all 0 = black, all 1 = white).
39 // Plane cell  $i$  (row-major,  $i = 8 \cdot \text{row} + \text{col}$  with row/col in  $0..7$ ) holds bit
40 //  $(63 - i)$  of the bitboard, i.e. square  $(63 - i)$ , since bit  $s$  IS square  $s$ 
41 // ( $\text{SQ\_A1} = 0 \dots \text{SQ\_H8} = 63$ ).
42 //
43 // Where the  $(63 - i)$  comes from: preprocess.py builds a plane with
44 //   np.unpackbits(np.array([bb], dtype='>u8').view(np.uint8)).reshape(8, 8)
45 // '>u8' dumps the integer big-endian (most significant byte first) and
46 // unpackbits takes the most significant bit of each byte first, so the values
47 // come out in the order bit 63, 62, ..., 0 – element  $i$  is bit  $63 - i$ .
48 //
49 // Geometrically, with  $s = 8 \cdot \text{rank} + \text{file}$ , that is row = 7 - rank and
50 // col = 7 - file: the ranks are in the usual order (rank 8 on the top row)
51 // but the FILES ARE MIRRORED, H on the left. Cell 0 is H8, cell 63 is A1.
52 // The network does not care – a fixed mirror of every input is just a
53 // relabelling of the board – but training and inference must agree exactly,
54 // so do not "fix" this: it is what the weights were trained on.
55 constexpr int PLANES      = 4;
56 constexpr int PLANE_CELLS = 64;

```

```

57 constexpr int INPUT_FLOATS = PLANES * PLANE_CELLS;
58 constexpr int NUM_ACTIONS = 64 * 64;
59
60 // Fill out[0..255] with the 4 planes of `y` in NCHW order (plane-major).
61 inline void encode_planes(const Yolah& y, float* out) {
62     const uint64_t bbs[3] = { y.bitboard(Yolah::BLACK), y.bitboard(Yolah::WHITE),
63         ⇨ y.empty_bitboard() };
64     for (int c = 0; c < 3; c++) {
65         const uint64_t bb = bbs[c];
66         float* plane = out + c * PLANE_CELLS;
67         for (int i = 0; i < PLANE_CELLS; i++) {
68             plane[i] = static_cast<float>((bb >> (63 - i)) & 1);
69         }
70     }
71     const float turn = (y.current_player() == Yolah::WHITE) ? 1.0f : 0.0f;
72     float* plane = out + 3 * PLANE_CELLS;
73     for (int i = 0; i < PLANE_CELLS; i++) {
74         plane[i] = turn;
75     }
76 }
77
78 // Policy head index of a move: from*64 + to (preprocess.py: pol = s1*64 + s2).
79 // A pass (Move::none() == al:al) maps to index 0 exactly like in training.
80 constexpr int action_index(Move m) {
81     return static_cast<int>(m.from_sq()) * 64 + static_cast<int>(m.to_sq());
82 }
83
84 // — Batch interface —————
85 struct Request {
86     const Yolah* state; // position to evaluate
87     const Move* moves; // its legal moves (Yolah::moves), nb_moves >= 1
88     uint16_t nb_moves;
89 };
90
91 struct Result {
92     float value; // for the player to move at Request::state
93     float logits[Yolah::MAX_NB_MOVES]; // policy logits of moves[0..nb_moves)
94 };
95
96 class Evaluator {
97 public:
98     virtual ~Evaluator() = default;
99     // Evaluate in.size() positions; out.size() must be >= in.size().
100     // Thread-safe: may be called concurrently from several search threads.
101     virtual void evaluate(std::span<const Request> in, std::span<Result> out) = 0;
102     // The batch size the backend is most efficient at (a hint for the search).
103     virtual size_t preferred_batch_size() const = 0;
104     // Human readable description (backend, device, weights file).
105     virtual std::string info() const = 0;
106     // Swap in new weights (same architecture). Must not be called while
107     // evaluate() is running.
108     virtual void reload(const std::string& weights_filename) = 0;
109 };
110
111 // Build an evaluator from the player's JSON config:
112 // "backend" : "cpu" (default) | "torch"
113 // "weights" : path to the .bin (cpu) or .ts (torch) exported file
114 // "device" : torch only – "cuda" (default if available) | "cpu"
115 // "fp16" : torch only – run the network in half precision (default true on cuda)
116 // "nb eval threads" : cpu only – OpenMP threads used inside one evaluate() (0 = all)
117 // "batch size" : preferred batch size reported to the search (default backend specific)
118 std::unique_ptr<Evaluator> make_evaluator(const json& config);

```

```

119 } // namespace nn
120
121 #endif

```

11.2.2 nnue/nn_evaluator.cpp

34 lines.

The factory. Nothing more: it reads "backend", "weights", "device", "fp16" and "nb eval threads", and fails loudly when the libtorch backend is asked for in a build that does not have it.

symbol	role
make_evaluator	"cpu" → ResNetCPU, "torch" → ResNetTorch (only with ENABLE_TORCH)

```

1  #include "nn_evaluator.h"
2  #include "resnet.h"
3  #include "batched_evaluator.h"
4  #ifdef ENABLE_TORCH
5  #include "resnet_torch.h"
6  #endif
7  #include <stdexcept>
8  #include <thread>
9
10 namespace nn {
11
12 std::unique_ptr<Evaluator> make_evaluator(const json& j) {
13     if (!j.contains("weights")) throw std::invalid_argument("weights key expected");
14     const std::string weights = j["weights"].get<std::string>();
15     const std::string backend = j.value("backend", std::string("cpu"));
16     if (backend == "cpu") {
17         const int eval_threads = j.value("nb eval threads", 0);
18         const size_t batch = j.value("batch size", 32);
19         return std::make_unique<ResNetCPU>(weights, eval_threads, batch);
20     }
21     if (backend == "torch") {
22 #ifdef ENABLE_TORCH
23         const std::string device = j.value("device", std::string("cuda"));
24         const bool fp16 = j.value("fp16", true);
25         const size_t batch = j.value("batch size", 256);
26         return std::make_unique<ResNetTorch>(weights, device, fp16, batch);
27 #else
28         throw std::invalid_argument("backend \"torch\" requires a build with -DENABLE_TORCH=ON");
29 #endif
30     }
31     throw std::invalid_argument("unknown backend \"" + backend + "\" (expected \"cpu\" or
32     ↪ \"torch\")");
33 }
34 } // namespace nn

```

11.2.3 nnue/batched_evaluator.h

85 lines.

The wrapper that merges several search threads into one backend call. The header comment explains the three conditions that fire a batch; the empty-request barrier is the part worth remembering, because it is what a search thread whose descents all collided uses instead of spinning.

symbol	role
BatchingEvaluator	Evaluator that owns a backend and a worker thread
BatchingEvaluator::Job	one caller's request/result spans plus its completion flag
BatchingEvaluator::evaluate	enqueue and block; an empty span is a barrier join
BatchingEvaluator::set_nb_clients	how many threads must be waiting before a batch fires early
BatchingEvaluator::worker_loop	wait, concatenate, call the backend once, scatter, wake the callers

```

1  #ifndef BATCHED_EVALUATOR_H
2  #define BATCHED_EVALUATOR_H
3  // batched_evaluator.h – merges the evaluate() calls of several search threads
4  // into larger batches before handing them to a backend.
5  //
6  // Why: a GPU (or the OpenMP CPU backend) is efficient only on large batches,
7  // whereas each MCTS thread can only gather a limited number of leaves per
8  // iteration without degrading the search (virtual-loss collisions). With T
9  // search threads each submitting ~B leaves, the backend sees batches of up to
10 // T·B positions instead of B.
11 //
12 // How: callers enqueue their (requests, results) spans and block. One worker
13 // thread wakes up as soon as
14 //   • the queued positions reach `max_batch`, or
15 //   • every registered client is waiting (`nb_clients`), or
16 //   • `max_wait` has elapsed with something queued,
17 // concatenates the queued requests, runs the backend once, scatters the
18 // results back and releases the callers. The queue is FIFO so no client can
19 // starve.
20 //
21 // An EMPTY request is a barrier join: the caller has nothing to evaluate but
22 // wants to block until the next batch has been processed (a search thread
23 // whose descents all collided with leaves owned by another thread). It counts
24 // as a waiting client, so it does not delay the batch.
25 #include "nn_evaluator.h"
26 #include <chrono>
27 #include <condition_variable>
28 #include <deque>
29 #include <memory>
30 #include <mutex>
31 #include <thread>
32 #include <vector>
33
34 namespace nn {
35
36 class BatchingEvaluator final : public Evaluator {
37 public:
38     // backend      : the evaluator that does the actual work (owned)
39     // nb_clients   : number of threads expected to call evaluate() concurrently;
40     //               a batch is launched as soon as all of them are waiting
41     // max_batch    : upper bound on the merged batch size
42     // max_wait     : how long the worker waits for stragglers before launching

```

```

43 //          a partial batch
44 BatchingEvaluator(std::unique_ptr<Evaluator> backend, size_t nb_clients, size_t max_batch,
45                  std::chrono::microseconds max_wait = std::chrono::microseconds(500));
46 ~BatchingEvaluator() override;
47
48 void evaluate(std::span<const Request> in, std::span<Result> out) override;
49 size_t preferred_batch_size() const override { return max_batch; }
50 std::string info() const override;
51 void reload(const std::string& weights_filename) override;
52
53 Evaluator& backend() { return *impl; }
54 void set_nb_clients(size_t n);
55
56 private:
57     struct Job {
58         std::span<const Request> in;
59         std::span<Result> out;
60         bool done = false;
61     };
62     void worker_loop();
63
64     std::unique_ptr<Evaluator> impl;
65     size_t nb_clients;
66     const size_t max_batch;
67     const std::chrono::microseconds max_wait;
68
69     std::mutex mutex;
70     std::condition_variable cv_worker; // signals the worker: new job / stop
71     std::condition_variable cv_clients; // signals clients: results ready
72     std::deque<Job*> queue;
73     size_t queued_positions = 0;
74     size_t waiting_clients = 0;
75     bool stop = false;
76     std::thread worker;
77
78     // Scratch owned by the worker.
79     std::vector<Request> batch_in;
80     std::vector<Result> batch_out;
81 };
82
83 } // namespace nn
84
85 #endif

```

11.2.4 nnue/batched_evaluator.cpp

98 lines.

The queue. FIFO, so no client can starve; the worker fires on whichever of the three conditions comes first, and `reload` takes the same lock as the worker so weights cannot change mid-batch.

```

1 #include "batched_evaluator.h"
2 #include <algorithm>
3
4 namespace nn {
5
6 BatchingEvaluator::BatchingEvaluator(std::unique_ptr<Evaluator> backend, size_t nb_clients,
7                                     size_t max_batch, std::chrono::microseconds max_wait)
8     : impl(std::move(backend)), nb_clients(std::max<size_t>(1, nb_clients)),
9       max_batch(std::max<size_t>(1, max_batch)), max_wait(max_wait) {

```

```

10     batch_in.reserve(this->max_batch);
11     batch_out.resize(this->max_batch);
12     worker = std::thread([this] { worker_loop(); });
13 }
14
15 BatchingEvaluator::~BatchingEvaluator() {
16     {
17         std::lock_guard lock(mutex);
18         stop = true;
19     }
20     cv_worker.notify_all();
21     worker.join();
22 }
23
24 std::string BatchingEvaluator::info() const {
25     return "batching(" + std::to_string(nb_clients) + " clients, max batch " +
26           std::to_string(max_batch) + ") over " + impl->info();
27 }
28
29 void BatchingEvaluator::reload(const std::string& weights_filename) {
30     // Serialise with the worker: no batch may be running while weights change.
31     std::lock_guard lock(mutex);
32     impl->reload(weights_filename);
33 }
34
35 void BatchingEvaluator::set_nb_clients(size_t n) {
36     std::lock_guard lock(mutex);
37     nb_clients = std::max<size_t>(1, n);
38 }
39
40 void BatchingEvaluator::evaluate(std::span<const Request> in, std::span<Result> out) {
41     Job job{in, out}; // in.empty() = barrier join, see header
42     std::unique_lock lock(mutex);
43     queue.push_back(&job);
44     queued_positions += in.size();
45     ++waiting_clients;
46     cv_worker.notify_one();
47     cv_clients.wait(lock, [&] { return job.done; });
48     --waiting_clients;
49 }
50
51 void BatchingEvaluator::worker_loop() {
52     std::unique_lock lock(mutex);
53     for (;;) {
54         // Wake-up conditions (see header). wait_for handles the straggler timeout.
55         auto ready = [&] {
56             return stop || queued_positions >= max_batch ||
57                    (!queue.empty() && waiting_clients >= nb_clients);
58         };
59         if (!ready()) {
60             if (!queue.empty()) cv_worker.wait_for(lock, max_wait, ready);
61             else cv_worker.wait(lock, [&] { return stop || !queue.empty(); });
62         }
63         if (stop && queue.empty()) return;
64         if (queue.empty()) continue;
65
66         // Take whole jobs (FIFO) until the next one would overflow max_batch.
67         // The first job is always taken, even if it alone exceeds max_batch:
68         // backends accept any batch size, max_batch is only the merge target.
69         std::vector<Job*> taken;
70         batch_in.clear();
71         while (!queue.empty()) {
72             Job* j = queue.front();

```

```

73         if (!taken.empty() && batch_in.size() + j->in.size() > max_batch) break;
74         taken.push_back(j);
75         batch_in.insert(batch_in.end(), j->in.begin(), j->in.end());
76         queued_positions -= j->in.size();
77         queue.pop_front();
78     }
79     if (batch_out.size() < batch_in.size()) batch_out.resize(batch_in.size());
80
81     // Run the backend without holding the lock so clients can keep queueing.
82     if (!batch_in.empty()) {
83         lock.unlock();
84         impl->evaluate(batch_in, std::span<Result>(batch_out.data(), batch_in.size()));
85         lock.lock();
86     }
87
88     size_t offset = 0;
89     for (Job* j : taken) {
90         std::copy_n(batch_out.begin() + offset, j->in.size(), j->out.begin());
91         offset += j->in.size();
92         j->done = true;
93     }
94     cv_clients.notify_all();
95 }
96 }
97
98 } // namespace nn

```

11.2.5 nnue/resnet.h

98 lines.

The pure-C++ backend’s declaration. The header comment is the performance rationale: NHWC activations, im2col + one GEMM per convolution, BatchNorm folded at export time, OpenMP over chunks of the batch.

symbol	role
ResNetCPU	Evaluator over the .bin written by <code>resnet_export.py</code>
ResNetCPU::Block	one residual block’s folded weights: two (scale, shift) pairs and two kernels
ResNetCPU::Scratch	per-thread activation and im2col buffers, grown lazily
ResNetCPU::forward	the reference path used by the self-check: planes in, values and all 4096 logits out
ResNetCPU::forward_chunk	trunk plus heads on at most MAX_CHUNK positions
ResNetCPU::policy_logit	one logit from the 128-d policy feature vector — only the legal moves are computed

```

1  #ifndef RESNET_H
2  #define RESNET_H
3  // resnet.h — pure C++ (Eigen + OpenMP) inference of the two-headed ResNet
4  // trained by nnue/cnn_resnet_value_policy_chunked.py.
5  //
6  // Architecture (see the Python file for the training-side definition):
7  //   input (4 planes, 8x8) — 3x3 conv → BN → ReLU           (stem, BN folded)

```

```

8 // - B × [ BN → ReLU → 3x3 conv → BN → ReLU → 3x3 conv, + skip ]
9 // - BN → ReLU (trunk output)
10 //   └ value : 1x1 conv → BN → ReLU → flatten(64) → FC → ReLU → FC → tanh
11 //   └ policy: 1x1 conv → BN → ReLU → flatten(128) → FC → 4096 logits
12 //
13 // Implementation notes
14 // -----
15 // • Activations are kept pixel-major ("NHWC": [position][pixel 0..63][channel])
16 //   so that the 3x3 convolution becomes im2col + one big GEMM per layer:
17 //       Out (64n × C) = Col (64n × 9C) · WT (9C × C)
18 //   Eigen's GEMM (with -march=native -mfma) runs close to peak on this shape.
19 //   The im2col copy itself is <5 % of the layer's cost.
20 // • BatchNorm is folded at export time (nnue/resnet_export.py): the stem and
21 //   the 1x1 head convs absorbed it into their weights; the pre-activation BNs
22 //   of the residual blocks become a per-channel affine map applied together
23 //   with the ReLU in a single pass over the activations.
24 // • Parallelism: one evaluate() call splits the batch into chunks of a few
25 //   positions and processes the chunks with an OpenMP team (each thread owns
26 //   its scratch buffers, Eigen runs single-threaded inside). The backend is
27 //   therefore most efficient with ONE search thread feeding it batches of
28 //   ~32-128 positions; several search threads should go through
29 //   nn::BatchingEvaluator so their requests are merged rather than nested.
30 // • Cost: ≈ 4.5 GFLOP per position for the 256×30 network – expect on the
31 //   order of 100 positions/s on a desktop CPU. The libtorch backend
32 //   (resnet_torch.h) is 20-50× faster on a GPU; this backend exists so the
33 //   player always works, and as the reference for correctness.
34 #include "nn_evaluator.h"
35 #include <Eigen/Core>
36 #include <string>
37 #include <vector>
38
39 namespace nn {
40
41 class ResNetCPU final : public Evaluator {
42 public:
43     using AlignedVec = std::vector<float, Eigen::aligned_allocator<float>>;
44
45     // weights_filename : .bin produced by resnet_export.py
46     // nb_threads       : OpenMP threads per evaluate() call (0 = OpenMP default)
47     // preferred_batch  : batch size hint reported to the search
48     explicit ResNetCPU(const std::string& weights_filename, int nb_threads = 0,
49                       size_t preferred_batch = 32);
50     ~ResNetCPU() override;
51
52     void evaluate(std::span<const Request> in, std::span<Result> out) override;
53     size_t preferred_batch_size() const override { return preferred_batch; }
54     std::string info() const override;
55     void reload(const std::string& weights_filename) override;
56
57     // Reference forward used by the self-test: `planes` holds n × 256 floats
58     // (NCHW, see encode_planes); writes n values and n × NUM_ACTIONS logits.
59     void forward(const float* planes, size_t n, float* values, float* logits) const;
60
61     int channels() const { return C; }
62     int nb_blocks() const { return B; }
63     int nb_threads() const { return threads; }
64
65 private:
66     struct Block {
67         AlignedVec bn1_scale, bn1_shift, conv1_w; // conv: (C, 9, C) row-major
68         AlignedVec bn2_scale, bn2_shift, conv2_w;
69     };
70     struct Scratch; // per-thread work buffers, defined in resnet.cpp

```

```

71
72 void load(const std::string& weights_filename);
73 // Runs the trunk + heads on  $n \leq \text{max\_chunk}$  positions. Writes  $n$  values and
74 // the  $n \times 128$  policy feature vectors (input of the last policy FC).
75 void forward_chunk(Scratch& s, const float* planes, size_t n,
76                    float* values, float* policy_vec) const;
77 // Policy logit of action index  $a$  given the 128-d policy feature vector.
78 float policy_logit(const float* policy_vec, int a) const;
79
80 std::string filename;
81 int threads;
82 size_t preferred_batch;
83 static constexpr size_t MAX_CHUNK = 8; // positions per GEMM (64 rows each)
84
85 int C = 0, B = 0, F = 0, A = 0; // channels, blocks, value fc size, actions
86 AlignedVec stem_w, stem_b; // (C, 9, 4), (C)
87 std::vector<Block> blocks;
88 AlignedVec out_scale, out_shift; // trunk output BN
89 AlignedVec vconv_w, vconv_b; // (C), (1)
90 AlignedVec vfc1_w, vfc1_b; // (F, 64), (F)
91 AlignedVec vfc2_w, vfc2_b; // (F), (1)
92 AlignedVec pconv_w, pconv_b; // (2, C), (2)
93 AlignedVec pfc_w, pfc_b; // (A, 128), (A)
94 };
95
96 } // namespace nn
97
98 #endif

```

11.2.6 nnue/resnet.cpp

299 lines.

The inference itself. The shape to keep in mind is $\text{Out}_{64n \times C} = \text{Col}_{64n \times 9C} \cdot W_{9C \times C}^T$: every 3×3 convolution is one GEMM, and the pre-activation BatchNorm and ReLU are applied while the `im2col` patches are gathered, so the activations are read once per layer.

`flush_denormals` is not an optimisation, it is a correctness-of-performance fix: the trained network produces activations around 10^{-41} , and denormal arithmetic is $\sim 100\times$ slower on x86. FTZ/DAZ are per-thread MXCSR flags, so every OpenMP worker sets them itself rather than relying on link flags the LTO link line does not carry.

symbol	role
<code>TapTable</code>	precomputed source pixel for each (output pixel, kernel tap), -1 outside the board
<code>Reader</code>	little-endian reader for the <code>.bin</code> format documented in <code>resnet_export.py</code>
<code>bn_relu</code>	$y = \max(0, s_c x + t_c)$ over a row-major block — the folded pre-activation BN
<code>im2col</code>	gather the nine taps of every output pixel into one GEMM operand row
<code>flush_denormals</code>	set FTZ/DAZ on the calling thread
<code>ResNetCPU::load</code>	read the header, then every folded tensor in export order
<code>ResNetCPU::forward_chunk</code>	stem, B residual blocks, output BN, then both heads

symbol	role
<code>ResNetCPU::evaluate</code>	split the batch into chunks, run them under OpenMP, fill in only the legal logits

```

1 // resnet.cpp – see resnet.h for the design overview.
2 #define EIGEN_DONT_PARALLELIZE // we parallelise over positions ourselves (OpenMP)
3 #include "resnet.h"
4 #include <Eigen/Dense>
5 #include <algorithm>
6 #include <cmath>
7 #include <cstdio>
8 #include <cstring>
9 #include <sstream>
10 #include <stdexcept>
11 #include <omp.h>
12 #include <pmmmintrin.h>
13 #include <xmmmintrin.h>
14
15 namespace nn {
16
17 namespace {
18
19 using RowMat = Eigen::Matrix<float, Eigen::Dynamic, Eigen::Dynamic, Eigen::RowMajor>;
20 using MapRow = Eigen::Map<RowMat, Eigen::Aligned32>;
21 using MapConstRow = Eigen::Map<const RowMat, Eigen::Aligned32>;
22
23 // SRC[p][tap] = source pixel of output pixel p for kernel tap (kh, kw),
24 // or -1 when the tap falls outside the board (zero padding).
25 // p = 8*row + col, tap = 3*kh + kw, kh/kw in 0..2.
26 struct TapTable {
27     int src[64][9];
28     TapTable() {
29         for (int p = 0; p < 64; p++) {
30             const int r = p / 8, c = p % 8;
31             for (int tap = 0; tap < 9; tap++) {
32                 const int rr = r + tap / 3 - 1, cc = c + tap % 3 - 1;
33                 src[p][tap] = (rr >= 0 && rr < 8 && cc >= 0 && cc < 8) ? rr * 8 + cc : -1;
34             }
35         }
36     }
37 };
38 const TapTable TAPS;
39
40 // Little-endian binary reader for the .bin format documented in resnet_export.py.
41 struct Reader {
42     FILE* f;
43     std::string name;
44     explicit Reader(const std::string& filename) : f(std::fopen(filename.c_str(), "rb")),
45         ↪ name(filename) {
46         if (!f) throw std::runtime_error("ResNetCPU: cannot open " + filename);
47     }
48     ~Reader() { if (f) std::fclose(f); }
49     uint32_t u32() {
50         uint32_t v;
51         if (std::fread(&v, 4, 1, f) != 1) throw std::runtime_error("ResNetCPU: truncated file " +
52             ↪ name);
53         return v;
54     }
55     void floats(ResNetCPU::AlignedVec& dst, size_t n) {
56         dst.resize(n);
57         if (std::fread(dst.data(), sizeof(float), n, f) != n)

```

```

56         throw std::runtime_error("ResNetCPU: truncated file " + name);
57     }
58 };
59
60 // y = max(0, scale[c] * x + shift[c]) over a (rows × C) row-major block.
61 inline void bn_relu(const float* __restrict x, float* __restrict y, size_t rows, int C,
62                   const float* __restrict scale, const float* __restrict shift) {
63     for (size_t r = 0; r < rows; r++) {
64         const float* xr = x + r * C;
65         float* yr = y + r * C;
66         for (int c = 0; c < C; c++) {
67             yr[c] = std::max(0.0f, scale[c] * xr[c] + shift[c]);
68         }
69     }
70 }
71
72 // Col[(b*64 + p) × (tap*C + ch)] = H[b][src(p, tap)][ch] (0 when padded).
73 inline void im2col(const float* __restrict h, float* __restrict col, size_t n, int C) {
74     const size_t row_len = 9 * static_cast<size_t>(C);
75     for (size_t b = 0; b < n; b++) {
76         const float* hb = h + b * 64 * C;
77         float* cb = col + b * 64 * row_len;
78         for (int p = 0; p < 64; p++) {
79             float* crow = cb + p * row_len;
80             for (int tap = 0; tap < 9; tap++) {
81                 float* dst = crow + tap * C;
82                 const int src = TAPS.src[p][tap];
83                 if (src < 0) std::memset(dst, 0, C * sizeof(float));
84                 else std::memcpy(dst, hb + src * C, C * sizeof(float));
85             }
86         }
87     }
88 }
89
90 // The trained network produces activations in the denormal range (1e-41 ...),
91 // and denormal arithmetic is ~100× slower on x86. Flush-to-zero / denormals-
92 // are-zero are per-thread MXCSR flags, so set them on every worker thread of
93 // the parallel region instead of relying on the link flags (-ffast-math adds
94 // crtfastmath.o, but only for the main thread and only if it is on the link
95 // line – the CMake LTO link line does not have it).
96 inline void flush_denormals() {
97     _MM_SET_FLUSH_ZERO_MODE(_MM_FLUSH_ZERO_ON);
98     _MM_SET_DENORMALS_ZERO_MODE(_MM_DENORMALS_ZERO_ON);
99 }
100
101 } // namespace
102
103 // Per-thread scratch: activations X, H, Y (rows × C), im2col buffer (rows × 9C),
104 // the stem im2col (rows × 36) and the head maps. Grown lazily; thread_local.
105 struct ResNetCPU::Scratch {
106     int C = 0;
107     size_t rows = 0;
108     AlignedVec X, H, Y, Col, Col0, VMap, PMap, V64, HV;
109     void ensure(int channels, size_t n_positions, int F) {
110         const size_t need = n_positions * 64;
111         if (channels == C && need <= rows) return;
112         C = channels; rows = std::max(rows, need);
113         X.assign(rows * C, 0.f); H.assign(rows * C, 0.f); Y.assign(rows * C, 0.f);
114         Col.assign(rows * 9 * C, 0.f);
115         Col0.assign(rows * 36, 0.f);
116         VMap.assign(rows, 0.f); PMap.assign(rows * 2, 0.f);
117         V64.assign(64, 0.f); HV.assign(F, 0.f);
118     }

```

```

119 };
120
121 ResNetCPU::ResNetCPU(const std::string& weights_filename, int nb_threads, size_t preferred_batch)
122 : filename(weights_filename), threads(nb_threads), preferred_batch(preferred_batch) {
123     load(weights_filename);
124 }
125
126 ResNetCPU::~ResNetCPU() = default;
127
128 void ResNetCPU::load(const std::string& weights_filename) {
129     Reader r(weights_filename);
130     char magic[8];
131     if (std::fread(magic, 1, 8, r.f) != 8 || std::memcmp(magic, "YOLAHRSN", 8) != 0)
132         throw std::runtime_error("ResNetCPU: bad magic in " + weights_filename +
133             " (expected a file written by resnet_export.py)");
134     const uint32_t version = r.u32();
135     if (version != 1) throw std::runtime_error("ResNetCPU: unsupported version " +
136         std::to_string(version));
137     const int in_channels = static_cast<int>(r.u32());
138     if (in_channels != PLANES) throw std::runtime_error("ResNetCPU: expected 4 input planes");
139     const int c = static_cast<int>(r.u32()), b = static_cast<int>(r.u32());
140     const int f = static_cast<int>(r.u32()), a = static_cast<int>(r.u32());
141     if (C != 0 && (c != C || b != B || f != F || a != A))
142         throw std::runtime_error("ResNetCPU::reload: architecture mismatch");
143     if (a != NUM_ACTIONS) throw std::runtime_error("ResNetCPU: expected 4096 actions");
144     C = c; B = b; F = f; A = a;
145
146     r.floats(stem_w, static_cast<size_t>(C) * 9 * PLANES);
147     r.floats(stem_b, C);
148     blocks.assign(B, Block{});
149     for (Block& blk : blocks) {
150         r.floats(blk.bn1_scale, C); r.floats(blk.bn1_shift, C);
151         r.floats(blk.conv1_w, static_cast<size_t>(C) * 9 * C);
152         r.floats(blk.bn2_scale, C); r.floats(blk.bn2_shift, C);
153         r.floats(blk.conv2_w, static_cast<size_t>(C) * 9 * C);
154     }
155     r.floats(out_scale, C); r.floats(out_shift, C);
156     r.floats(vconv_w, C); r.floats(vconv_b, 1);
157     r.floats(vfc1_w, static_cast<size_t>(F) * 64); r.floats(vfc1_b, F);
158     r.floats(vfc2_w, F); r.floats(vfc2_b, 1);
159     r.floats(pconv_w, 2 * static_cast<size_t>(C)); r.floats(pconv_b, 2);
160     r.floats(pfc_w, static_cast<size_t>(A) * 128); r.floats(pfc_b, A);
161     // Anything left over means the exporter and this reader disagree.
162     char probe;
163     if (std::fread(&probe, 1, 1, r.f) == 1)
164         throw std::runtime_error("ResNetCPU: trailing bytes in " + weights_filename);
165     filename = weights_filename;
166 }
167
168 void ResNetCPU::reload(const std::string& weights_filename) {
169     load(weights_filename);
170 }
171
172 std::string ResNetCPU::info() const {
173     std::ostringstream os;
174     os << "ResNetCPU " << C << "x" << B << " (" << filename << ", "
175         << (threads ? std::to_string(threads) : std::string("omp default")) << " threads)";
176     return os.str();
177 }
178
179 float ResNetCPU::policy_logit(const float* policy_vec, int a) const {
180     const float* w = pfc_w.data() + static_cast<size_t>(a) * 128;
181     float s = pfc_b[a];

```

```

181     for (int k = 0; k < 128; k++) s += w[k] * policy_vec[k];
182     return s;
183 }
184
185 void ResNetCPU::forward_chunk(Scratch& s, const float* planes, size_t n,
186                               float* values, float* policy_vec) const {
187     s.ensure(C, std::max(n, MAX_CHUNK), F);
188     const size_t rows = n * 64;
189
190     // — stem: im2col over the 4 input planes (NCHW → pixel-major 36-wide rows) —
191     for (size_t b = 0; b < n; b++) {
192         const float* pl = planes + b * INPUT_FLOATS;
193         for (int p = 0; p < 64; p++) {
194             float* crow = s.Col0.data() + (b * 64 + p) * 36;
195             for (int tap = 0; tap < 9; tap++) {
196                 const int src = TAPS.src[p][tap];
197                 for (int c = 0; c < PLANES; c++) {
198                     crow[tap * PLANES + c] = src < 0 ? 0.f : pl[c * 64 + src];
199                 }
200             }
201         }
202     }
203     MapRow X(s.X.data(), rows, C);
204     {
205         MapConstRow col0(s.Col0.data(), rows, 36);
206         MapConstRow w(stem_w.data(), C, 36);
207         X.noalias() = col0 * w.transpose();
208         X.rowwise() += Eigen::Map<const Eigen::RowVectorXf>(stem_b.data(), C);
209         X = X.cwiseMax(0.f); // the stem ends with a ReLU
210     }
211
212     // — residual tower —
213     MapRow Y(s.Y.data(), rows, C);
214     MapConstRow col(s.Col.data(), rows, 9 * C);
215     for (const Block& blk : blocks) {
216         bn_relu(s.X.data(), s.H.data(), rows, C, blk.bn1_scale.data(), blk.bn1_shift.data());
217         im2col(s.H.data(), s.Col.data(), n, C);
218         MapConstRow w1(blk.conv1_w.data(), C, 9 * C);
219         Y.noalias() = col * w1.transpose();
220         bn_relu(s.Y.data(), s.H.data(), rows, C, blk.bn2_scale.data(), blk.bn2_shift.data());
221         im2col(s.H.data(), s.Col.data(), n, C);
222         MapConstRow w2(blk.conv2_w.data(), C, 9 * C);
223         X.noalias() += col * w2.transpose(); // skip connection
224     }
225     bn_relu(s.X.data(), s.H.data(), rows, C, out_scale.data(), out_shift.data());
226     MapConstRow H(s.H.data(), rows, C);
227
228     // — heads —
229     // 1x1 convs are plain matrix products over the pixel rows.
230     Eigen::Map<Eigen::VectorXf> vmap(s.VMap.data(), rows);
231     vmap.noalias() = H * Eigen::Map<const Eigen::VectorXf>(vconv_w.data(), C);
232     MapRow pmap(s.PMap.data(), rows, 2);
233     pmap.noalias() = H * MapConstRow(pconv_w.data(), 2, C).transpose();
234
235     Eigen::Map<const RowMat> Wfc1(vfc1_w.data(), F, 64);
236     Eigen::Map<const Eigen::VectorXf> bfc1(vfc1_b.data(), F);
237     Eigen::Map<const Eigen::VectorXf> wfc2(vfc2_w.data(), F);
238     Eigen::Map<Eigen::VectorXf> v64(s.V64.data(), 64);
239     Eigen::Map<Eigen::VectorXf> hv(s.HV.data(), F);
240     for (size_t b = 0; b < n; b++) {
241         // value: relu(conv+bias) over the 64 pixels → FC → ReLU → FC → tanh
242         for (int p = 0; p < 64; p++) v64[p] = std::max(0.f, s.VMap[b * 64 + p] + vconv_b[0]);
243         hv.noalias() = Wfc1 * v64 + bfc1;

```

```

244     hv = hv.cwiseMax(0.f);
245     values[b] = std::tanh(wfc2.dot(hv) + vfc2_b[0]);
246     // policy: relu(conv+bias), flattened channel-major (c*64 + p) as in torch's flatten
247     float* pv = policy_vec + b * 128;
248     for (int p = 0; p < 64; p++) {
249         pv[p] = std::max(0.f, s.PMap[(b * 64 + p) * 2 + 0] + pconv_b[0]);
250         pv[64 + p] = std::max(0.f, s.PMap[(b * 64 + p) * 2 + 1] + pconv_b[1]);
251     }
252 }
253 }
254
255 void ResNetCPU::forward(const float* planes, size_t n, float* values, float* logits) const {
256     const size_t nb_chunks = (n + MAX_CHUNK - 1) / MAX_CHUNK;
257     #pragma omp parallel for schedule(dynamic, 1) num_threads(threads > 0 ? threads :
    ↪ omp_get_max_threads())
258     for (size_t k = 0; k < nb_chunks; k++) {
259         thread_local Scratch scratch;
260         flush_denormals();
261         const size_t start = k * MAX_CHUNK, cnt = std::min(MAX_CHUNK, n - start);
262         float pvec[MAX_CHUNK * 128];
263         forward_chunk(scratch, planes + start * INPUT_FLOATS, cnt, values + start, pvec);
264         for (size_t b = 0; b < cnt; b++) {
265             float* out = logits + (start + b) * NUM_ACTIONS;
266             for (int a = 0; a < NUM_ACTIONS; a++) out[a] = policy_logit(pvec + b * 128, a);
267         }
268     }
269 }
270
271 void ResNetCPU::evaluate(std::span<const Request> in, std::span<Result> out) {
272     const size_t n = in.size();
273     if (out.size() < n) throw std::invalid_argument("ResNetCPU::evaluate: output span too small");
274     // Chunk so that every OpenMP thread gets work even for small batches.
275     const int nthreads = threads > 0 ? threads : omp_get_max_threads();
276     const size_t chunk = std::clamp<size_t>(n / static_cast<size_t>(nthreads), 1, MAX_CHUNK);
277     const size_t nb_chunks = (n + chunk - 1) / chunk;
278     #pragma omp parallel for schedule(dynamic, 1) num_threads(nthreads)
279     for (size_t k = 0; k < nb_chunks; k++) {
280         thread_local Scratch scratch;
281         flush_denormals();
282         const size_t start = k * chunk, cnt = std::min(chunk, n - start);
283         float planes[MAX_CHUNK * INPUT_FLOATS];
284         float values[MAX_CHUNK];
285         float pvec[MAX_CHUNK * 128];
286         for (size_t b = 0; b < cnt; b++) encode_planes(*in[start + b].state, planes + b *
    ↪ INPUT_FLOATS);
287         forward_chunk(scratch, planes, cnt, values, pvec);
288         for (size_t b = 0; b < cnt; b++) {
289             const Request& rq = in[start + b];
290             Result& rs = out[start + b];
291             rs.value = values[b];
292             for (uint16_t m = 0; m < rq.nb_moves; m++) {
293                 rs.logits[m] = policy_logit(pvec + b * 128, action_index(rq.moves[m]));
294             }
295         }
296     }
297 }
298
299 } // namespace nn

```

11.2.7 nnue/resnet_torch.h

48 lines.

The libtorch backend's declaration, compiled only under `-DENABLE_TORCH=ON`.

symbol	role
ResNetTorch	Evaluator over the TorchScript module, on CUDA or CPU, optionally in fp16

```
1  #ifndef RESNET_TORCH_H
2  #define RESNET_TORCH_H
3  // resnet_torch.h – libtorch backend: runs the TorchScript module exported by
4  // nnue/resnet_export.py (<name>.ts) on the GPU (or CPU) through libtorch.
5  //
6  // Only compiled when the project is configured with -DENABLE_TORCH=ON; the
7  // build then needs a libtorch install (or the one that ships inside a pip
8  // torch package: -DCMAKE_PREFIX_PATH=<site-packages>/torch).
9  //
10 // The whole batch is one forward pass: (n, 4, 8, 8) → (n,), (n, 4096).
11 // In half precision on an RTX-class GPU the 256x30 network evaluates several
12 // thousand positions per second at batch 256, i.e. 20-50× the CPU backend.
13 #ifdef ENABLE_TORCH
14 #include "nn_evaluator.h"
15 #include <torch/script.h>
16 #include <torch/cuda.h>
17 #include <mutex>
18 #include <string>
19
20 namespace nn {
21
22 class ResNetTorch final : public Evaluator {
23 public:
24     // device: "cuda", "cuda:1", "cpu" ... ; fp16: run the network in half precision
25     ResNetTorch(const std::string& module_filename, const std::string& device = "cuda",
26                 bool fp16 = true, size_t preferred_batch = 256);
27
28     void evaluate(std::span<const Request> in, std::span<Result> out) override;
29     size_t preferred_batch_size() const override { return preferred_batch; }
30     std::string info() const override;
31     void reload(const std::string& module_filename) override;
32
33 private:
34     void load(const std::string& module_filename);
35
36     std::string filename;
37     torch::Device device;
38     bool fp16;
39     size_t preferred_batch;
40     torch::jit::script::Module module;
41     std::mutex mutex;           // one forward at a time (callers are batched anyway)
42     torch::Tensor input_cpu;   // pinned staging buffer, grown on demand
43 };
44
45 } // namespace nn
46
47 #endif // ENABLE_TORCH
48 #endif
```

11.2.8 nnue/resnet_torch.cpp

87 lines.

Load the traced module, stage the batch into a pinned CPU tensor, one forward, read the two outputs back. The mutex serialises forwards — callers are batched upstream anyway.

symbol	role
ResNetTorch::load	torch::jit::load, move to the device, eval(), optionally to(half)
ResNetTorch::evaluate	encode into the staging tensor, forward, scatter the legal logits back

```
1  #ifdef ENABLE_TORCH
2  #include "resnet_torch.h"
3  #include <sstream>
4  #include <stdexcept>
5
6  namespace nn {
7
8  ResNetTorch::ResNetTorch(const std::string& module_filename, const std::string& device_name,
9                          bool fp16, size_t preferred_batch)
10     : filename(module_filename),
11       device(device_name == "cuda" && !torch::cuda::is_available() ? torch::Device(torch::kCPU)
12                                                         : torch::Device(device_name)),
13       fp16(fp16 && device.is_cuda()), // half precision only makes sense on the GPU
14       preferred_batch(preferred_batch) {
15     load(module_filename);
16 }
17
18 void ResNetTorch::load(const std::string& module_filename) {
19     std::lock_guard lock(mutex);
20     // The default profiling executor re-specialises the graph for every new
21     // batch shape (several seconds each, and the search uses many shapes):
22     // use the plain executor. These are process-wide flags and must be set
23     // before the module is loaded (a module frozen/optimised under the other
24     // executor cannot be run by this one).
25     torch::jit::getProfilingMode() = false;
26     torch::jit::setGraphExecutorOptimize(false);
27     try {
28         module = torch::jit::load(module_filename, device);
29     } catch (const c10::Error& e) {
30         throw std::runtime_error("ResNetTorch: cannot load " + module_filename + ": " +
31                                   ↪ e.what_without_backtrace());
32     }
33     module.eval();
34     if (fp16) module.to(torch::kHalf);
35     filename = module_filename;
36     // Warm up cuDNN / the executor on the shapes the search is likely to use.
37     torch::NoGradGuard no_grad;
38     for (int64_t n : {int64_t(1), int64_t(8), int64_t(32), int64_t(128),
39 ↪ int64_t(preferred_batch)}) {
40         torch::Tensor x = torch::zeros({n, PLANES, 8, 8}, torch::TensorOptions().dtype(fp16 ?
41 ↪ torch::kHalf : torch::kFloat32).device(device));
42         module.forward({x});
43     }
44     if (device.is_cuda()) torch::cuda::synchronize();
45 }
46
47 void ResNetTorch::reload(const std::string& module_filename) {
```

```

45     load(module_filename);
46 }
47
48 std::string ResNetTorch::info() const {
49     std::ostringstream os;
50     os << "ResNetTorch (" << filename << ", " << device << (fp16 ? ", fp16" : ", fp32") << ")";
51     return os.str();
52 }
53
54 void ResNetTorch::evaluate(std::span<const Request> in, std::span<Result> out) {
55     const int64_t n = static_cast<int64_t>(in.size());
56     if (n == 0) return;
57     if (out.size() < in.size()) throw std::invalid_argument("ResNetTorch::evaluate: output span
58     ↪ too small");
59     std::lock_guard lock(mutex);
60     torch::NoGradGuard no_grad;
61     cl0::InferenceMode inference;
62
63     if (!input_cpu.defined() || input_cpu.size(0) < n) {
64         auto options =
65         ↪ torch::TensorOptions().dtype(torch::kFloat32).pinned_memory(device.is_cuda());
66         input_cpu = torch::empty({n, PLANES, 8, 8}, options);
67     }
68     float* dst = input_cpu.data_ptr<float>();
69     for (int64_t i = 0; i < n; i++) encode_planes(*in[i].state, dst + i * INPUT_FLOATS);
70
71     torch::Tensor x = input_cpu.narrow(0, 0, n).to(device, fp16 ? torch::kHalf : torch::kFloat32,
72     ↪ /*non_blocking=*/true);
73
74     auto outputs = module.forward({x}).toTuple();
75     // Both heads back to the host in fp32, one copy each.
76     torch::Tensor values = outputs->elements()[0].toTensor().to(torch::kCPU,
77     ↪ torch::kFloat32).contiguous();
78     torch::Tensor logits = outputs->elements()[1].toTensor().to(torch::kCPU,
79     ↪ torch::kFloat32).contiguous();
80
81     const float* v = values.data_ptr<float>();
82     const float* p = logits.data_ptr<float>();
83     for (int64_t i = 0; i < n; i++) {
84         const Request& rq = in[i];
85         Result& rs = out[i];
86         rs.value = v[i];
87         const float* row = p + i * NUM_ACTIONS;
88         for (uint16_t m = 0; m < rq.nb_moves; m++) rs.logits[m] = row[action_index(rq.moves[m])];
89     }
90 }
91
92 } // namespace nn
93 #endif // ENABLE_TORCH

```

11.3 Training and export

The Python half, `cnn_resnet_value_policy_chunked.py` is the script the shipped weights were trained with and the one `resnet_export.py` imports; its sibling `cnn_resnet_value_policy.py` defines the *same* network and differs only in the data pipeline (a stock `DataLoader` with worker processes, which is the right choice when the cache fits in page cache).

11.3.1 `nnue/cnn_resnet_value_policy_chunked.py`

838 lines.

The network definition, the encoding, and the training loop.

The loader is the part that is specific to this project rather than to AlphaZero. The position cache is a ~253 GB file; sampling it uniformly at random turns training into millions of scattered 256-byte reads, which on a spinning disk is seek-bound. **ChunkedShuffleLoader** reads ~1 GB **contiguous** chunks sequentially, shuffles within a chunk and reshuffles the chunk order each epoch — an approximate shuffle, but a chunk holds ~4 M positions, and sequential reads are fast without pre-warming 253 GB of RAM. One background thread per rank does the reading and batch construction while the main thread drives the GPU; the overlap is real because numpy's memcopy and CUDA's pin-memory both release the GIL.

The DDP detail that bites: every rank must yield *exactly* the same number of batches, or the first rank to finish hangs the others on the gradient all-reduce. Hence the floor division in the chunk sharding and the dropped trailing batches.

symbol	role
<code>_bitboard_to_plane, encode_cnn</code>	the Python side of Section 3 — must match <code>nn::encode_planes</code>
<code>ChunkedShuffleLoader</code>	sequential chunked reader over the memmap cache, double-buffered, DDP-sharded
<code>ChunkedShuffleLoader._producer</code>	the background thread: read a chunk, shuffle it, emit pinned batches
<code>ResBlock</code>	pre-activation residual block (Figure 4)
<code>Net</code>	stem, B blocks, output BN, value head and policy head (Figure 3)
<code>Net.forward</code>	returns <code>(value, policy_logits)</code>
<code>TrainerDDP</code>	one process per GPU: channels-last, AMP, <code>torch.compile</code> , cosine LR
<code>TrainerDDP._compute_loss</code>	$\lambda_v \text{MSE}(v, z) + \lambda_p \text{CE}(\ell, a^*)$
<code>TrainerDDP._run_epoch / _validate</code>	the loop, with value sign accuracy and policy top-1 as the readable metrics
<code>main</code>	contiguous train/val split, build the loaders inside the spawned process, train

```

1  """
2  cnn_resnet_value_policy_chunked.py
3  =====
4  AlphaZero-style two-headed ResNet for the game of Yohai, trained with a
5  custom double-buffered "chunked-shuffle" data loader.
6
7  It is functionally the same network as cnn_resnet_value_policy.py – the ONLY
8  difference is the data pipeline:
9
10     cnn_resnet_value_policy.py           torch DataLoader + 8 worker processes,
11                                         random-access reads into the memmap cache.
12     cnn_resnet_value_policy_chunked.py (this file) one background thread per
13                                         rank, large sequential chunk reads.
14
15  Why the chunked loader exists
16  -----
17  The cache (produced by preprocess.py) is a ~253 GB file of encoded positions.
18  A standard DataLoader samples positions uniformly at random, which becomes
19  millions of random ~256-byte reads scattered across that file. On an
20  HDD-backed filesystem random reads are seek-bound and collapse throughput;
21  the only rescue is pre-loading the whole 253 GB into RAM page cache.
22

```

```

23 This loader instead:
24 • reads large CONTIGUOUS chunks (~1 GB) *sequentially* – sequential reads
25   are fast even on spinning disks, so no RAM pre-warm is needed;
26 • shuffles positions WITHIN a chunk and reshuffles the chunk ORDER each
27   epoch. This is an approximate shuffle, but a chunk holds ~4M positions
28   (~80k games), so any one batch still mixes thousands of games – gradients
29   stay decorrelated;
30 • does the chunk read + batch construction in ONE background thread while
31   the main thread runs the GPU. A bounded queue between them is the
32   rotating "ping-pong" buffer. This overlap works because numpy's memcopy /
33   dtype-convert and CUDA's pin-memory all release the Python GIL, so the
34   helper thread genuinely runs concurrently with GPU compute;
35 • hands the main loop batch tensors that are already pinned, so the
36   host-GPU copy can be asynchronous.
37
38 Everything below the loader – the network, the loss, mixed precision,
39 channels-last memory format, DistributedDataParallel – is standard and
40 identical to the sibling script.
41 """
42
43 # — Imports —————
44 from tqdm import tqdm                # progress bar around the training loop
45 import torch
46 from torch import nn
47 import torch.multiprocessing as mp   # mp.spawn launches one process per GPU
48 from torch.nn.parallel import DistributedDataParallel as DDP
49 from torch.distributed import init_process_group, destroy_process_group
50 import os
51 import sys
52 import gc                            # disabled inside main() to avoid GC pauses
53 import json                          # the cache's meta.json sidecar
54 import random                        # epoch-level chunk-order shuffle
55 import threading                     # the background producer thread
56 import queue as queue_mod            # bounded hand-off queue (the buffer)
57 import numpy as np
58
59 # The position encoder needs the Yoloh game module, which lives in ../server.
60 sys.path.append("../server")
61 from yoloh import Yoloh, Move, Square
62
63 # Allow TF32 matmuls on Ampere+ (a speed/precision trade that is safe here).
64 torch.set_float32_matmul_precision('high')
65
66
67 # — Board encoding —————
68 #
69 # Every position is a (4, 8, 8) float tensor – four 8x8 "planes":
70 #   plane 0 – black pieces : 1 where a black stone sits, else 0
71 #   plane 1 – white pieces : 1 where a white stone sits, else 0
72 #   plane 2 – empty squares : 1 where a square has been vacated (scored), else 0
73 #   plane 3 – turn          : a constant plane, all-0 if black is to move,
74 #                           all-1 if white is to move
75 # (preprocess.py writes this same layout to disk as uint8.)
76
77 # Policy action space: a Yoloh move is (from_square, to_square), each in 0..63.
78 # We flatten it to a single index  action = from*64 + to → 64*64 = 4096 logits.
79 NUM_ACTIONS = 64 * 64
80
81 # A terminal position has no "next move" to predict. Its policy target is set
82 # to this sentinel, and CrossEntropyLoss(ignore_index=...) skips those rows.
83 POLICY_IGNORE_INDEX = -1
84
85 # Value target z ∈ {-1, 0, +1}, from the CURRENT player's point of view:

```

```

86 # +1 current player went on to win
87 # 0 draw
88 # -1 current player lost
89 # The network's value head ends in tanh, so its output  $v \in (-1, +1)$ ; the
90 # value loss is  $\text{MSE}(v, z)$ .
91
92 # Number of input planes – must match what preprocess.py wrote (a guard in the
93 # loader checks this against meta.json).
94 IN_CHANNELS = 4
95
96
97 def _bitboard_to_plane(n: int) -> np.ndarray:
98     """
99     Expand a 64-bit Yolah bitboard into an 8x8 float32 plane.
100
101     The flattened plane is filled MSB-first: array cell  $i$  receives bit
102      $(63 - i)$  of `n`, so cell 0 holds bit 63 and cell 63 holds bit 0.
103
104     Note this is the reverse of Yolah's own square numbering – in yolah.py
105     square  $s$  is bit  $s$ , i.e. square 0 is the LEAST-significant bit – so the
106     plane ends up in reverse-square order. That is harmless: a CNN does not
107     care about the absolute orientation of the 8x8 plane. What matters is
108     that this matches preprocess.py's bb_to_plane (np.unpackbits on a
109     big-endian word, which yields the same MSB-first order), so the inference
110     encoder and the cached training tensors agree. Do not "fix" the order
111     here without regenerating the cache.
112
113     Used only by encode_cnn() for inference; training reads pre-encoded
114     planes straight from the cache.
115     """
116     b = np.zeros(64, dtype=np.float32)
117     for i in range(64):
118         if n & (1 << (63 - i)):
119             b[i] = 1.0
120     return b.reshape(8, 8)
121
122
123 def encode_cnn(yolah) -> torch.Tensor:
124     """
125     Encode a live Yolah position into the (4, 8, 8) float32 tensor the network
126     expects. This is the INFERENCE-time encoder (e.g. for MCTS); training does
127     not call it – preprocess.py already baked these tensors into the cache.
128     """
129     black = _bitboard_to_plane(yolah.black)
130     white = _bitboard_to_plane(yolah.white)
131     empty = _bitboard_to_plane(yolah.empty)
132     # Black moves on even plies (0, 2, 4, ...), white on odd plies.
133     black_to_move = (yolah.nb_plies() & 1) == 0
134     turn = np.full((8, 8), 0.0 if black_to_move else 1.0, dtype=np.float32)
135     return torch.from_numpy(np.stack([black, white, empty, turn]))
136
137
138 # — Chunked-shuffle double-buffered loader —————
139 #
140 # CHUNK_SIZE – number of positions read per chunk.
141 # 4194304 = 2048 * 2048. Two deliberate properties:
142 # • it is an exact multiple of the batch size (2048) → every chunk yields
143 #   exactly 2048 whole batches, no ragged tail;
144 # • at 256 bytes/position it is ~1 GB of uint8 – small enough that a few
145 #   chunks fit comfortably in RAM, large enough to span ~80k games so the
146 #   within-chunk shuffle decorrelates batches well.
147 CHUNK_SIZE = 2048 * 2048
148

```

```

149 # BATCH_QUEUE_DEPTH – how many fully-built, pinned batches the producer thread
150 # may run ahead of the GPU. This is the depth of the ping-pong buffer: bigger
151 # = more slack to absorb a slow chunk read, but more pinned RAM held at once
152 # (each batch ≈ 2 MB → 32 batches ≈ 64 MB per rank).
153 BATCH_QUEUE_DEPTH = 32
154
155
156 class ChunkedShuffleLoader:
157     """
158     Streaming data loader over the memmap cache produced by preprocess.py.
159
160     It is a drop-in iterable for the trainer:
161     • iterating it yields (state, value_target, policy_target) tensors,
162       one batch at a time, already pinned for async host→GPU copy;
163     • len(loader) is the number of batches in one epoch;
164     • set_epoch(e) reseeds the shuffling so each epoch differs.
165
166     Distributed training (DDP):
167     With `world_size` GPUs, rank r reads a DISJOINT, strided subset of the
168     chunks (chunks r, r+world_size, r+2*world_size, ...). Remainder chunks
169     and any partial trailing batch are dropped so EVERY rank yields exactly
170     the same number of batches – this is essential: each backward pass does
171     a collective all-reduce, and if one rank ran fewer steps the others
172     would hang waiting on it.
173     """
174
175     def __init__(self, cache_dir, lo, hi, batch_size, rank, world_size,
176                 chunk_size=CHUNK_SIZE, shuffle=True, pin_memory=True,
177                 queue_depth=BATCH_QUEUE_DEPTH):
178         # cache_dir – directory holding positions.u8 / values.i8 /
179         #             policies.il6 / meta.json
180         # [lo, hi) – the slice of the global dataset this loader covers
181         #             (the caller passes the train slice or the val slice)
182         # batch_size – positions per batch, PER GPU
183         # rank/world_size – this process's DDP rank and the total #GPUs
184         # shuffle – True for training, False for validation
185         # pin_memory – pin produced batches (page-locked RAM → async H2D copy)
186         self.cache_dir = cache_dir
187         self.batch_size = batch_size
188         self.rank = rank
189         self.world_size = world_size
190         self.chunk_size = chunk_size
191         self.shuffle = shuffle
192         self.pin_memory = pin_memory
193         self.queue_depth = queue_depth
194         self.epoch = 0
195         self._open() # open the memmaps
196
197         # Tile the half-open range [lo, hi) into whole, non-overlapping chunks.
198         # `all_starts` is the list of every chunk's starting position. The
199         # `hi - chunk_size + 1` upper bound discards a final partial chunk.
200         all_starts = list(range(lo, hi - chunk_size + 1, chunk_size))
201
202         # Shard the chunks across ranks. `per_rank` is floor-divided so every
203         # rank gets the SAME count; up to (world_size - 1) leftover chunks are
204         # dropped. Rank r then takes a strided slice starting at index r.
205         per_rank = len(all_starts) // world_size
206         self.my_chunks = all_starts[rank : per_rank * world_size : world_size]
207
208         # With CHUNK_SIZE an exact multiple of batch_size this division is
209         # clean; otherwise the trailing partial batch of each chunk is dropped.
210         self.batches_per_chunk = chunk_size // batch_size
211

```

```

212     # Total batches this rank yields per epoch – identical across ranks.
213     self.n_batches = len(self.my_chunks) * self.batches_per_chunk
214
215     def _open(self):
216         """
217         Open the three cache files as read-only memory maps.
218
219         np.memmap does not read any data here – it only maps the file into the
220         process's virtual address space. Bytes are paged in lazily by the OS as
221         the producer thread actually touches them, and the page cache is shared
222         by every process on the node (so 3 ranks do not triple RAM use).
223         """
224         with open(os.path.join(self.cache_dir, "meta.json")) as f:
225             meta = json.load(f)
226
227         # Fail loudly on a stale cache (e.g. a 6-plane cache from an older
228         # preprocess.py) rather than silently mis-reading bytes.
229         n_planes = meta["positions"]["shape"][1]
230         if n_planes != IN_CHANNELS:
231             raise ValueError(
232                 f"Cache has {n_planes} planes but training expects {IN_CHANNELS}. "
233                 "Re-run preprocess.py.")
234
235         n = int(meta["n_positions"])
236         # positions.u8 : (N, 4, 8, 8) uint8 – the board planes
237         self.positions = np.memmap(
238             os.path.join(self.cache_dir, meta["positions"]["path"]),
239             dtype=np.uint8, mode='r', shape=tuple(meta["positions"]["shape"]))
240         # values.i8 : (N,) int8 – value target z ∈ {-1, 0, +1}
241         self.values = np.memmap(
242             os.path.join(self.cache_dir, meta["values"]["path"]),
243             dtype=np.int8, mode='r', shape=(n,))
244         # policies.i16 : (N,) int16 – action index, or -1 if terminal
245         self.policies = np.memmap(
246             os.path.join(self.cache_dir, meta["policies"]["path"]),
247             dtype=np.int16, mode='r', shape=(n,))
248
249     def set_epoch(self, epoch):
250         """Reseed shuffling for a new epoch (chunk order + within-chunk perm)."""
251         self.epoch = epoch
252
253     def __len__(self):
254         """Number of batches per epoch – lets tqdm show a correct total."""
255         return self.n_batches
256
257     def _producer(self, chunk_order, q):
258         """
259         The background worker thread (one per loader, started by __iter__).
260
261         For each chunk, in the epoch's shuffled order:
262         1. read the whole chunk sequentially from the memmap into RAM;
263         2. build a permutation that shuffles positions within the chunk;
264         3. slice it into batches, convert dtypes, optionally pin, and push
265            each batch onto the bounded queue `q`.
266
267         The queue's `maxsize` is the back-pressure mechanism: once it holds
268         `queue_depth` batches, `q.put(...)` blocks, so this thread can run at
269         most that far ahead of the consumer and never grows memory unbounded.
270
271         Why a thread (not a process) overlaps the GPU: numpy's bulk memcopy /
272         astype and torch's pin_memory all drop the GIL for their heavy work,
273         and so does every CUDA call in the main thread – so the two threads
274         truly run at the same time.

```

```

275     """
276     bs = self.batch_size
277     try:
278         for chunk_id in chunk_order:
279             start = self.my_chunks[chunk_id]
280             end = start + self.chunk_size
281
282             # --- step 1: sequential read of the chunk into RAM -----
283             # np.array(mmap_slice) forces a contiguous copy, i.e. an
284             # actual sequential read of this region off disk.
285             pos = np.array(self.positions[start:end]) # (C, 4, 8, 8) uint8
286             val = np.array(self.values[start:end])    # (C,)          int8
287             pol = np.array(self.policies[start:end])  # (C,)          int16
288
289             # --- step 2: within-chunk shuffle -----
290             # Seed depends on both epoch and chunk start, so a given chunk
291             # is permuted differently every epoch. `& 0xFFFFFFFF` keeps the
292             # seed inside the 32-bit range numpy's Generator expects.
293             if self.shuffle:
294                 rng = np.random.default_rng(
295                     (self.epoch * 1_000_003 + start) & 0xFFFFFFFF)
296                 perm = rng.permutation(self.chunk_size)
297             else:
298                 # Validation: keep natural order – no need to shuffle.
299                 perm = np.arange(self.chunk_size)
300
301             # --- step 3: cut the chunk into batches -----
302             for b in range(self.batches_per_chunk):
303                 # `idx` are the rows of this chunk that form batch b.
304                 idx = perm[b * bs : (b + 1) * bs]
305                 # Gather rows and convert to the dtypes the model wants:
306                 # X : float32 board planes
307                 # v : float32 value target
308                 # p : int64 policy target (CrossEntropyLoss needs int64)
309                 X = torch.from_numpy(pos[idx].astype(np.float32))
310                 v = torch.from_numpy(val[idx].astype(np.float32))
311                 p = torch.from_numpy(pol[idx].astype(np.int64))
312                 # Pin into page-locked RAM so the later .to(gpu,
313                 # non_blocking=True) can be a true asynchronous DMA copy.
314                 if self.pin_memory:
315                     X, v, p = X.pin_memory(), v.pin_memory(), p.pin_memory()
316                 # Blocks here if the consumer is behind (queue full).
317                 q.put((X, v, p))
318             except Exception as e: # pragma: no cover
319                 # A crash in a thread is otherwise silent; surface it.
320                 print(f"[ChunkedShuffleLoader] producer error: {e}", flush=True)
321             finally:
322                 # Always post the end-of-epoch sentinel, even on error, so the
323                 # consumer's loop terminates instead of blocking forever.
324                 q.put(None)
325
326     def __iter__(self):
327         """
328         Start one epoch: spawn the producer thread and yield its batches.
329
330         Called once per epoch (by the trainer's `for ... in loader`). Each call
331         creates a fresh queue + thread, so epochs are independent.
332         """
333         # Decide the order in which this rank visits its chunks this epoch.
334         order = list(range(len(self.my_chunks)))
335         if self.shuffle:
336             random.Random(self.epoch).shuffle(order)
337

```

```

338     # The bounded queue IS the rotating buffer between producer & consumer.
339     q = queue_mod.Queue(maxsize=self.queue_depth)
340     # daemon=True → the thread will not keep the process alive on exit.
341     t = threading.Thread(target=self._producer, args=(order, q), daemon=True)
342     t.start()
343
344     # Consume batches until the producer posts the None sentinel.
345     while True:
346         item = q.get()
347         if item is None:
348             break
349         yield item
350     t.join() # tidy up the (already finished) producer thread
351
352
353 # — Network —
354 class ResBlock(nn.Module):
355     """
356     One pre-activation residual block (the "ResNet v2" ordering:
357     BatchNorm → ReLU → Conv, twice, then add the input back).
358
359     The skip connection (x + residual) lets gradients flow straight through,
360     which is what makes a deep (30-block) stack trainable.
361     """
362     def __init__(self, channels: int):
363         super().__init__()
364         # bias=False on the convs because the following BatchNorm has its own
365         # learnable shift, making a conv bias redundant.
366         self.bn1 = nn.BatchNorm2d(channels)
367         self.conv1 = nn.Conv2d(channels, channels, 3, padding=1, bias=False)
368         self.bn2 = nn.BatchNorm2d(channels)
369         self.conv2 = nn.Conv2d(channels, channels, 3, padding=1, bias=False)
370
371     def forward(self, x):
372         residual = x # save the input
373         x = self.conv1(torch.relu(self.bn1(x))) # BN → ReLU → Conv
374         x = self.conv2(torch.relu(self.bn2(x))) # BN → ReLU → Conv
375         return x + residual # add the skip connection
376
377
378 class Net(nn.Module):
379     """
380     AlphaZero-style two-headed network.
381
382     Trunk : (B, 4, 8, 8) — input conv — N residual blocks — BN — ReLU
383             → a shared (B, C, 8, 8) feature map.
384
385     Value head : 1×1 conv to 1 channel → flatten(64) → FC → ReLU → FC
386                  → tanh → a scalar  $v \in (-1, +1)$  per position.
387                  Interpretation: expected game outcome for the player to
388                  move;  $P(\text{win}) = (v + 1) / 2$ .
389
390     Policy head : 1×1 conv to 2 channels → flatten(128) → FC
391                   → 4096 logits, one per (from, to) action.
392
393     forward() returns the pair (value, policy_logits).
394     """
395     def __init__(self, channels: int = 256, nb_blocks: int = 30,
396                  value_fc_size: int = 256, num_actions: int = NUM_ACTIONS):
397         super().__init__()
398
399         # Stem: lift the 4 input planes to `channels` feature maps.
400         self.input_conv = nn.Sequential(

```

```

401         nn.Conv2d(4, channels, kernel_size=3, padding=1, bias=False),
402         nn.BatchNorm2d(channels),
403         nn.ReLU(inplace=True),
404     )
405     # Shared trunk: nb_blocks residual blocks, then a final BN.
406     self.res_blocks = nn.Sequential(*[ResBlock(channels) for _ in range(nb_blocks)])
407     self.output_bn = nn.BatchNorm2d(channels)
408
409     # Value head. The 1x1 conv squeezes C channels down to 1, giving a
410     # (B, 1, 8, 8) map = 64 numbers, which the FC layers turn into a scalar.
411     self.value_conv = nn.Conv2d(channels, 1, kernel_size=1, bias=False)
412     self.value_bn = nn.BatchNorm2d(1)
413     self.value_fc1 = nn.Linear(64, value_fc_size)
414     self.value_fc2 = nn.Linear(value_fc_size, 1)
415
416     # Policy head. 1x1 conv to 2 channels → (B, 2, 8, 8) = 128 numbers →
417     # one FC straight to the 4096-way action logits.
418     self.policy_conv = nn.Conv2d(channels, 2, kernel_size=1, bias=False)
419     self.policy_bn = nn.BatchNorm2d(2)
420     self.policy_fc = nn.Linear(2 * 64, num_actions)
421
422     def forward(self, x):
423         # Shared trunk.
424         x = self.input_conv(x)
425         x = self.res_blocks(x)
426         x = torch.relu(self.output_bn(x)) # shared (B, C, 8, 8)
427
428         # Value head: (B,C,8,8) → (B,1,8,8) → (B,64) → (B,fc) → (B,1) → (B,)
429         v = torch.relu(self.value_bn(self.value_conv(x)))
430         v = v.flatten(1) # keep batch dim, flatten rest
431         v = torch.relu(self.value_fc1(v))
432         v = torch.tanh(self.value_fc2(v)).squeeze(-1) # tanh → (-1,1); drop last dim
433
434         # Policy head: (B,C,8,8) → (B,2,8,8) → (B,128) → (B, NUM_ACTIONS)
435         p = torch.relu(self.policy_bn(self.policy_conv(x)))
436         p = p.flatten(1)
437         p = self.policy_fc(p) # raw logits (no softmax)
438         return v, p
439
440     def clip(self):
441         # No-op: kept so the training loop can call model.clip() uniformly
442         # across networks (the NNUE variants clamp weights here for int8
443         # quantization; this CNN does not need it).
444         pass
445
446
447 # — Training configuration —————
448 # Override at runtime with YOLAH_NB_EPOCHS – lets you change epoch count
449 # without rebuilding the .sif (the .sh's `singularity exec --env ...` passes
450 # it through). Default keeps the original value.
451 NB_EPOCHS = int(os.environ.get("YOLAH_NB_EPOCHS", "20"))
452 MODEL_PATH = "/mnt/" # checkpoint dir (bind-mounted)
453 MODEL_NAME = "cnn_resnet_256x30_value_policy"
454 LAST_MODEL = f"{MODEL_PATH}{MODEL_NAME}.pt" # if present, resume from it
455 # Cache directory inside the container (the .sh bind-mounts the host cache
456 # onto /cache); overridable via env var for local runs.
457 CACHE_DIR = os.environ.get("YOLAH_CACHE_DIR", "/cache")
458
459 # Total loss = VALUE_LOSS_WEIGHT * MSE(value) + POLICY_LOSS_WEIGHT * CE(policy).
460 # AlphaZero weights both at 1.0; raise the value weight if value learning lags.
461 VALUE_LOSS_WEIGHT = 1.0
462 POLICY_LOSS_WEIGHT = 1.0
463

```

```

464
465 def ddp_setup(rank, world_size):
466     """
467     Initialise the process group for DistributedDataParallel.
468
469     All ranks live on one node here, so they rendezvous over localhost. NCCL is
470     the GPU-to-GPU communication backend used for the gradient all-reduce.
471     """
472     os.environ["MASTER_ADDR"] = "localhost"
473     os.environ["MASTER_PORT"] = "65437"
474     init_process_group(backend="nccl", rank=rank, world_size=world_size)
475
476
477 class TrainerDDP:
478     """Owns one GPU's model replica, optimizer, and the train/validate loops."""
479
480     def __init__(self, gpu_id, model, train_loader, val_loader, save_every=1):
481         self.gpu_id = gpu_id
482
483         # Move the model to this rank's GPU and convert it to channels-last
484         # (NHWC) memory format: cuDNN's NHWC convolution kernels are faster on
485         # tensor-core GPUs, and it makes the gradient layout match what DDP's
486         # buckets expect (silences the "grad strides" warning).
487         self.model = model.to(gpu_id).to(memory_format=torch.channels_last)
488
489         self.train_loader = train_loader
490         self.val_loader = val_loader
491         self.save_every = save_every
492
493         # Adam with a little weight decay (L2 regularisation).
494         self.optimizer = torch.optim.Adam(self.model.parameters(), lr=0.001,
495                                           weight_decay=1e-4)
496
497         # Value head: regression → mean-squared error against  $z \in \{-1, 0, +1\}$ .
498         self.value_loss_fn = nn.MSELoss()
499         # Policy head: 4096-way classification; ignore_index skips terminal
500         # positions whose policy target is POLICY_IGNORE_INDEX.
501         self.policy_loss_fn = nn.CrossEntropyLoss(ignore_index=POLICY_IGNORE_INDEX)
502         # Cosine schedule: lr eases from 1e-3 down to 1e-5 over the run.
503         self.scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
504             self.optimizer, T_max=NB_EPOCHS, eta_min=1e-5)
505
506         # Mixed precision. bf16 has wide dynamic range and needs no loss
507         # scaling, but only Ampere+ (compute capability major ≥ 8) has bf16
508         # tensor cores. Turing GPUs (e.g. Quadro RTX 8000, cc 7.5) would run
509         # bf16 emulated and SLOW – they do, however, have fp16 tensor cores.
510         # So: gate strictly on the capability major version.
511         major, _ = torch.cuda.get_device_capability(gpu_id)
512         self.amp_dtype = torch.bfloat16 if major ≥ 8 else torch.float16
513         # fp16 gradients can underflow → need a GradScaler. bf16 does not, so
514         # the scaler is disabled there and its scale/step/update become no-ops.
515         self.scaler = torch.amp.GradScaler(
516             'cuda', enabled=(self.amp_dtype == torch.float16))
517
518         torch.cuda.set_device(gpu_id)
519         torch.cuda.empty_cache()
520
521         # Wrap in DDP: every backward pass all-reduces gradients across ranks.
522         # gradient_as_bucket_view: gradients live directly in the all-reduce
523         # bucket memory → one less memcopy + one less stream sync per bucket.
524         # static_graph: tells DDP the autograd graph is identical every step
525         # (true for us – no conditionals, no unused params, no model surgery)
526         # so the reducer is built once and DDP can fuse/overlap aggressively.
527         # Both are aimed at the periodic ~50 s all-reduce stall observed on
528         # multi-GPU runs that vanishes on single-GPU.

```

```

527     self.model = DDP(self.model, device_ids=[gpu_id],
528                       gradient_as_bucket_view=True, static_graph=True)
529     # torch.compile JIT-fuses the graph for extra speed (first iterations
530     # are slow while it compiles, then much faster).
531     self.model = torch.compile(self.model)
532     # A side CUDA stream used to prefetch the next batch's H2D copy.
533     self.stream = torch.cuda.Stream(device=gpu_id)
534
535     def _save_checkpoint(self, epoch):
536         """Save the raw model weights (unwrap DDP via .module)."""
537         torch.save(self.model.module.state_dict(),
538                   f"{MODEL_PATH}/{MODEL_NAME}.{epoch}.pt")
539
540     def _compute_loss(self, value_pred, policy_logits, value_target, policy_target):
541         """Combined value + policy loss; also returns the two parts for logging."""
542         v_loss = self.value_loss_fn(value_pred, value_target)
543         p_loss = self.policy_loss_fn(policy_logits, policy_target)
544         return (VALUE_LOSS_WEIGHT * v_loss
545               + POLICY_LOSS_WEIGHT * p_loss, v_loss, p_loss)
546
547     @staticmethod
548     def _policy_correct(policy_logits, policy_target):
549         """
550         Count correct top-1 policy predictions, ignoring terminal positions.
551         Returns (n_correct, n_scored) so the caller can accumulate an accuracy.
552         """
553         mask = policy_target != POLICY_IGNORE_INDEX # valid (non-terminal) rows
554         if mask.sum().item() == 0:
555             return 0, 0
556         preds = torch.argmax(policy_logits, dim=1)
557         correct = ((preds == policy_target) & mask).sum().item()
558         return correct, int(mask.sum().item())
559
560     def _h2d_async(self, batch_cpu):
561         """
562         Issue an asynchronous host->GPU transfer for one (X, v, p) batch on
563         self.stream, return the GPU tensors.
564
565         PREFETCH: callers `wait_stream` on the default stream *before* using
566         these tensors. Because the H2D runs on the side stream, it can proceed
567         concurrently with the previous iteration's compute on the default
568         stream – that is the whole point of the prefetch.
569         """
570         X_cpu, v_cpu, p_cpu = batch_cpu
571         with torch.cuda.stream(self.stream):
572             X = X_cpu.to(self.gpu_id, non_blocking=True)
573             v = v_cpu.to(self.gpu_id, non_blocking=True)
574             p = p_cpu.to(self.gpu_id, non_blocking=True)
575         return (X, v, p)
576
577     def _run_epoch(self, epoch):
578         """One training pass over this rank's shard of the data.
579
580         PREFETCH: while iteration N is computing on the default stream, the
581         H2D transfer for iteration N+1 is in flight on self.stream. The body's
582         wall-time therefore measures only compute, not transfer, and the
583         transfer is hidden behind compute instead of adding to the iteration.
584         """
585         n = 0 # positions seen
586         # Three value-accuracy variants tracked side-by-side:
587         #   v_correct – sign(v) == sign(y); draws always count as wrong
588         #   (ceiling = 1 - P(draw)).
589         #   v_bucket_correct – bucket(v) ∈ {-1, 0, +1} via |v| < 0.33; fair for draws.

```

```

590     # v_signed_correct / v_signed_n - sign-acc over non-draw rows only.
591     v_correct      = 0
592     v_bucket_correct = 0
593     v_signed_correct = 0
594     v_signed_n      = 0
595     p_correct = 0                # policy top-1 hits
596     p_total   = 0                # non-terminal positions
597     v_running, p_running = 0.0, 0.0 # running loss sums
598     it = 0                      # iteration counter (for periodic gc)
599
600     # Pre-load the first batch onto the GPU (side stream). We keep the
601     # CPU batch reference (cpu_cur) alive across the iteration so its
602     # pinned tensors are not freed before the async H2D completes.
603     loader_iter = iter(self.train_loader)
604     cpu_cur = next(loader_iter, None)
605     if cpu_cur is None:
606         return
607     gpu_cur = self._h2d_async(cpu_cur)
608
609     pbar = tqdm(total=len(self.train_loader))
610     while True:
611         # Make the default stream wait until the current batch's H2D is
612         # complete on self.stream. This is just a stream dependency - the
613         # host does not block here.
614         torch.cuda.current_stream(self.gpu_id).wait_stream(self.stream)
615         X, v_target, p_target = gpu_cur
616         bs = len(X)
617         n += bs
618
619         # Issue the NEXT iteration's H2D RIGHT NOW on self.stream. It will
620         # run concurrently with this iteration's forward/backward/step on
621         # the default stream - that overlap is the whole point.
622         cpu_next = next(loader_iter, None)
623         if cpu_next is not None:
624             gpu_next = self._h2d_async(cpu_next)
625
626         # Match the model's channels-last memory format (sub-ms reformat).
627         X = X.contiguous(memory_format=torch.channels_last)
628
629         self.optimizer.zero_grad()
630         with torch.autocast('cuda', dtype=self.amp_dtype):
631             v_pred, p_logits = self.model(X)
632             loss, v_loss, p_loss = self._compute_loss(v_pred, p_logits,
633                                                         v_target, p_target)
634
635         self.scaler.scale(loss).backward()
636         self.scaler.step(self.optimizer)
637         self.scaler.update()
638
639         # Accumulate metrics (.item() pulls scalars back to the CPU).
640         v_running      += v_loss.item() * bs
641         p_running      += p_loss.item() * bs
642         v_correct      += (torch.sign(v_pred) == torch.sign(v_target)).sum().item()
643         # 3-way bucketed accuracy: bucket v_pred to {-1, 0, +1} via |v|<0.33.
644         v_bucket = torch.where(v_pred > 0.33, 1.0,
645                                torch.where(v_pred < -0.33, -1.0, 0.0))
646         v_bucket_correct += (v_bucket == v_target).sum().item()
647         # Sign-acc on non-draws only (mask out v_target == 0).
648         v_mask = v_target != 0
649         if v_mask.any():
650             v_signed_correct += (torch.sign(v_pred[v_mask]) == v_target[v_mask]).sum().item()
651             v_signed_n      += int(v_mask.sum().item())
652         pc, pt = self._policy_correct(p_logits, p_target)
653         p_correct += pc

```

```

653     p_total += pt
654
655     self.model.module.clip()          # no-op for this CNN
656
657     # Manual GC every 1000 iters – auto-GC is disabled in main(), so
658     # we mop up reference cycles in one controlled pause instead of
659     # whatever the auto-collector would decide.
660     it += 1
661     if it % 1000 == 0:
662         gc.collect()
663
664     pbar.update(1)
665
666     # Advance to the next batch, or stop if the loader is exhausted.
667     # Re-binding here drops the previous iter's CPU tensors – by now
668     # their H2D has long completed (we waited for it at the top of
669     # this iter), so it is safe to free them.
670     if cpu_next is None:
671         break
672     cpu_cur = cpu_next
673     gpu_cur = gpu_next
674
675 pbar.close()
676 self.scheduler.step()                # advance the lr schedule
677
678 # Only rank 0 logs, so the output is not printed three times.
679 if self.gpu_id == 0:
680     lr = self.optimizer.param_groups[0]['lr']
681     p_acc = (p_correct / p_total) if p_total > 0 else 0.0
682     v_signed_acc = (v_signed_correct / v_signed_n) if v_signed_n > 0 else 0.0
683     print(f'epoch {epoch+1} '
684           f'train value_mse: {v_running/n:.4f} '
685           f'value_sign_acc: {v_correct/n:.4f} '
686           f'value_bucket_acc: {v_bucket_correct/n:.4f} '
687           f'value_signed_acc: {v_signed_acc:.4f} '
688           f'policy_ce: {p_running/n:.4f} '
689           f'policy_acc: {p_acc:.4f} '
690           f'lr: {lr:.6f}', flush=True)
691
692 def _validate(self, epoch):
693     """One pass over the validation shard – no gradients, no optimizer.
694     Uses the same prefetch pattern as _run_epoch.
695     """
696     self.model.train(False)           # eval mode (freezes BatchNorm)
697     n = 0
698     v_correct = 0
699     v_bucket_correct = 0
700     v_signed_correct = 0
701     v_signed_n = 0
702     p_correct = 0
703     p_total = 0
704     v_running, p_running = 0.0, 0.0
705     with torch.no_grad():             # no autograd graph → less memory
706         loader_iter = iter(self.val_loader)
707         cpu_cur = next(loader_iter, None)
708         if cpu_cur is None:
709             self.model.train()
710             return
711         gpu_cur = self._h2d_async(cpu_cur)
712         pbar = tqdm(total=len(self.val_loader))
713         while True:
714             torch.cuda.current_stream(self.gpu_id).wait_stream(self.stream)
715             X, v_target, p_target = gpu_cur

```

```

716         bs = len(X)
717         n += bs
718
719         cpu_next = next(loader_iter, None)
720         if cpu_next is not None:
721             gpu_next = self._h2d_async(cpu_next)
722
723         X = X.contiguous(memory_format=torch.channels_last)
724         with torch.autocast('cuda', dtype=self.amp_dtype):
725             v_pred, p_logits = self.model(X)
726             _, v_loss, p_loss = self._compute_loss(v_pred, p_logits,
727                                                    v_target, p_target)
728             v_running      += v_loss.item() * bs
729             p_running      += p_loss.item() * bs
730             v_correct      += (torch.sign(v_pred) == torch.sign(v_target)).sum().item()
731             v_bucket = torch.where(v_pred > 0.33, 1.0,
732                                   torch.where(v_pred < -0.33, -1.0, 0.0))
733             v_bucket_correct += (v_bucket == v_target).sum().item()
734             v_mask = v_target != 0
735             if v_mask.any():
736                 v_signed_correct += (torch.sign(v_pred[v_mask]) ==
↪ v_target[v_mask]).sum().item()
737                 v_signed_n      += int(v_mask.sum().item())
738             pc, pt = self._policy_correct(p_logits, p_target)
739             p_correct += pc
740             p_total  += pt
741
742             pbar.update(1)
743
744             if cpu_next is None:
745                 break
746             cpu_cur = cpu_next
747             gpu_cur = gpu_next
748         pbar.close()
749         if self.gpu_id == 0:
750             p_acc      = (p_correct / p_total)      if p_total > 0 else 0.0
751             v_signed_acc = (v_signed_correct / v_signed_n) if v_signed_n > 0 else 0.0
752             print(f'epoch {epoch+1} '
753                 f'val value_mse: {v_running/n:.4f} '
754                 f'value_sign_acc: {v_correct/n:.4f} '
755                 f'value_bucket_acc: {v_bucket_correct/n:.4f} '
756                 f'value_signed_acc: {v_signed_acc:.4f} '
757                 f'policy_ce: {p_running/n:.4f} '
758                 f'policy_acc: {p_acc:.4f}', flush=True)
759         self.model.train() # back to train mode
760
761     def train(self, nb_epochs):
762         """Top-level loop: for each epoch, train, validate, checkpoint."""
763         self.model.train()
764         for epoch in range(nb_epochs):
765             # Reseed the loader's shuffling so this epoch differs from the last.
766             self.train_loader.set_epoch(epoch)
767             self._run_epoch(epoch)
768             self._validate(epoch)
769             if self.gpu_id == 0 and epoch % self.save_every == 0:
770                 self._save_checkpoint(epoch)
771             self._save_checkpoint(nb_epochs - 1) # always save the final model
772
773
774 def main(rank, world_size, batch_size, cache_dir):
775     """
776     Per-process entry point – mp.spawn runs one copy of this on each GPU,
777     passing the GPU's index as `rank`.

```

```

778 """
779 ddp_setup(rank, world_size)
780
781 # Read the dataset size from meta.json and take a contiguous 95/5 split.
782 # The split is contiguous (not random) because positions are stored
783 # game-by-game, so a contiguous cut keeps whole games on one side – less
784 # train/val leakage than a per-position random split.
785 #
786 # The loaders are built HERE, inside the spawned process – never passed in
787 # through mp.spawn. np.memmap is a lazy mmap (virtual address space only),
788 # so constructing a loader is cheap and the OS page cache is shared.
789 with open(os.path.join(cache_dir, "meta.json")) as f:
790     n_total = int(json.load(f)["n_positions"])
791     n_train = int(0.95 * n_total)
792
793     train_loader = ChunkedShuffleLoader(cache_dir, 0, n_train, batch_size,
794                                         rank, world_size, shuffle=True)
795     val_loader = ChunkedShuffleLoader(cache_dir, n_train, n_total, batch_size,
796                                       rank, world_size, shuffle=False)
797
798 if rank == 0:
799     print(f'Dataset: {n_total:,} positions -> '
800           f'train {n_train:,} val {n_total - n_train:,}', flush=True)
801     print(f'Batches/rank/epoch: train {len(train_loader):,} '
802           f'val {len(val_loader):,} (chunk_size={CHUNK_SIZE:,})', flush=True)
803
804 net = Net()
805 # Resume from a previous checkpoint if one exists.
806 if os.path.isfile(LAST_MODEL):
807     net.load_state_dict(torch.load(LAST_MODEL, map_location='cpu'))
808 if rank == 0:
809     nb_params = sum(p.numel() for p in net.parameters())
810     print(net, flush=True)
811     print(f'Parameters: {nb_params:,}', flush=True)
812
813 # Disable Python's cyclic GC for the duration of training; reference
814 # counting still frees everything that is not in a cycle, and we mop up
815 # cycles ourselves with gc.collect() every 1000 iters in _run_epoch.
816 # This trades unpredictable Gen-2 pauses for one controlled pause.
817 gc.disable()
818
819 trainer = TrainerDDP(rank, net, train_loader, val_loader)
820 trainer.train(NB_EPOCHS)
821 destroy_process_group() # clean up the NCCL group
822
823
824 if __name__ == "__main__":
825     print(torch.cuda.is_available())
826     # One training process per ALLOCATED GPU. world_size honours
827     # CUDA_VISIBLE_DEVICES (the SLURM convention) when set, falling back to
828     # device_count() only for local dev. This avoids the cluster pitfall
829     # where Singularity --nv exposes all physical GPUs and mp.spawn over
830     # device_count() then races for one the job does not own → NCCL hang.
831     cvd = os.environ.get("CUDA_VISIBLE_DEVICES", "").strip()
832     world_size = (len([x for x in cvd.split(",") if x.strip()])
833                  if cvd else torch.cuda.device_count())
834     print(world_size, flush=True)
835     # mp.spawn calls main(rank, world_size, 1024, CACHE_DIR) on each GPU; it
836     # prepends `rank` itself. batch_size is PER GPU → effective batch is
837     # 1024 * world_size.
838     mp.spawn(main, args=(world_size, 1024, CACHE_DIR), nprocs=world_size)

```

11.3.2 nnue/resnet_export.py

236 lines.

One checkpoint in, three artefacts out. The module docstring is the authoritative .bin layout; Section 4.3 is the same thing with the algebra spelled out.

The asymmetry to remember: a BatchNorm that *follows* a convolution folds into its weights and bias; a BatchNorm that *precedes* one cannot, so its (s, t) pair is written out for the C++ side to apply.

symbol	role
bn_affine	eval-mode BatchNorm $\rightarrow (s, t)$ with $s = \gamma / \sqrt{\sigma^2 + \epsilon}$, $t = \beta - \mu s$
conv_to_out_tap_in	$(out, in, 3, 3) \rightarrow (out, 9, in)$, the layout the CPU GEMM wants
Writer	little-endian float32 sink, counting what it wrote
export_bin	header, stem, blocks, output BN, both heads — the order <code>ResNetCPU::load</code> reads
export_torchscript	trace in eval mode on a dummy batch and save
export_test_vectors	random positions with PyTorch's exact fp32 outputs, for the C++ self-check
main	tolerates a DDP-wrapped checkpoint, infers C , B , F , A from the tensors

```

1  """
2  resnet_export.py – export the trained two-headed ResNet (cnn_resnet_256x30_
3  value_policy.pt) for C++ inference by the AlphaZero-style MCTS player.
4
5  Three artefacts are produced next to the checkpoint:
6
7  1. <name>.bin – a flat little-endian float32 dump with every BatchNorm
8                  folded into the adjacent conv / affine op. This is what the
9                  pure-C++ CPU backend (nnue/resnet.h) loads. No PyTorch needed
10                 at run time.
11  2. <name>.ts – a TorchScript module (traced in eval mode) for the optional
12                 libtorch backend (nnue/resnet_torch.h, built with
13                 -DENABLE_TORCH=ON). This one runs on the GPU.
14  3. <name>.test.bin – a handful of random positions with the exact PyTorch
15                 outputs (value + 4096 policy logits) so the C++ backends can
16                 self-check bit-for-bit-ish (see test/alphazero_mcts_check.cpp).
17
18  Usage (from the nnue/ directory, the module imports ../server/yolah.py):
19
20  python3 resnet_export.py [checkpoint.pt] [--no-ts] [--nb-test 32]
21
22  Binary layout of <name>.bin (all little-endian; float32 unless stated)
23
24  char[8]  magic      = "YOLAHRSN"
25  uint32   version    = 1
26  uint32   in_channels = 4
27  uint32   channels    = C   (256)
28  uint32   nb_blocks   = B   (30)
29  uint32   value_fc    = F   (256)
30  uint32   num_actions = A   (4096)
31  — trunk —

```

```

32     stem_w    [C][9][4]      conv weight, BN folded, layout (out, tap, in) with
33                               tap = dr*3 + dc, dr/dc ∈ {0,1,2} (kernel row/col)
34     stem_b    [C]            folded bias
35     for each block b in 0..B-1:
36         bn1_scale [C], bn1_shift [C]      pre-activation affine (y = s*x + t)
37         conv1_w    [C][9][C]              (out, tap, in), no bias
38         bn2_scale [C], bn2_shift [C]
39         conv2_w    [C][9][C]
40     out_bn_scale [C], out_bn_shift [C]
41     — value head —
42     value_conv_w [C]          1x1 conv (C→1) with value_bn folded
43     value_conv_b [1]
44     value_fc1_w  [F][64], value_fc1_b [F]
45     value_fc2_w  [1][F],  value_fc2_b [1]
46     — policy head —
47     policy_conv_w [2][C]     1x1 conv (C→2) with policy_bn folded
48     policy_conv_b [2]
49     policy_fc_w   [A][128], policy_fc_b [A]
50
51     Why fold BatchNorm: in eval mode BN is a per-channel affine map
52      $y = \gamma(x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta = s \cdot x + t$ . Where BN directly follows a conv (stem,
53     value_conv, policy_conv) it folds into the conv weights ( $W \cdot s$ ) and a bias ( $t$ ).
54     Where BN precedes a ReLU+conv (the pre-activation residual blocks and the
55     trunk's output_bn) it cannot be merged into the conv, so we keep ( $s$ ,  $t$ ) and the
56     C++ code applies  $\max(0, s \cdot x + t)$  while it gathers the im2col patches – no extra
57     pass over the activations.
58     """
59     import os
60     import sys
61     import struct
62     import argparse
63     import numpy as np
64     import torch
65
66     sys.path.append("../server")
67     from yolah import Yolah, Move, Square # noqa: E402
68     from cnn_resnet_value_policy_chunked import Net, encode_cnn # noqa: E402
69
70     MAGIC    = b"YOLAHRSN"
71     VERSION  = 1
72     BN_EPS   = 1e-5 # nn.BatchNorm2d default
73
74
75     def bn_affine(sd, prefix):
76         """Return (scale, shift) float32 arrays for the eval-mode BatchNorm `prefix`."""
77         gamma = sd[f"{prefix}.weight"].double()
78         beta  = sd[f"{prefix}.bias"].double()
79         mean  = sd[f"{prefix}.running_mean"].double()
80         var   = sd[f"{prefix}.running_var"].double()
81         scale = gamma / torch.sqrt(var + BN_EPS)
82         shift = beta - mean * scale
83         return scale.float().numpy(), shift.float().numpy()
84
85
86     def conv_to_out_tap_in(w):
87         """(out, in, 3, 3) torch conv weight → (out, 9, in) float32 numpy, tap = dr*3+dc."""
88         w = w.detach().double().numpy()
89         out, inn, kh, kw = w.shape
90         assert kh == 3 and kw == 3
91         return np.ascontiguousarray(w.transpose(0, 2, 3, 1).reshape(out, 9, inn)).astype(np.float32)
92
93
94     class Writer:

```

```

95     def __init__(self, path):
96         self.f = open(path, "wb")
97         self.nb_floats = 0
98
99     def u32(self, v):
100         self.f.write(struct.pack("<I", v))
101
102     def floats(self, arr):
103         arr = np.ascontiguousarray(arr, dtype="<f4")
104         self.f.write(arr.tobytes())
105         self.nb_floats += arr.size
106
107     def close(self):
108         self.f.close()
109
110
111 def export_bin(net, sd, path):
112     C = net.input_conv[0].weight.shape[0]
113     B = len(net.res_blocks)
114     F = net.value_fc1.weight.shape[0]
115     A = net.policy_fc.weight.shape[0]
116
117     w = Writer(path)
118     w.f.write(MAGIC)
119     for v in (VERSION, 4, C, B, F, A):
120         w.u32(v)
121
122     # — stem: conv(4→C, no bias) followed by BN → fold BN into the conv —
123     s, t = bn_affine(sd, "input_conv.1")
124     stem = conv_to_out_tap_in(sd["input_conv.0.weight"]) * s[:, None, None]
125     w.floats(stem)
126     w.floats(t)
127
128     # — residual blocks (pre-activation: BN→ReLU→conv, twice) —
129     for b in range(B):
130         p = f"res_blocks.{b}"
131         s1, t1 = bn_affine(sd, f"{p}.bn1")
132         w.floats(s1); w.floats(t1)
133         w.floats(conv_to_out_tap_in(sd[f"{p}.conv1.weight"]))
134         s2, t2 = bn_affine(sd, f"{p}.bn2")
135         w.floats(s2); w.floats(t2)
136         w.floats(conv_to_out_tap_in(sd[f"{p}.conv2.weight"]))
137
138     # — trunk output BN (followed by ReLU in forward) —
139     so, to = bn_affine(sd, "output_bn")
140     w.floats(so); w.floats(to)
141
142     # — value head: 1x1 conv(C→1, no bias) + BN(1) folded —
143     sv, tv = bn_affine(sd, "value_bn")
144     vconv = sd["value_conv.weight"].double().numpy().reshape(1, C) * sv[:, None]
145     w.floats(vconv.astype(np.float32))
146     w.floats(tv)
147     w.floats(sd["value_fc1.weight"].numpy()); w.floats(sd["value_fc1.bias"].numpy())
148     w.floats(sd["value_fc2.weight"].numpy()); w.floats(sd["value_fc2.bias"].numpy())
149
150     # — policy head: 1x1 conv(C→2, no bias) + BN(2) folded —
151     sp, tp = bn_affine(sd, "policy_bn")
152     pconv = sd["policy_conv.weight"].double().numpy().reshape(2, C) * sp[:, None]
153     w.floats(pconv.astype(np.float32))
154     w.floats(tp)
155     w.floats(sd["policy_fc.weight"].numpy()); w.floats(sd["policy_fc.bias"].numpy())
156
157     w.close()

```

```

158     print(f"wrote {path}: C={C} blocks={B} value_fc={F} actions={A}, "
159           f"{w.nb_floats:,} floats ({os.path.getsize(path) / 1e6:.1f} MB)")
160
161
162 def export_torchscript(net, path):
163     """Trace the eval-mode network on a dummy batch and save it for libtorch."""
164     example = torch.zeros(1, 4, 8, 8)
165     with torch.no_grad():
166         traced = torch.jit.trace(net, example)
167     traced.save(path)
168     print(f"wrote {path} ({os.path.getsize(path) / 1e6:.1f} MB)")
169
170
171 def random_position(rng, max_plies=60):
172     """Play a random game for a random number of plies; return the Yolah state."""
173     y = Yolah()
174     n = int(rng.integers(0, max_plies))
175     for _ in range(n):
176         if y.game_over():
177             break
178         moves = y.moves()
179         y.play(moves[int(rng.integers(0, len(moves)))])
180     return y
181
182
183 def export_test_vectors(net, path, nb, seed=1234):
184     """
185     Record `nb` random (non-terminal preferred) positions with PyTorch's
186     fp32 outputs. Record layout, repeated nb times after a uint32 count:
187         uint64 black, uint64 white, uint64 empty, uint32 turn(0/1),
188         float32 value, float32[4096] policy logits
189     """
190     rng = np.random.default_rng(seed)
191     positions = [Yolah()] # always include the initial position
192     while len(positions) < nb:
193         y = random_position(rng)
194         if not y.game_over():
195             positions.append(y)
196     X = torch.stack([encode_cnn(y) for y in positions])
197     with torch.no_grad():
198         v, p = net(X)
199     v = v.numpy().astype("<f4"); p = p.numpy().astype("<f4")
200     with open(path, "wb") as f:
201         f.write(struct.pack("<I", nb))
202         for i, y in enumerate(positions):
203             f.write(struct.pack("<QQQI", y.black, y.white, y.empty, y.nb_plies() & 1))
204             f.write(struct.pack("<f", float(v[i])))
205             f.write(p[i].tobytes())
206     print(f"wrote {path}: {nb} positions "
207           f"(value range [{v.min():+.3f}, {v.max():+.3f}])")
208
209
210 def main():
211     ap = argparse.ArgumentParser(description=__doc__,
212                                 formatter_class=argparse.RawDescriptionHelpFormatter)
213     ap.add_argument("checkpoint", nargs="?", default="cnn_resnet_256x30_value_policy.pt")
214     ap.add_argument("--no-ts", action="store_true", help="skip the TorchScript export")
215     ap.add_argument("--nb-test", type=int, default=32, help="number of test vectors")
216     args = ap.parse_args()
217
218     sd = torch.load(args.checkpoint, map_location="cpu")
219     # Tolerate a DDP-wrapped checkpoint ("module." prefix).
220     sd = {(k[7:] if k.startswith("module.") else k): v for k, v in sd.items()}

```

```

221     C = sd["input_conv.0.weight"].shape[0]
222     B = sum(1 for k in sd if k.endswith(".conv1.weight"))
223     net = Net(channels=C, nb_blocks=B, value_fc_size=sd["value_fc1.weight"].shape[0],
224              num_actions=sd["policy_fc.weight"].shape[0])
225     net.load_state_dict(sd)
226     net.eval()
227
228     base = os.path.splitext(args.checkpoint)[0]
229     export_bin(net, sd, base + ".bin")
230     if not args.no_ts:
231         export_torchscript(net, base + ".ts")
232     export_test_vectors(net, base + ".test.bin", args.nb_test)
233
234
235 if __name__ == "__main__":
236     main()

```

11.4 The self-checks

alphazero_check is the regression suite: it verifies the C++ backends against PyTorch's own outputs, benchmarks them, and asserts the search's invariants.

11.4.1 test/resnet_check.cpp

102 lines.

Replays *.test.bin through a backend and compares. The tolerance is 2e-3 for the fp32 paths and 2e-2 for GPU fp16; the policy argmax is compared separately, because a logit difference that never changes the argmax is harmless to the search.

symbol	role
resnet_check	max
resnet_bench	positions per second at a given batch size

```

1  #include "resnet_check.h"
2  #include "misc.h"
3  #include <chrono>
4  #include <cmath>
5  #include <cstdio>
6  #include <fstream>
7  #include <iostream>
8  #include <vector>
9
10 namespace test {
11
12 namespace {
13     struct Vector {
14         Yolah state;
15         float value;
16         std::vector<float> logits;    // 4096
17     };
18
19     std::vector<Vector> read_vectors(const std::string& filename) {
20         std::ifstream ifs(filename, std::ios::binary);
21         if (!ifs) throw std::runtime_error("cannot open " + filename);
22         uint32_t n;

```

```

23     ifs.read(reinterpret_cast<char*>(&n), 4);
24     std::vector<Vector> res(n);
25     for (Vector& v : res) {
26         uint64_t black, white, empty;
27         uint32_t turn;
28         ifs.read(reinterpret_cast<char*>(&black), 8);
29         ifs.read(reinterpret_cast<char*>(&white), 8);
30         ifs.read(reinterpret_cast<char*>(&empty), 8);
31         ifs.read(reinterpret_cast<char*>(&turn), 4);
32         ifs.read(reinterpret_cast<char*>(&v.value), 4);
33         v.logits.resize(nn::NUM_ACTIONS);
34         ifs.read(reinterpret_cast<char*>(v.logits.data()), nn::NUM_ACTIONS * 4);
35         // Scores are irrelevant to the network; ply parity encodes the turn.
36         v.state.set_state(black, white, empty, 0, 0, static_cast<uint16_t>(turn));
37     }
38     if (!ifs) throw std::runtime_error("truncated " + filename);
39     return res;
40 }
41 }
42
43 bool resnet_check(nn::Evaluator& evaluator, const std::string& test_vectors_filename, float
↳ tolerance) {
44     const auto vectors = read_vectors(test_vectors_filename);
45     std::vector<Yolah::MoveList> moves(vectors.size());
46     std::vector<nn::Request> requests(vectors.size());
47     std::vector<nn::Result> results(vectors.size());
48     for (size_t i = 0; i < vectors.size(); i++) {
49         vectors[i].state.moves(moves[i]);
50         requests[i] = { &vectors[i].state, moves[i].begin(),
↳ static_cast<uint16_t>(moves[i].size()) };
51     }
52     evaluator.evaluate(requests, results);
53     float max_value_err = 0, max_logit_err = 0;
54     size_t argmax_mismatches = 0;
55     for (size_t i = 0; i < vectors.size(); i++) {
56         max_value_err = std::max(max_value_err, std::fabs(results[i].value - vectors[i].value));
57         size_t best_ref = 0, best_out = 0;
58         for (size_t m = 0; m < moves[i].size(); m++) {
59             const float ref = vectors[i].logits[nn::action_index(moves[i][m])];
60             max_logit_err = std::max(max_logit_err, std::fabs(results[i].logits[m] - ref));
61             if (ref > vectors[i].logits[nn::action_index(moves[i][best_ref])]) best_ref = m;
62             if (results[i].logits[m] > results[i].logits[best_out]) best_out = m;
63         }
64         argmax_mismatches += best_ref != best_out;
65     }
66     std::cout << "[resnet check] " << evaluator.info() << "\n"
67         << " positions          : " << vectors.size() << "\n"
68         << " max |value error|    : " << max_value_err << "\n"
69         << " max |logit error|    : " << max_logit_err << "\n"
70         << " policy argmax diffs : " << argmax_mismatches << "\n";
71     const bool ok = max_value_err <= tolerance && max_logit_err <= tolerance * 10;
72     std::cout << " " << (ok ? "OK" : "MISMATCH") << std::endl;
73     return ok;
74 }
75
76 double resnet_bench(nn::Evaluator& evaluator, size_t batch_size, size_t nb_batches) {
77     PRNG prng(42);
78     std::vector<Yolah> states(batch_size);
79     std::vector<Yolah::MoveList> moves(batch_size);
80     std::vector<nn::Request> requests(batch_size);
81     std::vector<nn::Result> results(batch_size);
82     for (size_t i = 0; i < batch_size; i++) {
83         // random non-terminal position

```

```

84     const size_t plies = reduce(prng.rand<uint32_t>(), 50);
85     for (size_t k = 0; k < plies && !states[i].game_over(); k++) {
86         states[i].moves(moves[i]);
87         states[i].play(moves[i][reduce(prng.rand<uint32_t>(), moves[i].size())]);
88     }
89     states[i].moves(moves[i]);
90     requests[i] = { &states[i], moves[i].begin(), static_cast<uint16_t>(moves[i].size()) };
91 }
92 evaluator.evaluate(requests, results); // warm-up
93 const auto start = std::chrono::steady_clock::now();
94 for (size_t k = 0; k < nb_batches; k++) evaluator.evaluate(requests, results);
95 const double secs = std::chrono::duration<double>(std::chrono::steady_clock::now() -
96     ↪ start).count();
97 const double pps = static_cast<double>(batch_size * nb_batches) / secs;
98 std::cout << "[resnet bench] batch " << batch_size << ": " << pps << " positions/s ("
99     << secs / nb_batches * 1e3 << " ms per batch)" << std::endl;
100 return pps;
101 }
102 } // namespace test

```

11.4.2 test/alphazero_mcts_check.cpp

163 lines.

The search's invariants. `check_result` is the interesting part: it asserts that the child visits sum to the root visits, that π sums to 1, that Q and the priors are in range, that the budget was honoured exactly — unless this was a playout-cap fast search — and that the move played is the first child in the reported (play-value) order.

`alphazero_reuse_check` deserves its shape. Asserting that a *random* opponent reply lands on a subtree the search explored is a coin flip — with ~100 legal replies and a few hundred simulations it usually does not — so that walk only reports, and the two cases that must always reuse are checked explicitly: searching the same position twice must re-root on the root itself, and playing our own move must re-root on a child.

symbol	role
<code>random_position</code>	a random legal position, for the search check
<code>check_result</code>	the invariants above
<code>alphazero_search_check</code>	four positions, fixed simulation budget, no tree reuse
<code>alphazero_reuse_check</code>	the random walk (reported) plus the root and child reuse cases (asserted)
<code>alphazero_vs</code>	a head-to-head match against another player config, colours alternating

```

1  #include "alphazero_mcts_check.h"
2  #include "alphazero_mcts_player.h"
3  #include "misc.h"
4  #include <cmath>
5  #include <iostream>
6  #include <memory>
7
8  using std::cout;
9
10 namespace test {

```

```

11
12 namespace {
13     Yolah random_position(PRNG& prng, size_t max_plies) {
14         Yolah y;
15         Yolah::MoveList moves;
16         const size_t n = reduce(prng.rand<uint32_t>(), static_cast<uint32_t>(max_plies));
17         for (size_t i = 0; i < n && !y.game_over(); i++) {
18             y.moves(moves);
19             y.play(moves[reduce(prng.rand<uint32_t>(), moves.size())]);
20         }
21         return y;
22     }
23
24     bool check_result(const az::SearchResult& r, uint64_t nb_simulations, bool reused) {
25         bool ok = true;
26         auto fail = [&](const std::string& what) { cout << "  FAIL: " << what << "\n"; ok = false;
27             ↵ };
28         if (r.best_move == Move::none() && r.children.size() > 1) fail("no move chosen");
29         uint64_t sum = 0;
30         float pi = 0.0f;
31         for (const az::ChildStat& c : r.children) {
32             sum += c.visits;
33             if (c.q < -1.0001f || c.q > 1.0001f) fail("Q out of range");
34             if (c.prior < 0.0f || c.prior > 1.0001f) fail("prior out of range");
35         }
36         for (float p : r.policy) pi += p;
37         if (!r.policy.empty() && std::fabs(pi - 1.0f) > 1e-3f) fail("policy does not sum to 1");
38         if (!reused && sum != r.root_visits) fail("children visits != root visits");
39         // A playout-cap "fast" search deliberately runs a smaller budget.
40         if (r.full_search) {
41             if (!reused && r.nb_simulations != nb_simulations) fail("simulation budget not
42                 ↵ honoured exactly");
43             if (r.nb_simulations < nb_simulations) fail("fewer simulations than requested");
44         } else if (r.nb_simulations == 0) {
45             fail("fast search ran no simulation");
46         }
47         if (!r.children.empty() && r.children[0].move != r.best_move &&
48             r.children[0].visits != r.children[1].visits)
49             fail("best move is not the most visited child");
50         return ok;
51     }
52 }
53
54 bool alphazero_search_check(const json& player_config, uint64_t nb_simulations) {
55     json cfg = player_config;
56     cfg["microseconds"] = 0;
57     cfg["nb simulations"] = nb_simulations;
58     cfg["verbose"] = false;
59     cfg["reuse tree"] = false;
60     cfg["temperature"] = 0.0;
61     AlphaZeroMCTSPPlayer player(cfg);
62     cout << "[search check] " << player.info() << "\n";
63     bool ok = true;
64     PRNG prng(7);
65     for (int k = 0; k < 4; k++) {
66         Yolah y = k == 0 ? Yolah() : random_position(prng, 40);
67         if (y.game_over()) continue;
68         player.play(y);
69         const az::SearchResult& r = player.last_result();
70         cout << "position " << k << " (ply " << y.nb_plies() << "):\n" << r.to_string(5);
71         ok &= check_result(r, nb_simulations, false);
72     }
73     cout << "  " << (ok ? "OK" : "FAILED") << std::endl;

```

```

72     return ok;
73 }
74
75 bool alphazero_reuse_check(const json& player_config, uint64_t nb_simulations) {
76     json cfg = player_config;
77     cfg["microseconds"] = 0;
78     cfg["nb simulations"] = nb_simulations;
79     cfg["verbose"] = false;
80     cfg["reuse tree"] = true;
81     AlphaZeroMCTSPlayer player(cfg);
82     cout << "[reuse check] " << player.info() << "\n";
83     PRNG prng(11);
84     Yolah y;
85     Yolah::MoveList moves;
86     bool ok = true;
87     size_t reused_plies = 0;
88     for (int ply = 0; ply < 6 && !y.game_over(); ply++) {
89         Move m = player.play(y);
90         const az::SearchResult& r = player.last_result();
91         ok &= check_result(r, nb_simulations, ply > 0);
92         const uint64_t new_nodes = r.tree_visits;
93         cout << " ply " << y.nb_plies() << ": " << m << " root visits " << r.root_visits
94             << " (this search: " << r.nb_simulations << "), nodes " << new_nodes << "\n";
95         if (ply > 0 && r.root_visits > r.nb_simulations) ++reused_plies;
96         y.play(m);
97         if (y.game_over()) break;
98         // random opponent reply
99         y.moves(moves);
100        y.play(moves[reduce(prng.rand<uint32_t>(), moves.size())]);
101    }
102    // The opponent replies at random, so whether its move is one the search
103    // actually explored – and the subtree therefore reusable – is luck; the
104    // count above is informational. The two cases below must always reuse.
105    cout << " reused subtrees on " << reused_plies << " of 5 plies (random replies)\n";
106
107    Yolah z = random_position(prng, 12);
108    if (!z.game_over()) {
109        player.game_over(z); // drop the tree
110        player.play(z);
111        const uint64_t first = player.last_result().root_visits;
112        player.play(z); // same position again → the root itself
113        const az::SearchResult& again = player.last_result();
114        ok &= check_result(again, nb_simulations, true);
115        if (again.root_visits <= first) { cout << " FAIL: the root was not reused\n"; ok = false;
116            ↪ }
117        cout << " same position twice: " << first << " → " << again.root_visits << " root
118            ↪ visits\n";
119        // Same search driving both sides: the next position is a child of the root.
120        z.play(again.best_move);
121        if (!z.game_over()) {
122            player.play(z);
123            const az::SearchResult& child = player.last_result();
124            ok &= check_result(child, nb_simulations, true);
125            // More visits at the root than this search alone produced: the
126            // rest came from the previous one (the budget may differ from
127            // nb_simulations when playout cap randomization is on).
128            if (child.root_visits <= child.nb_simulations) { cout << " FAIL: the child was not
129                ↪ reused\n"; ok = false; }
130            cout << " after our own move: " << child.root_visits << " root visits\n";
131        }
132    }
133    cout << " " << (ok ? "OK" : "FAILED") << std::endl;
134    return ok;

```

```

132 }
133
134 double alphazero_vs(const json& player_config, const json& opponent_config, size_t nb_games, bool
↪ verbose) {
135     double score = 0.0;
136     for (size_t g = 0; g < nb_games; g++) {
137         const bool az_black = (g % 2) == 0;
138         std::unique_ptr<Player> az = Player::create(player_config);
139         std::unique_ptr<Player> opp = Player::create(opponent_config);
140         Player* black = az_black ? az.get() : opp.get();
141         Player* white = az_black ? opp.get() : az.get();
142         Yolah y;
143         while (!y.game_over()) {
144             Player* p = y.current_player() == Yolah::BLACK ? black : white;
145             Move m = p->play(y);
146             if (verbose) cout << (y.current_player() == Yolah::BLACK ? "black " : "white ") << m
↪             << "\n";
147             y.play(m);
148         }
149         az->game_over(y);
150         opp->game_over(y);
151         const auto [bs, ws] = y.score();
152         const int az_diff = az_black ? int(bs) - int(ws) : int(ws) - int(bs);
153         score += az_diff > 0 ? 1.0 : (az_diff == 0 ? 0.5 : 0.0);
154         cout << "game " << g + 1 << ": alphazero as " << (az_black ? "black" : "white")
155             << ", score " << bs << "/" << ws << " → " << (az_diff > 0 ? "win" : az_diff == 0 ?
↪             "draw" : "loss")
156             << std::endl;
157     }
158     cout << "[alphazero vs " << opponent_config.value("name", std::string("?")) << "] "
159         << score / nb_games * 100 << " %" << std::endl;
160     return score / nb_games;
161 }
162
163 } // namespace test

```

11.5 The learning loop

The self-play learning loop of Section 6: the C++ self-play driver and its command-line front end, the Python trainer that orchestrates everything, the two search configurations, and the cluster image and job scripts.

11.5.1 player/alphazero_selfplay.h

87 lines.

The row format and the self-play entry point. `TrainingSample` is written to disk as is (336 bytes, no padding — the `static_asserts` guard that) and read by numpy through `SAMPLE_DTYPE` in `alphazero_learn.py`: the two must change together.

symbol	role
<code>TrainingSample</code>	one training row: the bitboards, the side to move, z , the sparse π , the search value, the network step
<code>SampleFileHeader</code>	24-byte file header: magic, version, row size, row and game counts

symbol	role
ModelRef / read_latest_model	parse <code>latest.json</code> , resolving a relative path against the work directory
SelfPlayOptions	the driver's knobs: concurrency, flush policy, poll period
run_selfplay	play until stopped — Section 6.3

```

1  #ifndef ALPHAZERO_SELFPLAY_H
2  #define ALPHAZERO_SELFPLAY_H
3  // alphazero_selfplay.h – the self-play half of the AlphaZero learning loop.
4  //
5  // The loop (driven by nnue/alphazero_learn.py, see player/README_ALPHAZERO_LEARN.md):
6  //
7  //   self-play (this file, C++) —samples→ <work>/selfplay/*.bin
8  //           ▲                               |
9  //           | latest.json                   ▼
10 //   <work>/models/az_<step>.ts ←export— trainer (alphazero_learn.py)
11 //
12 // run_selfplay() plays many games at once, one az::Search per game, all the
13 // searches sharing ONE network behind an nn::BatchingEvaluator: with ~100
14 // games each asking for a few leaves at a time, the GPU sees large batches
15 // while every individual search stays close to sequential (few virtual-loss
16 // collisions). It polls <work>/latest.json and hot-swaps the weights as soon
17 // as the trainer exports a new network.
18 //
19 // Only the "full" searches of playout cap randomization are recorded (KataGo:
20 // the fast searches exist to make games longer/cheaper, their visit counts are
21 // too noisy to be a policy target). A sample is the position, the policy
22 // target  $\pi$  over its legal moves and – once the game is over – its outcome  $z$ .
23 #include "alphazero_mcts.h"
24 #include "nn_evaluator.h"
25 #include <atomic>
26 #include <cstdint>
27 #include <string>
28
29 namespace az {
30
31 // — On-disk format (little-endian). Mirrored by SAMPLE_DTYPE in
32 //   nnue/alphazero_learn.py – change both together. —————
33 struct TrainingSample {
34     uint64_t black, white, empty;    // bitboards (the network input, see nn::encode_planes)
35     uint8_t  turn;                   // 0 = black to move, 1 = white
36     int8_t   z;                      // outcome for the side to move: -1, 0, +1
37     uint8_t  nb_moves;               // entries used in action[] / prob[]
38     uint8_t  flags;                  // reserved (0)
39     float    root_q;                 // search value of the root, side to move
40     uint32_t model_step;              // trainer step of the network that searched
41     uint16_t action[Yolah::MAX_NB_MOVES]; // policy index from *64+to (nn::action_index)
42     uint16_t prob[Yolah::MAX_NB_MOVES]; //  $\pi(\text{action}) \cdot 65535$ , rounded
43 };
44 static_assert(sizeof(TrainingSample) == 336, "keep in sync with alphazero_learn.py");
45
46 struct SampleFileHeader {
47     char    magic[8];                // "YOLAHSP1"
48     uint32_t version;                // 1
49     uint32_t sample_size;            // sizeof(TrainingSample)
50     uint32_t nb_samples;
51     uint32_t nb_games;
52 };
53 static_assert(sizeof(SampleFileHeader) == 24);
54

```

```

55 // The network self-play must use, published by the trainer in
56 // <work>/latest.json as {"step": 12000, "ts": "models/az_00012000.ts"}.
57 // A relative path is relative to the work directory.
58 struct ModelRef {
59     uint64_t    step = 0;
60     std::string path;
61 };
62 bool read_latest_model(const std::string& work_dir, ModelRef& out);
63
64 struct SelfPlayOptions {
65     json        player;                // AlphaZeroMCTSPlayer keys: search + backend (weights
66     ↪ ignored)
67     std::string work_dir;              // reads latest.json, writes selfplay/
68     std::string tag;                  // part of the file names (default host_pid)
69     size_t      nb_games              = 128; // games played concurrently
70     uint64_t    max_games             = 0;  // stop after this many games (0 = until stopped)
71     size_t      flush_samples         = 2000; // write a file every N samples...
72     double      flush_seconds         = 600; // ...or every N seconds, whichever comes first
73     double      poll_seconds          = 30;  // how often latest.json is checked
74     double      report_seconds        = 300; // how often the throughput line is printed
75     // Stop when latest.json has not changed for this long (0 = never). For
76     // self-play jobs running apart from the trainer (alphazero_selfplay.sh):
77     // when the trainer's job is over, no new network comes and they stop
78     // instead of burning GPU hours on an outdated one.
79     double      max_stale_hours       = 0;
80 };
81 // Plays until `stop` is set (or max_games). Unfinished games are dropped;
82 // finished ones are always written before returning.
83 void run_selfplay(const SelfPlayOptions& options, const std::atomic<bool>& stop);
84
85 } // namespace az
86
87 #endif

```

11.5.2 player/alphazero_selfplay.cpp

271 lines.

One thread per game, all sharing one **BatchingEvaluator**; the calling thread is the supervisor (hot swap, periodic flush, statistics). Games stopped mid-way are dropped; finished ones always reach the disk before `run_selfplay` returns.

symbol	role
SampleWriter	collects finished games; writes ~2000-row files under a temporary name and renames them
make_sample	a SearchResult → a TrainingSample (π quantised to 16 bits)
run_selfplay / game_loop	one game after another; z filled in at the end of each game

```

1 #include "alphazero_selfplay.h"
2 #include "alphazero_mcts_player.h"
3 #include "batched_evaluator.h"
4 #include <unistd.h>
5 #include <algorithm>
6 #include <chrono>

```

```

7  #include <cmath>
8  #include <cstring>
9  #include <filesystem>
10 #include <format>
11 #include <fstream>
12 #include <iostream>
13 #include <memory_resource>
14 #include <mutex>
15 #include <thread>
16 #include <vector>
17
18 namespace fs = std::filesystem;
19 using std::string;
20
21 namespace az {
22
23 bool read_latest_model(const string& work_dir, ModelRef& out) {
24     std::ifstream in(fs::path(work_dir) / "latest.json");
25     if (!in) return false;
26     try {
27         const json j = json::parse(in);
28         ModelRef r;
29         r.step = j.at("step").get<uint64_t>();
30         fs::path p = j.at("ts").get<string>();
31         if (p.is_relative()) p = fs::path(work_dir) / p;
32         r.path = p.string();
33         out = r;
34         return true;
35     } catch (const std::exception&) {
36         return false; // being rewritten (the trainer renames atomically, but be safe)
37     }
38 }
39
40 namespace {
41
42 double now_seconds() {
43     using namespace std::chrono;
44     return duration<double>(steady_clock::now().time_since_epoch()).count();
45 }
46
47 string default_tag() {
48     char host[256] = {0};
49     gethostname(host, sizeof(host) - 1);
50     return std::format("{}_{}", host, getpid());
51 }
52
53 // Collects finished games and writes them in files of ~flush_samples samples.
54 // Files are written under a temporary name and renamed, so the trainer never
55 // sees a partial file. Names start with the wall-clock time in milliseconds:
56 // sorting them by name is sorting them by age, across hosts and restarts.
57 class SampleWriter {
58 public:
59     SampleWriter(fs::path dir, string tag, size_t flush_samples, double flush_seconds)
60         : dir(std::move(dir)), tag(std::move(tag)), flush_samples(flush_samples),
61           flush_seconds(flush_seconds), last_flush(now_seconds()) {
62         fs::create_directories(this->dir);
63     }
64
65     void add_game(std::vector<TrainingSample>&& game) {
66         std::lock_guard lock(mutex);
67         pending.insert(pending.end(), game.begin(), game.end());
68         ++pending_games;
69         if (pending.size() >= flush_samples) write_locked();

```

```

70     }
71
72     void flush_if_old() {
73         std::lock_guard lock(mutex);
74         if (!pending.empty() && now_seconds() - last_flush >= flush_seconds) write_locked();
75     }
76
77     void flush() {
78         std::lock_guard lock(mutex);
79         if (!pending.empty()) write_locked();
80     }
81
82 private:
83     void write_locked() {
84         using namespace std::chrono;
85         const auto ms =
86             ↪ duration_cast<milliseconds>(system_clock::now().time_since_epoch()).count();
87         const string name = std::format("{:013d}_{:05d}.bin", ms, tag, seq++);
88         const fs::path tmp = dir / ( "." + name + ".tmp");
89         {
90             std::ofstream out(tmp, std::ios::binary);
91             SampleFileHeader h{};
92             std::memcpy(h.magic, "YOLAHSP1", 8);
93             h.version = 1;
94             h.sample_size = sizeof(TrainingSample);
95             h.nb_samples = static_cast<uint32_t>(pending.size());
96             h.nb_games = static_cast<uint32_t>(pending_games);
97             out.write(reinterpret_cast<const char*>(&h), sizeof(h));
98             out.write(reinterpret_cast<const char*>(pending.data()),
99                 static_cast<std::streamsize>(pending.size() * sizeof(TrainingSample)));
100             if (!out) {
101                 std::cerr << "selfplay: cannot write " << tmp << ", keeping the samples in
102                 ↪ memory\n";
103                 return;
104             }
105             fs::rename(tmp, dir / name);
106             pending.clear();
107             pending_games = 0;
108             last_flush = now_seconds();
109         }
110
111         const fs::path dir;
112         const string tag;
113         const size_t flush_samples;
114         const double flush_seconds;
115         std::mutex mutex;
116         std::vector<TrainingSample> pending;
117         size_t pending_games = 0;
118         size_t seq = 0;
119         double last_flush;
120     };
121
122 TrainingSample make_sample(const Yolah& y, const SearchResult& r, uint32_t model_step) {
123     TrainingSample s{};
124     s.black = y.bitboard(Yolah::BLACK);
125     s.white = y.bitboard(Yolah::WHITE);
126     s.empty = y.empty_bitboard();
127     s.turn = y.current_player();
128     s.root_q = r.root_value;
129     s.model_step = model_step;
130     const size_t n = std::min<size_t>(r.children.size(), Yolah::MAX_NB_MOVES);
131     s.nb_moves = static_cast<uint8_t>(n);

```

```

131     for (size_t i = 0; i < n; i++) {
132         s.action[i] = static_cast<uint16_t>(nn::action_index(r.children[i].move));
133         const float p = std::clamp(r.policy[i], 0.0f, 1.0f);
134         s.prob[i] = static_cast<uint16_t>(std::lround(p * 65535.0f));
135     }
136     return s;
137 }
138
139 } // namespace
140
141 void run_selfplay(const SelfPlayOptions& opt, const std::atomic<bool>& stop) {
142     ModelRef model;
143     if (!read_latest_model(opt.work_dir, model))
144         throw std::runtime_error("selfplay: no readable " + opt.work_dir + "/latest.json");
145
146     // One backend for all the games, behind the batching wrapper.
147     json cfg = opt.player;
148     cfg["weights"] = model.path;
149     SearchParams params = AlphaZeroMCTSPPlayer::search_params(cfg);
150     params.nb_threads = 1; // parallelism comes from the games
151     std::unique_ptr<nn::Evaluator> backend = nn::make_evaluator(cfg);
152     if (params.batch_size == 0) params.batch_size = 8;
153     const size_t nb_games = std::max<size_t>(1, opt.nb_games);
154     const size_t merged = cfg.value("merged batch size", nb_games * params.batch_size);
155     nn::BatchingEvaluator evaluator(std::move(backend), nb_games, merged);
156
157     SampleWriter writer(fs::path(opt.work_dir) / "selfplay",
158                        opt.tag.empty() ? default_tag() : opt.tag,
159                        opt.flush_samples, opt.flush_seconds);
160
161     std::atomic<uint64_t> generation{0}; // bumped at every weight swap
162     std::atomic<uint32_t> model_step{static_cast<uint32_t>(model.step)};
163     std::atomic<uint64_t> games_started{0}, games_done{0}, samples_done{0}, plies_done{0};
164     std::atomic<uint64_t> evaluations{0}, black_wins{0}, white_wins{0};
165     std::atomic<size_t> active{nb_games};
166     // The games stop on `halt`: set by the caller's `stop` or by staleness.
167     std::atomic<bool> halt{false};
168
169     std::cout << std::format("selfplay: {} concurrent games, merged batch {}, {} sims ({} fast,
170                               ↪ p={}), model step {}\n",
171                               "selfplay: network {}\n",
172                               nb_games, merged, params.nb_simulations, params.nb_simulations_fast,
173                               params.playout_cap_fast_prob, model.step, evaluator.info()) <<
174     ↪ std::flush;
175
176     auto game_loop = [&] {
177         std::pmr::synchronized_pool_resource memory;
178         {
179             Search search(evaluator, params, &memory);
180             uint64_t seen = generation.load();
181             while (!halt.load()) {
182                 if (opt.max_games && games_started.fetch_add(1) >= opt.max_games) break;
183                 Yolah y;
184                 std::vector<TrainingSample> samples;
185                 search.reset();
186                 while (!y.game_over() && !halt.load()) {
187                     // New weights: the graph's priors/values and the cached
188                     // evaluations belong to the old network.
189                     const uint64_t g = generation.load(std::memory_order_acquire);
190                     if (g != seen) {
191                         search.reset();
192                         search.clear_cache();
193                         seen = g;

```

```

192         }
193         const SearchResult r = search.search(y);
194         evaluations.fetch_add(r.nb_evaluations, std::memory_order_relaxed);
195         if (r.full_search && !r.children.empty())
196             samples.push_back(make_sample(y, r, model_step.load()));
197         y.play(r.best_move);
198     }
199     if (!y.game_over()) break; // stopped mid-game: drop it
200     // Outcome for the side to move at each recorded position.
201     const auto [bs, ws] = y.score();
202     const int black_diff = int(bs) - int(ws);
203     for (TrainingSample& s : samples) {
204         const int d = s.turn == Yolah::BLACK ? black_diff : -black_diff;
205         s.z = static_cast<int8_t>((d > 0) - (d < 0));
206     }
207     if (black_diff > 0) black_wins.fetch_add(1);
208     if (black_diff < 0) white_wins.fetch_add(1);
209     plies_done.fetch_add(y.nb_plies());
210     samples_done.fetch_add(samples.size());
211     games_done.fetch_add(1);
212     writer.add_game(std::move(samples));
213 }
214 }
215 // Fewer clients: the batcher must stop waiting for this one.
216 evaluator.set_nb_clients(active.fetch_sub(1) - 1);
217 };
218
219 std::vector<std::jthread> threads;
220 for (size_t i = 0; i < nb_games; i++) threads.emplace_back(game_loop);
221
222 // Supervisor: hot-swap the network, flush old samples, report.
223 const double start = now_seconds();
224 double last_poll = start, last_report = start;
225 uint64_t last_games = 0, last_samples = 0, last_evals = 0;
226 double last_model_change = start;
227 while (!halt.load() && active.load() > 0) {
228     std::this_thread::sleep_for(std::chrono::milliseconds(200));
229     const double t = now_seconds();
230     if (stop.load()) halt.store(true);
231     if (opt.max_stale_hours > 0 && t - last_model_change > opt.max_stale_hours * 3600) {
232         std::cout << std::format("selfplay: no new network for {:.2f} h, stopping\n",
233             ↪ opt.max_stale_hours) << std::flush;
234         halt.store(true);
235     }
236     if (t - last_poll >= opt.poll_seconds) {
237         last_poll = t;
238         ModelRef latest;
239         if (read_latest_model(opt.work_dir, latest) && latest.step != model.step) {
240             try {
241                 evaluator.reload(latest.path);
242                 model = latest;
243                 last_model_change = t;
244                 model_step.store(static_cast<uint32_t>(model.step));
245                 generation.fetch_add(1, std::memory_order_release);
246                 std::cout << std::format("selfplay: now playing with step {} ({})\n",
247                     ↪ model.step, model.path) << std::flush;
248             } catch (const std::exception& e) {
249                 // A half-written file or a transient FS error: retry at the next poll.
250                 std::cerr << "selfplay: reload of " << latest.path << " failed: " << e.what()
251                     << "\n";
252             }
253         }
254     }
255     writer.flush_if_old();

```

```

252     }
253     if (t - last_report >= opt.report_seconds) {
254         const uint64_t g = games_done.load(), s = samples_done.load(), e = evaluations.load();
255         const double dt = t - last_report;
256         std::cout << std::format("selfplay: {:.1f}h  games {} (+{:.2f}/s)  samples {}
257             ↳ (+{:.1f}/s) "
258                                     "evals/s {:.0f}  avg plies {:.1f}  black wins {:.1f}%  white
259             ↳ wins {:.1f}%\n",
260                                     (t - start) / 3600, g, (g - last_games) / dt, s, (s -
261                                     ↳ last_samples) / dt,
262                                     (e - last_evals) / dt, g ? double(plies_done.load()) / g :
263                                     ↳ 0.0,
264                                     g ? 100.0 * black_wins.load() / g : 0.0, g ? 100.0 *
265                                     ↳ white_wins.load() / g : 0.0)
266                                     << std::flush;
267         last_report = t;
268         last_games = g; last_samples = s; last_evals = e;
269     }
270 }
271 threads.clear(); // join (the game threads watch `halt` / max_games)
writer.flush();
std::cout << std::format("selfplay: done, {} games, {} samples\n", games_done.load(),
    ↳ samples_done.load()) << std::flush;
} // namespace az

```

11.5.3 test/alphazero_learn_main.cpp

208 lines.

The `alphazero_learn` binary. `selfplay` wraps `run_selfplay` with a signal handler; `match` plays two networks against each other with paired random openings (Section 6.5) and writes the full record as JSON.

symbol	role
<code>selfplay_main</code>	parse options, install SIGINT/SIGTERM, run
<code>match_main</code>	A vs B, colours alternated, a random opening
	per pair of games
<code>apply_set</code>	--set 'key=value': override a key of the
	JSON config

```

1 // alphazero_learn_main.cpp – the C++ side of the AlphaZero learning loop
2 // (driven by nnue/alphazero_learn.py; see player/README_ALPHAZERO_LEARN.md).
3 //
4 //   alphazero_learn selfplay --config ../config/alphazero_mcts_selfplay_player.cfg --work <dir>
5 //       [--games 128] [--max-games 0] [--flush-samples 2000] [--flush-seconds 600]
6 //       [--poll 30] [--report 300] [--max-stale-hours 0] [--tag name] [--set 'key=value' ...]
7 //   Plays self-play games with the network named in <dir>/latest.json (hot-
8 //   swapped when it changes) and writes training samples to <dir>/selfplay/.
9 //   Stops cleanly on SIGINT/SIGTERM (finished games are written).
10 //
11 //   alphazero_learn match --config ../config/alphazero_mcts_eval_player.cfg
12 //       --a new.ts --b old.ts [--games 10] [--time 2000000] [--opening-plies 4]
13 //       [--seed 1] [--out result.json] [--set 'key=value' ...]
14 //   Plays A against B, alternating colours, and reports A's result. Games
15 //   2k and 2k+1 start from the same random opening of --opening-plies plies

```

```

16 //      (A black in one, white in the other): with deterministic players and no
17 //      opening, the 5 games of each colour would be the same game 5 times.
18 //
19 // --set overrides any key of the player JSON config ('key=value', value parsed as JSON).
20 #include "alphazero_mcts_player.h"
21 #include "alphazero_selfplay.h"
22 #include "magic.h"
23 #include "zobrist.h"
24 #include <atomic>
25 #include <csignal>
26 #include <format>
27 #include <fstream>
28 #include <iostream>
29 #include <random>
30 #include <sstream>
31 #include <string>
32 #include <vector>
33
34 using std::string;
35
36 namespace {
37
38 std::atomic<bool> stop_requested{false};
39 void on_signal(int) { stop_requested.store(true); }
40
41 struct Args {
42     std::vector<string> argv;
43     size_t i = 0;
44     bool more() const { return i < argv.size(); }
45     string next(const string& opt) {
46         if (i >= argv.size()) { std::cerr << "missing value for " << opt << "\n"; std::exit(2); }
47         return argv[i++];
48     }
49 };
50
51 void apply_set(json& cfg, const string& kv) {
52     const size_t eq = kv.find('=');
53     if (eq == string::npos) { std::cerr << "--set expects key=value\n"; std::exit(2); }
54     cfg[kv.substr(0, eq)] = json::parse(kv.substr(eq + 1));
55 }
56
57 json load_config(const string& path) {
58     std::ifstream in(path);
59     if (!in) { std::cerr << "cannot open " << path << "\n"; std::exit(2); }
60     return json::parse(in);
61 }
62
63 int selfplay_main(Args& a) {
64     az::SelfPlayOptions opt;
65     string config;
66     std::vector<string> sets;
67     while (a.more()) {
68         const string o = a.next("");
69         if (o == "--config") config = a.next(o);
70         else if (o == "--work") opt.work_dir = a.next(o);
71         else if (o == "--games") opt.nb_games = std::stoul(a.next(o));
72         else if (o == "--max-games") opt.max_games = std::stoull(a.next(o));
73         else if (o == "--flush-samples") opt.flush_samples = std::stoul(a.next(o));
74         else if (o == "--flush-seconds") opt.flush_seconds = std::stod(a.next(o));
75         else if (o == "--poll") opt.poll_seconds = std::stod(a.next(o));
76         else if (o == "--report") opt.report_seconds = std::stod(a.next(o));
77         else if (o == "--max-stale-hours") opt.max_stale_hours = std::stod(a.next(o));
78         else if (o == "--tag") opt.tag = a.next(o);

```

```

79         else if (o == "--set") sets.push_back(a.next(o));
80         else { std::cerr << "unknown option " << o << "\n"; return 2; }
81     }
82     if (config.empty() || opt.work_dir.empty()) { std::cerr << "selfplay needs --config and
    ↪ --work\n"; return 2; }
83     opt.player = load_config(config);
84     for (const string& kv : sets) apply_set(opt.player, kv);
85     std::signal(SIGINT, on_signal);
86     std::signal(SIGTERM, on_signal);
87     az::run_selfplay(opt, stop_requested);
88     return 0;
89 }
90
91 string move_list(const std::vector<Move>& moves) {
92     std::ostringstream os;
93     for (size_t i = 0; i < moves.size(); i++) os << (i ? " " : "") << moves[i];
94     return os.str();
95 }
96
97 int match_main(Args& a) {
98     string config, model_a, model_b, out;
99     size_t games = 10, opening_plies = 4;
100    uint64_t time_us = 2'000'000, seed = 1;
101    std::vector<string> sets;
102    while (a.more()) {
103        const string o = a.next("");
104        if (o == "--config") config = a.next(o);
105        else if (o == "--a") model_a = a.next(o);
106        else if (o == "--b") model_b = a.next(o);
107        else if (o == "--games") games = std::stoul(a.next(o));
108        else if (o == "--time") time_us = std::stoull(a.next(o));
109        else if (o == "--opening-plies") opening_plies = std::stoul(a.next(o));
110        else if (o == "--seed") seed = std::stoull(a.next(o));
111        else if (o == "--out") out = a.next(o);
112        else if (o == "--set") sets.push_back(a.next(o));
113        else { std::cerr << "unknown option " << o << "\n"; return 2; }
114    }
115    if (config.empty() || model_a.empty() || model_b.empty()) {
116        std::cerr << "match needs --config, --a and --b\n";
117        return 2;
118    }
119    json cfg = load_config(config);
120    for (const string& kv : sets) apply_set(cfg, kv);
121    cfg["microseconds"] = time_us;
122    cfg["nb simulations"] = 0;
123    json cfg_a = cfg, cfg_b = cfg;
124    cfg_a["weights"] = model_a;
125    cfg_b["weights"] = model_b;
126    AlphaZeroMCTSPPlayer player_a(cfg_a), player_b(cfg_b);
127
128    std::mt19937_64 rng(seed);
129    int wins = 0, draws = 0, losses = 0;
130    json records = json::array();
131    std::vector<Move> opening;
132    for (size_t g = 0; g < games; g++) {
133        const bool a_black = (g % 2 == 0);
134        if (g % 2 == 0) {
135            // A fresh random opening for the pair (not a finished game).
136            for (;;) {
137                Yolah y;
138                opening.clear();
139                for (size_t p = 0; p < opening_plies && !y.game_over(); p++) {
140                    Yolah::MoveList moves;

```

```

141         y.moves(moves);
142         const Move m = moves[rng() % moves.size()];
143         opening.push_back(m);
144         y.play(m);
145     }
146     if (!y.game_over()) break;
147 }
148 }
149
150 Yolah y;
151 for (Move m : opening) y.play(m);
152 std::vector<Move> played;
153 while (!y.game_over()) {
154     const bool black_to_move = y.current_player() == Yolah::BLACK;
155     AlphaZeroMCTSPlayer& p = (black_to_move == a_black) ? player_a : player_b;
156     const Move m = p.play(y);
157     played.push_back(m);
158     y.play(m);
159 }
160 player_a.game_over(y);
161 player_b.game_over(y);
162 const auto [bs, ws] = y.score();
163 const int a_diff = (a_black ? 1 : -1) * (int(bs) - int(ws));
164 const char* result = a_diff > 0 ? "win" : (a_diff < 0 ? "loss" : "draw");
165 (a_diff > 0 ? wins : (a_diff < 0 ? losses : draws))++;
166 records.push_back({{"a_color", a_black ? "black" : "white"},
167                    {"opening", move_list(opening)},
168                    {"moves", move_list(played)},
169                    {"black_score", bs}, {"white_score", ws},
170                    {"plies", y.nb_plies()}, {"result_for_a", result}});
171 std::cout << std::format("game {}/{}: A {} score {}-{} ({} plies) -> A {} [A {}W {}D
172 ↪ {}L]\n",
173                          g + 1, games, a_black ? "black" : "white", bs, ws, y.nb_plies(),
174                          result, wins, draws, losses) << std::flush;
175 }
176 const double score = games ? (wins + 0.5 * draws) / games : 0.0;
177 json res = {{"a", model_a}, {"b", model_b}, {"games", games}, {"time_us", time_us},
178            {"opening_plies", opening_plies}, {"seed", seed},
179            {"wins", wins}, {"draws", draws}, {"losses", losses}, {"score", score},
180            {"records", records}};
181 if (!out.empty()) {
182     std::ofstream f(out);
183     f << res.dump(2) << "\n";
184 }
185 std::cout << std::format("match: A {}W {}D {}L, score {:.3f}\n", wins, draws, losses, score)
186 ↪ << std::flush;
187 return 0;
188 }
189 // namespace
190
191 int main(int argc, char* argv[]) {
192     magic::init();
193     zobrist::init();
194     if (argc < 2) {
195         std::cerr << "usage: alphazero_learn selfplay|match [options] (see
196 ↪ alphazero_learn_main.cpp)\n";
197         return 2;
198     }
199     Args a;
200     for (int i = 2; i < argc; i++) a.argv.emplace_back(argv[i]);
201     const string cmd = argv[1];
202     try {
203         if (cmd == "selfplay") return selfplay_main(a);

```



```

13         every --eval-hours (or --eval-every exports):
14         `alphazero_learn match` new vs old (10 games,
15         2 s per move, colours alternated) → evals.csv
16
17 Everything lives in the work directory and survives a job that is killed:
18
19 <work>/checkpoint.pt      trainer state (weights, EMA, optimizer, counters)
20 <work>/state.json         human-readable counters + evaluation state (Elo)
21 <work>/latest.json       {"step": s, "ts": "models/az_<s>.ts"} → self-play
22 <work>/models/           az_<step>.pt (plain state_dict, resnet_export.py
23                          can read it) and az_<step>.ts (TorchScript)
24 <work>/selfplay/*.bin     training samples (format: player/alphazero_selfplay.h)
25 <work>/evals.csv         one line per evaluation match
26 <work>/evals/*.json      full match records (every move of every game)
27 <work>/train_log.csv     losses, throughput
28 <work>/logs/            self-play / match output
29
30 Resubmitting the job with the same work directory continues where it stopped.
31
32 What comes from KataGo (github.com/lightvector/KataGo, python/train.py,
33 python/shuffle.py and the paper "Accelerating Self-Play Learning in Go"):
34 • playout cap randomization, forced playouts + policy target pruning and
35   graph search on the self-play side (C++, alphazero_mcts.cpp);
36 • only the full searches become training rows;
37 • the replay window grows sub-linearly with the data generated,
38   window =  $N_0 \cdot (1 + \beta \cdot ((N/N_0)^\alpha - 1)/\alpha)$ ,  $\alpha = 0.75$ ,  $\beta = 0.4$ 
39   (shuffle.py), so early, weak data ages out quickly;
40 • a fixed ratio of training samples per generated row (--reuse);
41 • the exported network is an average of recent weights (KataGo uses SWA
42   snapshots; here an exponential moving average, BatchNorm statistics
43   included);
44 • no gating: self-play always uses the newest network. The evaluation
45   matches only monitor progress.
46 • the 8 symmetries of the board as data augmentation.
47 Not ported: KataGo's auxiliary heads (score, ownership, opponent policy) –
48 they would change the architecture the C++ backends read.
49
50 Usage (from nnue/, inside the .sif or locally):
51
52 python3 alphazero_learn.py --work /work --init-model cnn_resnet_256x30_value_policy.pt \
53     --bin ../build/alphazero_learn
54 """
55 import argparse
56 import copy
57 import csv
58 import glob
59 import json
60 import math
61 import os
62 import signal
63 import socket
64 import subprocess
65 import sys
66 import threading
67 import time
68 from collections import deque
69
70 import numpy as np
71 import torch
72 import torch.nn.functional as F
73
74 HERE = os.path.dirname(os.path.abspath(__file__))
75 sys.path.append(os.path.join(HERE, "../server"))

```

```

76 sys.path.append(HERE)
77 from cnn_resnet_value_policy_chunked import Net # noqa: E402
78
79 # — sample format: MUST match az::TrainingSample (player/alphazero_selfplay.h) —
80 MAX_NB_MOVES = 75
81 SAMPLE_DTYPE = np.dtype([
82     ("black", "<u8"), ("white", "<u8"), ("empty", "<u8"),
83     ("turn", "u1"), ("z", "i1"), ("nb_moves", "u1"), ("flags", "u1"),
84     ("root_q", "<f4"), ("model_step", "<u4"),
85     ("action", "<u2", (MAX_NB_MOVES,)), ("prob", "<u2", (MAX_NB_MOVES,)),
86 ])
87 assert SAMPLE_DTYPE.itemsize == 336
88 HEADER_DTYPE = np.dtype([("magic", "S8"), ("version", "<u4"), ("sample_size", "<u4"),
89     ("nb_samples", "<u4"), ("nb_games", "<u4")])
90 assert HEADER_DTYPE.itemsize == 24
91
92
93 def log(msg):
94     print(f"[{time.strftime('%F %T')}] {msg}", flush=True)
95
96
97 def read_header(path):
98     h = np.fromfile(path, dtype=HEADER_DTYPE, count=1)
99     if len(h) != 1 or h[0]["magic"] != b"YOLAHSP1" or h[0]["sample_size"] !=
100         ↳ SAMPLE_DTYPE.itemsize:
101         raise ValueError(f"{path}: not a self-play sample file")
102     return int(h[0]["nb_samples"]), int(h[0]["nb_games"])
103
104
105 def read_samples(path):
106     n, _ = read_header(path)
107     return np.fromfile(path, dtype=SAMPLE_DTYPE, count=n, offset=HEADER_DTYPE.itemsize)
108
109
110 def atomic_write_text(path, text):
111     tmp = path + ".tmp"
112     with open(tmp, "w") as f:
113         f.write(text)
114     os.replace(tmp, path)
115
116 # — symmetries —————
117 def symmetry_tables():
118     """
119     The 8 symmetries of the board (same group as distill_common.symmetries).
120
121     Returns
122     src (8, 64): new plane cell j takes the value of old cell src[s, j]
123     act (8, 4096): policy index of the image of move index a under symmetry s
124     Plane cell c holds square 63 - c (see nn_evaluator.h), so a square maps
125     through cells: sq' = 63 - inv[63 - sq]. Index 0 (a1a1) is the pass and
126     stays the pass.
127     """
128     cells = torch.arange(64).view(8, 8)
129     srcs = []
130     for k in range(4):
131         r = torch.rot90(cells, k, dims=(0, 1))
132         srcs.append(r.flatten())
133         srcs.append(torch.flip(r, dims=(1,)).flatten())
134     src = torch.stack(srcs)
135     inv = torch.argsort(src, dim=1)
136     sq = 63 - inv[:, 63 - torch.arange(64)]
137     act = (sq[:, :, None] * 64 + sq[:, None, :]).reshape(8, 4096).clone()

```

```

138     act[:, 0] = 0
139     return src, act
140
141
142 # — replay window —————
143 class ReplayWindow:
144     """
145     Ring buffer of the most recent rows, filled from <work>/selfplay/*.bin in
146     file-name (= creation time) order. Sampling is uniform over the last
147     window_size(total) rows.
148     """
149     def __init__(self, directory, capacity, min_rows, alpha=0.75, beta=0.4):
150         self.dir = directory
151         self.capacity = capacity
152         self.min_rows = min_rows
153         self.alpha, self.beta = alpha, beta
154         self.buf = np.zeros(capacity, dtype=SAMPLE_DTYPE)
155         self.head = 0          # next write position
156         self.filled = 0
157         self.total = 0         # rows generated since the beginning (all files)
158         self.games = 0
159         self.known = set()
160
161     def window_size(self):
162         """KataGo's W(N) (shuffle.py), bounded by what the buffer holds."""
163         n0, n = self.min_rows, self.total
164         if n <= n0:
165             w = n
166         else:
167             w = n0 * (1 + self.beta * ((n / n0) ** self.alpha - 1) / self.alpha)
168         return int(min(w, self.filled, self.capacity))
169
170     def _append(self, rows):
171         """Copy rows in at the head, wrapping around (overwriting the oldest)."""
172         rows = rows[-self.capacity:]
173         n = len(rows)
174         first = min(n, self.capacity - self.head)
175         self.buf[self.head:self.head + first] = rows[:first]
176         if n > first:
177             self.buf[n - first:] = rows[first:]
178         self.head = (self.head + n) % self.capacity
179         self.filled = min(self.capacity, self.filled + n)
180
181     def scan(self):
182         """Pick up new files; returns the number of new rows."""
183         files = sorted(f for f in glob.glob(os.path.join(self.dir, "*.bin")))
184             if os.path.basename(f) not in self.known)
185         if not files:
186             return 0
187         headers = []
188         for f in files:
189             try:
190                 headers.append((f, *read_header(f)))
191             except (ValueError, OSError) as e:
192                 log(f"skipping {f}: {e}")
193                 self.known.add(os.path.basename(f))
194         # Only the rows that can still be in the buffer need to be read (on a
195         # restart that is the tail of a possibly huge history).
196         need, start = 0, len(headers)
197         while start > 0 and need < self.capacity:
198             start -= 1
199             need += headers[start][1]
200         new_rows = 0

```

```

201     for i, (f, n, g) in enumerate(headers):
202         self.known.add(os.path.basename(f))
203         self.total += n
204         self.games += g
205         new_rows += n
206         if i >= start:
207             self._append(read_samples(f))
208     return new_rows
209
210 def sample(self, batch_size, rng):
211     w = self.window_size()
212     back = rng.integers(1, w + 1, size=batch_size)
213     return self.buf[(self.head - back) % self.capacity]
214
215
216 # — batches —————
217 class BatchMaker:
218     def __init__(self, device):
219         self.device = device
220         src, act = symmetry_tables()
221         self.src = src.to(device)
222         self.act = act.to(device)
223         self.shifts = (63 - torch.arange(64, device=device)).view(1, 1, 64)
224
225     def __call__(self, rows, augment=True):
226         """
227         rows (numpy, SAMPLE_DTYPE) → (x, z, actions, probs) on the device:
228             x      (B, 4, 8, 8) float, channels-last – the network input
229             z      (B,)   float – game result for the side to move
230             actions (B, 75) long – policy indices of the legal moves (0-padded)
231             probs  (B, 75) float –  $\pi$  over those moves (0 on the padding)
232         The planes are decoded from the bitboards on the GPU (only 25 bytes
233         per row cross the bus instead of 1 KB of floats).
234         """
235         B = len(rows)
236         dev = self.device
237         # uint64 bitboards viewed as int64: `>>>` then sign-extends, but `& 1`
238         # keeps only the bit asked for, so the sign never matters.
239         bbs = np.stack([rows["black"], rows["white"], rows["empty"]], axis=1).view(np.int64)
240         bbs = torch.from_numpy(bbs).to(dev, non_blocking=True) # (B, 3)
241         # Cell i of a plane is bit 63 - i (see nn_evaluator.h): shifts = 63 - i.
242         planes = ((bbs[:, :, None] >> self.shifts) & 1).float() # (B, 3, 64)
243         turn = torch.from_numpy(rows["turn"].astype(np.float32)).to(dev)
244         planes = torch.cat([planes, turn.view(B, 1, 1).expand(B, 1, 64)], dim=1)
245         actions = torch.from_numpy(rows["action"].astype(np.int64)).to(dev) # (B, M)
246         #  $\pi$  was quantised to 16 bits per move: renormalise so it sums to 1.
247         probs = torch.from_numpy(rows["prob"].astype(np.float32)).to(dev)
248         probs = probs / probs.sum(1, keepdim=True).clamp_min(1.0)
249         z = torch.from_numpy(rows["z"].astype(np.float32)).to(dev)
250         if augment:
251             # One random symmetry per row: permute the 64 cells of every plane
252             # (one gather) and map every move index through the same symmetry
253             # (one table lookup). z is invariant.
254             sym = torch.randint(0, 8, (B,), device=dev)
255             planes = planes.gather(2, self.src[sym].unsqueeze(1).expand(B, 4, 64))
256             actions = self.act[sym].unsqueeze(1), actions]
257         x = planes.view(B, 4, 8, 8).contiguous(memory_format=torch.channels_last)
258         return x, z, actions, probs
259
260
261 # — model files —————
262 def load_state_dict(path):
263     """A state_dict from a plain .pt, a DDP one ("module." prefix) or our checkpoint.pt."""

```

```

264     with open(path, "rb") as f:
265         if f.read(100).startswith(b"version https://git-lfs"):
266             sys.exit(f"{path} is a Git LFS pointer, not a network: copy the real file "
267                     f"(the repository stores it with Git LFS)")
268     sd = torch.load(path, map_location="cpu")
269     if isinstance(sd, dict) and "model" in sd and "optimizer" in sd:
270         sd = sd["model"]
271     return {(k[7:] if k.startswith("module.") else k): v for k, v in sd.items()}
272
273
274 def make_net(sd):
275     """Build a Net whose sizes (channels, blocks, heads) are read off the state_dict."""
276     C = sd["input_conv.0.weight"].shape[0]
277     B = sum(1 for k in sd if k.endswith(".conv1.weight"))
278     net = Net(channels=C, nb_blocks=B, value_fc_size=sd["value_fc1.weight"].shape[0],
279             num_actions=sd["policy_fc.weight"].shape[0])
280     net.load_state_dict(sd)
281     return net
282
283
284 def model_paths(work, step):
285     """(<work>/models/az_<step>.pt, .ts) - step 0 is the initial network."""
286     base = os.path.join(work, "models", f"az_{step:08d}")
287     return base + ".pt", base + ".ts"
288
289
290 def export_model(net, work, step):
291     """Write az_<step>.pt (state_dict) + az_<step>.ts (TorchScript) atomically."""
292     pt, ts = model_paths(work, step)
293     cpu = copy.deepcopy(net).float().cpu().eval()
294     torch.save(cpu.state_dict(), pt + ".tmp")
295     os.replace(pt + ".tmp", pt)
296     with torch.no_grad():
297         traced = torch.jit.trace(cpu, torch.zeros(1, 4, 8, 8))
298     traced.save(ts + ".tmp")
299     os.replace(ts + ".tmp", ts)
300     return pt, ts
301
302
303 def publish_latest(work, step):
304     """Point self-play at az_<step>.ts (relative path: the work dir may be bind-mounted
305     ↪ elsewhere)."""
306     _, ts = model_paths(work, step)
307     atomic_write_text(os.path.join(work, "latest.json"),
308                     json.dumps({"step": step, "ts": os.path.relpath(ts, work)}) + "\n")
309
310 # — state shared with the evaluator thread —————
311 class State:
312     """Persistent counters + evaluation bookkeeping (state.json)."""
313     def __init__(self, work):
314         self.path = os.path.join(work, "state.json")
315         self.lock = threading.Lock()
316         self.d = {"step": 0, "samples_trained": 0, "exports": [0], "pending_evals": [],
317                 "evaluated": [0], "elo": {"0": 0.0}}
318         if os.path.exists(self.path):
319             with open(self.path) as f:
320                 self.d.update(json.load(f))
321
322     def save(self):
323         with self.lock:
324             atomic_write_text(self.path, json.dumps(self.d, indent=1) + "\n")
325

```

```

326
327 def elo_from_score(w, d, l):
328     """Elo difference from a match result, with a +1/2 prior so 10-0 stays finite."""
329     n = w + d + l
330     s = (w + 0.5 * d + 0.5) / (n + 1)
331     return 400.0 * math.log10(s / (1.0 - s))
332
333
334 # — subprocesses —————
335 class SelfPlayPool:
336     """One `alphazero_learn selfplay` per GPU, restarted if it dies."""
337     def __init__(self, args, gpus):
338         self.args = args
339         self.gpus = gpus
340         self.procs = {}
341         self.logs = {}
342         self.next_start = {}
343
344     def _start(self, gpu):
345         a = self.args
346         # Unique across jobs: two jobs on one node both see their GPU as "0".
347         job = os.environ.get("SLURM_JOB_ID", str(os.getpid()))
348         tag = f"{socket.gethostname()}_{job}_gpu{gpu}"
349         cmd = [a.bin, "selfplay", "--config", a.selfplay_config, "--work", a.work,
350              "--games", str(a.selfplay_games), "--tag", tag]
351         for kv in a.selfplay_set:
352             cmd += ["--set", kv]
353         env = dict(os.environ, CUDA_VISIBLE_DEVICES=gpu)
354         logf = open(os.path.join(a.work, "logs", f"selfplay_gpu{gpu}.log"), "a")
355         log(f"starting self-play on GPU {gpu}: {' '.join(cmd)}")
356         self.procs[gpu] = subprocess.Popen(cmd, env=env, stdout=logf, stderr=subprocess.STDOUT)
357         self.logs[gpu] = logf
358
359     def start(self):
360         for g in self.gpus:
361             self._start(g)
362
363     def check(self):
364         now = time.time()
365         for g, p in list(self.procs.items()):
366             if p.poll() is None:
367                 continue
368             if g not in self.next_start:
369                 log(f"self-play on GPU {g} exited with code {p.returncode}; restarting in 60 s "
370                    f"(see logs/selfplay_gpu{g}.log)")
371                 self.next_start[g] = now + 60
372             elif now >= self.next_start.pop(g):
373                 self.logs[g].close()
374                 self._start(g)
375
376     def stop(self):
377         for p in self.procs.values():
378             if p.poll() is None:
379                 p.send_signal(signal.SIGTERM)
380         for g, p in self.procs.items():
381             try:
382                 p.wait(timeout=120) # finishes its current searches and writes finished games
383             except subprocess.TimeoutExpired:
384                 p.kill()
385             self.logs[g].close()
386
387
388 class Evaluator(threading.Thread):

```

```

389     """Plays the queued evaluation matches one after the other."""
390     CSV_FIELDS = ["time", "model_step", "opponent_step", "games", "wins", "draws", "losses",
391                  "score", "elo_diff", "model_elo", "samples_trained", "rows_generated", "hours"]
392
393     def __init__(self, args, state, gpu, stop_event, progress):
394         super().__init__(daemon=True)
395         self.args, self.state, self.gpu = args, state, gpu
396         self.stop_event = stop_event
397         self.progress = progress          # callable → (samples_trained, rows_generated, hours)
398         self.proc = None
399         self.wake = threading.Event()
400
401     def opponents(self, step):
402         with self.state.lock:
403             evaluated = [s for s in self.state.d["evaluated"] if s < step]
404             res = []
405             for o in self.args.eval_opponents:
406                 if o == "previous" and evaluated:
407                     res.append(evaluated[-1])
408                 elif o == "initial":
409                     res.append(0)
410                 elif o.startswith("lag:") and evaluated:
411                     res.append(evaluated[max(0, len(evaluated) - int(o[4:]))])
412             return sorted(set(res), reverse=True)
413
414     def ensure_ts(self, step):
415         """The TorchScript file of `step`, re-traced from its .pt if it was pruned."""
416         pt, ts = model_paths(self.args.work, step)
417         if not os.path.exists(ts) and os.path.exists(pt):
418             export_model(make_net(load_state_dict(pt)), self.args.work, step)
419         return ts
420
421     def run_match(self, step, opp):
422         a = self.args
423         ts_a = self.ensure_ts(step)
424         ts_b = self.ensure_ts(opp)
425         out = os.path.join(a.work, "evals", f"az_{step:08d}_vs_az_{opp:08d}.json")
426         cmd = [a.bin, "match", "--config", a.eval_config, "--a", ts_a, "--b", ts_b,
427              "--games", str(a.eval_games), "--time", str(int(a.eval_seconds * 1e6)),
428              "--opening-plies", str(a.eval_opening_plies), "--seed", str(step + 1),
429              "--out", out]
430         env = dict(os.environ, CUDA_VISIBLE_DEVICES=self.gpu)
431         log(f"evaluation: az_{step} vs az_{opp} ({a.eval_games} games, {a.eval_seconds} s/move)")
432         with open(os.path.join(a.work, "logs", "eval.log"), "a") as logf:
433             self.proc = subprocess.Popen(cmd, env=env, stdout=logf, stderr=subprocess.STDOUT)
434             rc = self.proc.wait()
435             self.proc = None
436             if rc != 0 or not os.path.exists(out):
437                 return None
438             with open(out) as f:
439                 return json.load(f)
440
441     def evaluate(self, step):
442         results = []
443         for opp in self.opponents(step):
444             r = self.run_match(step, opp)
445             if self.stop_event.is_set():
446                 return False          # interrupted: stays pending
447             if r is None:
448                 log(f"evaluation az_{step} vs az_{opp} FAILED (see logs/eval.log)")
449                 continue
450             results.append((opp, r))
451         with self.state.lock:

```

```

452         elo = self.state.d["elo"]
453         for opp, r in results:
454             diff = elo_from_score(r["wins"], r["draws"], r["losses"])
455             if str(step) not in elo and str(opp) in elo:
456                 elo[str(step)] = elo[str(opp)] + diff # Elo chained through the first
↪ opponent
457         trained, rows, hours = self.progress()
458         row = {"time": time.strftime("%F %T"), "model_step": step, "opponent_step": opp,
459               "games": r["games"], "wins": r["wins"], "draws": r["draws"], "losses":
↪               r["losses"],
460               "score": f"{r['score']:.3f}", "elo_diff": f"{diff:+.0f}",
461               "model_elo": f"{elo.get(str(step), float('nan')):+.0f}",
462               "samples_trained": trained, "rows_generated": rows, "hours":
↪               f"{hours:.2f}"}
463         path = os.path.join(self.args.work, "evals.csv")
464         new = not os.path.exists(path)
465         with open(path, "a", newline="") as f:
466             w = csv.DictWriter(f, fieldnames=self.CSV_FIELDS)
467             if new:
468                 w.writeheader()
469                 w.writerow(row)
470             log(f"EVAL az_{step} vs az_{opp}: {r['wins']}W {r['draws']}D {r['losses']}L "
471               f"score {r['score']:.2f} Elo {diff:+.0f} (az_{step} ≈ {row['model_elo']} vs
↪               initial)")
472         if results:
473             self.state.d["evaluated"].append(step)
474             self.state.d["pending_evals"].remove(step)
475         self.state.save()
476         return True
477
478     def run(self):
479         while not self.stop_event.is_set():
480             with self.state.lock:
481                 pending = list(self.state.d["pending_evals"])
482             if not pending:
483                 self.wake.wait(30)
484                 self.wake.clear()
485                 continue
486             self.evaluate(pending[0])
487
488     def stop(self):
489         if self.proc is not None and self.proc.poll() is None:
490             self.proc.terminate()
491
492
493 # — trainer —————
494 def main():
495     """
496     Setup (arguments, signals, GPUs), resume or start (checkpoint, initial
497     export), start the self-play processes and the evaluator thread, then the
498     training loop until a signal or --max-hours, then a clean shutdown.
499     """
500     ap = argparse.ArgumentParser(description=__doc__,
↪     formatter_class=argparse.RawDescriptionHelpFormatter)
501     ap.add_argument("--work", required=True, help="work directory (everything is kept there)")
502     ap.add_argument("--init-model", default=os.path.join(HERE,
↪     "cnn_resnet_256x30_value_policy.pt"),
503                     help="starting network (used only when the work directory is new)")
504     ap.add_argument("--bin", default=os.path.join(HERE, "../build/alphazero_learn"))
505     ap.add_argument("--selfplay-config", default=os.path.join(HERE,
↪     "../config/alphazero_mcts_selfplay_player.cfg"))
506     ap.add_argument("--eval-config", default=os.path.join(HERE,
↪     "../config/alphazero_mcts_eval_player.cfg"))

```

```

507     ap.add_argument("--selfplay-games", type=int, default=128, help="concurrent games per
↪ self-play process")
508     ap.add_argument("--selfplay-set", action="append", default=[], help="'key=value' override for
↪ self-play")
509     ap.add_argument("--no-selfplay", action="store_true", help="train only (self-play runs
↪ elsewhere)")
510     # optimisation
511     ap.add_argument("--batch-size", type=int, default=1024)
512     ap.add_argument("--lr", type=float, default=1e-4)
513     ap.add_argument("--weight-decay", type=float, default=1e-4)
514     ap.add_argument("--warmup-steps", type=int, default=500)
515     ap.add_argument("--ema-decay", type=float, default=0.995, help="0 = export the raw weights")
516     ap.add_argument("--value-weight", type=float, default=1.0)
517     ap.add_argument("--policy-weight", type=float, default=1.0)
518     ap.add_argument("--reuse", type=float, default=4.0, help="training samples per generated row")
519     # data
520     ap.add_argument("--min-rows", type=int, default=100_000, help="rows before training starts
↪ (window N0)")
521     ap.add_argument("--max-window", type=int, default=4_000_000)
522     # cadence
523     ap.add_argument("--export-every", type=int, default=131_072, help="training samples between
↪ exports")
524     ap.add_argument("--eval-hours", type=float, default=4.0,
525                     help="evaluate the newest network when this long has passed since the last
↪     one")
526     ap.add_argument("--eval-every", type=int, default=0,
527                     help="evaluate every N exports instead (0 = use --eval-hours)")
528     ap.add_argument("--eval-games", type=int, default=20)
529     ap.add_argument("--eval-seconds", type=float, default=2.0)
530     ap.add_argument("--eval-opening-plies", type=int, default=4)
531     ap.add_argument("--eval-opponents", default="previous",
532                     help="comma separated: previous (last evaluated model), initial, lag:K")
533     ap.add_argument("--keep-all", action="store_true", help="keep every exported model")
534     ap.add_argument("--max-hours", type=float, default=0, help="stop cleanly after this long (0 =
↪ never)")
535     ap.add_argument("--train-gpu", type=int, default=0, help="index among the visible GPUs")
536     ap.add_argument("--eval-gpu", type=int, default=-1,
537                     help="index among the visible GPUs (-1 = the last one: training uses the
↪     first)")
538     ap.add_argument("--allow-cpu", action="store_true", help="run even without CUDA (tests only)")
539     ap.add_argument("--amp", choices=["auto", "bf16", "fp16"], default="auto",
540                     help="mixed precision of the trainer (auto: bf16 on Ampere+, fp16 before)")
541     args = ap.parse_args()
542     args.work = os.path.abspath(args.work)
543     args.eval_opponents = [o.strip() for o in args.eval_opponents.split(",") if o.strip()]
544     for d in ("models", "selfplay", "evals", "logs"):
545         os.makedirs(os.path.join(args.work, d), exist_ok=True)
546
547     t_start = time.time()
548     stop_event = threading.Event()
549
550     def on_signal(signum, _frame):
551         log(f"signal {signum}: stopping cleanly")
552         stop_event.set()
553     for s in (signal.SIGTERM, signal.SIGINT, signal.SIGUSR1):
554         signal.signal(s, on_signal)
555
556     cvd = os.environ.get("CUDA_VISIBLE_DEVICES", "").strip()
557     gpus = [g.strip() for g in cvd.split(",") if g.strip()] if cvd else \
558         [str(i) for i in range(max(1, torch.cuda.device_count()))]
559     # No GPU is almost always a container problem (driver not bound): the
560     # self-play backend would silently fall back to the CPU and run ~50x
561     # slower for two weeks. Refuse instead.

```

```

562     if not torch.cuda.is_available() and not args.allow_cpu:
563         sys.exit("alphazero_learn: CUDA is not available (use --allow-cpu to run anyway)")
564     device = torch.device(f"cuda:{args.train_gpu}" if torch.cuda.is_available() else "cpu")
565     log(f"trainer on {device}" + (f" ({torch.cuda.get_device_name(device)})" if device.type ==
↪ "cuda" else ""))
566     if device.type == "cuda":
567         torch.cuda.set_device(device)
568         torch.backends.cudnn.benchmark = True
569         torch.set_float32_matmul_precision("high")
570
571     # — model / optimizer / checkpoint —
572     state = State(args.work)
573     ckpt_path = os.path.join(args.work, "checkpoint.pt")
574     ckpt = torch.load(ckpt_path, map_location="cpu") if os.path.exists(ckpt_path) else None
575     if ckpt is None:
576         log(f"new run: initial network {args.init_model}")
577         net = make_net(load_state_dict(args.init_model))
578     else:
579         net = make_net(ckpt["model"])
580     net = net.to(device).to(memory_format=torch.channels_last)
581     ema = None
582     if args.ema_decay > 0:
583         ema = torch.optim.swa_utils.AveragedModel(
584             net, multi_avg_fn=torch.optim.swa_utils.get_ema_multi_avg_fn(args.ema_decay),
↪ use_buffers=True)
585     opt = torch.optim.Adam(net.parameters(), lr=args.lr, weight_decay=args.weight_decay)
586     # Mixed precision: bfloat16 on Ampere and newer (A40), float16 + gradient scaler
587     # on Turing (RTX 8000), whose bfloat16 is emulated and slow. The cluster has
588     # both, so consecutive jobs of one run may use either.
589     if args.amp == "auto":
590         cc_major = torch.cuda.get_device_capability(device)[0] if device.type == "cuda" else 0
591         amp_dtype = torch.bfloat16 if cc_major >= 8 else torch.float16
592     else:
593         amp_dtype = torch.bfloat16 if args.amp == "bfloat16" else torch.float16
594     log(f"mixed precision: {amp_dtype}")
595     scaler = torch.amp.GradScaler("cuda", enabled=(device.type == "cuda" and amp_dtype ==
↪ torch.float16))
596     if ckpt is not None:
597         if ema is not None and ckpt.get("ema") is not None:
598             ema.load_state_dict(ckpt["ema"])
599             opt.load_state_dict(ckpt["optimizer"])
600             # A bfloat16 job saves the state of a *disabled* scaler, which is empty
601             # and which an enabled scaler refuses to load: an float16 job resuming
602             # it (A40 then RTX 8000) starts with a fresh scale – harmless, the
603             # scale adapts within a few steps.
604             if scaler.is_enabled() and ckpt.get("scaler"):
605                 scaler.load_state_dict(ckpt["scaler"])
606             # The checkpoint is authoritative: state.json may have been saved by
607             # the evaluator after it, with counters the lost steps never kept.
608             state.d["step"] = ckpt["step"]
609             state.d["samples_trained"] = ckpt["samples_trained"]
610             log(f"resumed at step {state.d['step']}, {state.d['samples_trained']:,} samples trained")
611         else:
612             pt0, _ = model_paths(args.work, 0)
613             if not os.path.exists(pt0):
614                 export_model(net, args.work, 0)
615                 publish_latest(args.work, 0)
616                 state.save()
617             if not os.path.exists(os.path.join(args.work, "latest.json")):
618                 publish_latest(args.work, state.d["exports"][-1])
619
620     def save_checkpoint():
621         torch.save({"model": net.state_dict(), "ema": ema.state_dict() if ema is not None else
↪ None,

```

```

622         "optimizer": opt.state_dict(), "scaler": scaler.state_dict(),
623         "step": state.d["step"], "samples_trained": state.d["samples_trained"]},
624         ckpt_path + ".tmp")
625     os.replace(ckpt_path + ".tmp", ckpt_path)
626     state.save()
627
628     # — data —
629     window = ReplayWindow(os.path.join(args.work, "selfplay"), args.max_window, args.min_rows)
630     window.scan()
631     log(f"replay: {window.total:,} rows from {window.games:,} games on disk, {window.filled:,} in
↪ memory")
632     make_batch = BatchMaker(device)
633     rng = np.random.default_rng()
634
635     # — subprocesses —
636     pool = None
637     if not args.no_selfplay:
638         pool = SelfPlayPool(args, gpus)
639         pool.start()
640     evaluator = Evaluator(args, state, gpus[min(args.eval_gpu, len(gpus) - 1)] if args.eval_gpu >=
↪ 0 else gpus[-1],
641         stop_event,
642         lambda: (state.d["samples_trained"], window.total, (time.time() -
↪ t_start) / 3600))
643     evaluator.start()
644
645     def cleanup_models():
646         """
647         Disk budget (a 256x30 network is 141 MB per file, and a month makes
648         hundreds of exports): an export that was never evaluated disappears
649         at the next export; an evaluated one keeps its .pt (the history of
650         the run) but loses its .ts unless it may still be needed as an
651         opponent – Evaluator.ensure_ts re-traces it from the .pt if so.
652         """
653         if args.keep_all:
654             return
655         with state.lock:
656             evaluated = list(state.d["evaluated"])
657             live = {0, state.d["exports"][-1], *state.d["pending_evals"]}
658             keep_pt = live | set(evaluated)
659             keep_ts = live | set(evaluated[-1:])
660             exports = list(state.d["exports"])
661             for s in exports:
662                 pt, ts = model_paths(args.work, s)
663                 if s not in keep_pt and os.path.exists(pt):
664                     os.remove(pt)
665                 if s not in keep_ts and os.path.exists(ts):
666                     os.remove(ts)
667
668     log_path = os.path.join(args.work, "train_log.csv")
669     new_log = not os.path.exists(log_path)
670     log_file = open(log_path, "a", newline="")
671     log_csv = csv.writer(log_file)
672     if new_log:
673         log_csv.writerow(["time", "hours", "step", "samples_trained", "rows_generated", "games",
674             "window", "value_loss", "policy_loss", "policy_kl", "lr",
675             ↪ "samples_per_s"])
676
677     # — main loop —
678     net.train()
679     acc = {"v": 0.0, "p": 0.0, "kl": 0.0, "n": 0}
680     last_scan = last_report = time.time()
681     samples_at_report = state.d["samples_trained"]

```

```

681 since_export = state.d["samples_trained"] - state.d.get("samples_at_last_export", 0)
682 waiting_logged = False
683 while not stop_event.is_set():
684     if args.max_hours and time.time() - t_start > args.max_hours * 3600:
685         log("--max-hours reached")
686         break
687     now = time.time()
688     if now - last_scan > 20:
689         window.scan()
690         if pool is not None:
691             pool.check()
692         last_scan = now
693
694     # Keep the trainer in step with self-play: at most `reuse` training
695     # samples per generated row (samples_trained <= r·N), and nothing
696     # before N0 rows exist. Ahead of the data → sleep and rescan.
697     allowed = args.reuse * window.total
698     if window.total < args.min_rows or state.d["samples_trained"] + args.batch_size > allowed:
699         if not waiting_logged:
700             log(f"waiting for self-play data: {window.total:,} rows "
701                 f"(training starts at {args.min_rows:,} rows and trains at most
702                  ↪ {args.reuse:g} samples per row)")
703             waiting_logged = True
704             stop_event.wait(10)
705             last_scan = 0
706             continue
707         waiting_logged = False
708
709     # — one SGD step —
710     # Linear warm-up: Adam's moment estimates start at zero after a fresh
711     # start, so the first steps would otherwise be too large.
712     step = state.d["step"]
713     lr = args.lr * min(1.0, (step + 1) / max(1, args.warmup_steps))
714     for g in opt.param_groups:
715         g["lr"] = lr
716     x, z, actions, probs = make_batch(window.sample(args.batch_size, rng))
717     with torch.autocast(device.type, dtype=amp_dtype, enabled=device.type == "cuda"):
718         v, logits = net(x)
719     v = v.float()
720     # Policy: cross-entropy with the soft target  $\pi$ ,  $-\sum_a \pi(a) \log p(a|s)$ .
721     # The softmax runs over the 4096 logits;  $\pi$  lives on the legal moves
722     # only (sparse: `actions` are their indices, the padding has  $\pi = 0$ ).
723     logp = F.log_softmax(logits.float(), dim=1).gather(1, actions)
724     p_loss = -(probs * logp).sum(1).mean()
725     # Value: squared error against the game result  $z \in \{-1, 0, +1\}$ .
726     v_loss = F.mse_loss(v, z)
727     loss = args.value_weight * v_loss + args.policy_weight * p_loss
728     opt.zero_grad(set_to_none=True)
729     scaler.scale(loss).backward()
730     scaler.unscale_(opt)
731     torch.nn.utils.clip_grad_norm_(net.parameters(), 5.0)
732     scaler.step(opt)
733     scaler.update()
734     #  $\bar{\theta} \leftarrow \mu \bar{\theta} + (1-\mu) \theta$ : the averaged weights are the ones exported.
735     if ema is not None:
736         ema.update_parameters(net)
737     state.d["step"] = step + 1
738     state.d["samples_trained"] += args.batch_size
739     since_export += args.batch_size
740
741     # KL( $\pi||p$ ) = CE( $\pi$ ,  $p$ ) - H( $\pi$ ): the part of the policy loss the network
742     # can still reduce ( $\theta$  = it already predicts what the search finds).
743     with torch.no_grad():

```

```

743         entropy = -(probs * torch.log(probs.clamp_min(1e-12))).sum(1).mean()
744         acc["v"] += v_loss.item(); acc["p"] += p_loss.item(); acc["kl"] += (p_loss -
↪ entropy).item(); acc["n"] += 1
745
746         if time.time() - last_report > 120:
747             n = max(1, acc["n"])
748             dt = time.time() - last_report
749             sps = (state.d["samples_trained"] - samples_at_report) / dt
750             hours = (time.time() - t_start) / 3600
751             log(f"step {state.d['step']} trained {state.d['samples_trained']:,} rows
↪ {window.total:,} "
752                f"({window.games:,} games) window {window.window_size():,} "
753                f"v {acc['v']/n:.4f} p {acc['p']/n:.4f} kl {acc['kl']/n:.4f} lr {lr:.2e}
↪ {sps:.0f} samples/s")
754             log_csv.writerow([time.strftime("%F %T"), f"hours:.3f", state.d["step"],
↪ state.d["samples_trained"],
755                               window.total, window.games, window.window_size(),
↪ f"{acc['v']/n:.5f}",
756                               f"{acc['p']/n:.5f}", f"{acc['kl']/n:.5f}", f"{lr:.3e}",
↪ f"{sps:.0f}"])
757             log_file.flush()
758             acc = {"v": 0.0, "p": 0.0, "kl": 0.0, "n": 0}
759             last_report = time.time()
760             samples_at_report = state.d["samples_trained"]
761
762         # — export: a new network for self-play (and, every eval_every, a match) —
763         if since_export >= args.export_every:
764             since_export = 0
765             s = state.d["step"]
766             export_model(ema.module if ema is not None else net, args.work, s)
767             publish_latest(args.work, s)
768             with state.lock:
769                 state.d["exports"].append(s)
770                 state.d["samples_at_last_export"] = state.d["samples_trained"]
771                 # Evaluation schedule: every eval_every exports, or else the
772                 # first export at least eval_hours (wall clock, across jobs)
773                 # after the previously scheduled one.
774                 if args.eval_every > 0:
775                     due = (len(state.d["exports"]) - 1) % args.eval_every == 0
776                 else:
777                     due = time.time() - state.d.get("last_eval_time", 0) >= args.eval_hours * 3600
778                 if due and args.eval_games > 0:
779                     state.d["pending_evals"].append(s)
780                     state.d["last_eval_time"] = time.time()
781             save_checkpoint()
782             cleanup_models()
783             evaluator.wake.set()
784             log(f"exported az_{s:08d} (self-play switches to it)")
785
786         # — shutdown —
787         stop_event.set()
788         log("saving checkpoint")
789         save_checkpoint()
790         evaluator.stop()
791         if pool is not None:
792             pool.stop()
793         evaluator.join(timeout=60)
794         log_file.close()
795         log("stopped; resubmit with the same --work to continue")
796
797     if __name__ == "__main__":
798         main()
799

```

11.5.5 config/alphazero_mcts_selfplay_player.cfg

25 lines.

Self-play search settings (the `weights` key is replaced by `latest.json`).

```
1 {
2     "name": "AlphaZeroMCTSPlayer",
3     "microseconds": 0,
4     "nb simulations": 800,
5     "nb threads": 1,
6     "batch size": 8,
7     "backend": "torch",
8     "device": "cuda",
9     "fp16": true,
10    "weights": "../nnue/cnn_resnet_256x30_value_policy.ts",
11
12    "dirichlet alpha": 0.3,
13    "dirichlet epsilon": 0.25,
14    "temperature": 1.0,
15    "temperature cutoff": 20,
16    "forced playouts k": 2.0,
17    "plout cap fast prob": 0.75,
18    "nb simulations fast": 200,
19    "policy target pruning": true,
20
21    "graph search": true,
22    "reuse tree": true,
23    "nn cache": 16,
24    "verbose": false
25 }
```

11.5.6 config/alphazero_mcts_eval_player.cfg

20 lines.

Evaluation-match settings: competitive play, no noise, no sampling. The time per move comes from `alphazero_learn match --time`.

```
1 {
2     "name": "AlphaZeroMCTSPlayer",
3     "microseconds": 2000000,
4     "nb simulations": 0,
5     "nb threads": 4,
6     "batch size": 32,
7     "backend": "torch",
8     "device": "cuda",
9     "fp16": true,
10    "weights": "set by alphazero_learn match",
11    "graph search": true,
12    "use lcb": true,
13    "dirichlet epsilon": 0.0,
14    "temperature": 0.0,
15    "forced playouts k": 0.0,
16    "plout cap fast prob": 0.0,
17    "reuse tree": true,
18    "nn cache": 64,
19    "verbose": false
20 }
```

11.5.7 nnue/alphazero_learn.def

62 lines.

The cluster image: a toolchain only, the sources are bind-mounted (Section 6.6).

```
1 Bootstrap: docker
2 From: nvidia/cuda:12.4.1-cudnn-devel-ubuntu22.04
3
4 # — alphazero_learn.def — toolchain image for the AlphaZero learning loop —
5 #
6 # Build (needs root or --fakeroot, no GPU needed):
7 #     sudo singularity build alphazero_learn.sif alphazero_learn.def
8 #
9 # Unlike the other .def files, the Yolah sources are NOT baked into the image:
10 # the job bind-mounts the repository at /Yolah and compiles the C++ part
11 # (alphazero_learn) on the compute node at the start of every job (a couple of
12 # minutes, cached per CPU model in the work directory). Two reasons:
13 #   • the CMake flags use -march=native: a binary compiled on the machine that
14 #     builds the image may use instructions the cluster CPUs lack (SIGILL);
15 #   • a code change needs a `git pull` on the cluster, not an image rebuild.
16 #
17 # Contents: CUDA 12.4 + cuDNN (headers for libtorch's CMake config), g++-13
18 # (the code is C++23: <format>, std::jthread ...), Boost, oneTBB, Google
19 # benchmark, CMake, Python 3 with torch 2.6.0 — whose wheel also provides
20 # libtorch (headers + CMake package) for the C++ TorchScript backend.
21 #
22 # See alphazero_learn.sh for the SLURM glue and
23 # player/README_ALPHAZERO_LEARN.md for the whole loop.
24
25 %post
26     set -eu
27     export DEBIAN_FRONTEND=noninteractive
28     apt-get update -y
29     apt-get install -y --no-install-recommends software-properties-common gpg-agent
30     ↪ ca-certificates
31     # g++-13: Ubuntu 22.04 ships g++-11, which lacks <format>.
32     add-apt-repository -y ppa:ubuntu-toolchain-r/test
33     apt-get update -y
34     apt-get install -y --no-install-recommends \
35         g++-13 gcc-13 make cmake git \
36         libboost-program-options-dev libtbb-dev libbenchmark-dev \
37         python3 python3-dev python3-pip
38     update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-13 100
39     update-alternatives --install /usr/bin/g++ g++ /usr/bin/g++-13 100
40     rm -rf /var/lib/apt/lists/*
41
42     pip3 install --no-cache-dir --upgrade pip
43     # torch 2.6.0 bundles the CUDA 12.4 runtime, which the cluster driver
44     # supports (see cnn_resnet_value_policy.def). Keep the two pins together.
45     pip3 install --no-cache-dir torch==2.6.0 numpy tqdm
46
47     # Build-time sanity checks.
48     g++ --version | head -1
49     cmake --version | head -1
50     python3 -c "import torch; print('torch', torch.__version__, torch.utils.cmake_prefix_path)"
51
52     mkdir -p /Yolah /work
53
54 %environment
55     export CUDA_HOME=/usr/local/cuda
56     export PATH=$CUDA_HOME/bin:$PATH
57     export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH
```

```

57 export CC=gcc-13
58 export CXX=g++-13
59 # Only used to skip libtorch's CMake GPU probe (see CMakeLists.txt):
60 # Turing, Ampere, Ada, Hopper.
61 export TORCH_CUDA_ARCH_LIST="7.5;8.0;8.6;8.9;9.0"
62 export TORCH_ALLOW_TF32=1

```

11.5.8 nnue/alphazero_common.sh

85 lines.

Sourced by the two job scripts: the paths (/data), the container launcher, the build step (with its lock), and the signal forwarding that makes a clean stop possible.

symbol	role
run	run a command in the container, repository at /Yolah, run at /work
build_alphazero_learn	compile for this node's CPU model, serialised by flock
start_in_container	start in the background with exec, so that \$! is Singularity itself
wait_forwarding_signals	wait, forwarding SIGUSR1/SIGTERM as SIGTERM for a clean stop
max_hours_for_job	the job's remaining time minus a margin, from queue

```

1  # -----
2  # alphazero_common.sh – sourced by alphazero_learn.sh and alphazero_selfplay.sh
3  # (not a job by itself). Paths, the container launcher and the build step.
4  #
5  # SLURM runs a *copy* of the batch script from its spool directory, so the
6  # scripts source this file through YOLAH_DIR, never through a relative path.
7  # -----
8
9  # — Paths (override via env / `sbatch --export=ALL,VAR=...`) -----
10 # /data is the redundant storage visible from every node: the run lives there.
11 # Its I/O is light (~1.2 GB/day of self-play rows, a 141 MB network per export).
12 SIF="${SIF:-/data/${USER}/AlphaZeroLearn/alphazero_learn.sif}"
13 WORK_DIR="${WORK_DIR:-/data/${USER}/AlphaZeroLearn/work}"
14
15 # One build per CPU model: -march=native binaries are not portable across them.
16 CPU_TAG=$(grep -m1 'model name' /proc/cpuinfo | md5sum | cut -c1-8)
17 BUILD_DIR="/work/build/${CPU_TAG}"          # path inside the container
18
19 # WSL2 (local tests): the CUDA driver lives in /usr/lib/wsl/lib, which --nv
20 # does not bind. Harmless no-op on the cluster.
21 WSL_BIND=()
22 if [[ -d /usr/lib/wsl/lib ]]; then
23     WSL_BIND=(--bind /usr/lib/wsl:/usr/lib/wsl --env LD_LIBRARY_PATH=/usr/lib/wsl/lib)
24 fi
25
26 # Run a command in the container: the repository at /Yolah, the run at /work.
27 run() {
28     singularity exec --nv "${WSL_BIND[@]}" \
29         --bind "${YOLAH_DIR}:/Yolah" \
30         --bind "${WORK_DIR}:/work" \

```

```

31     "${SIF}" "$@"
32 }
33
34 # Compile alphazero_learn for this node's CPU (skipped by make when up to
35 # date). flock: two jobs starting together on identical nodes share the build
36 # directory and must not run CMake in it at the same time.
37 build_alphazero_learn() {
38     mkdir -p "${WORK_DIR}/build"
39     echo "[$(date '+%F %T')] === Building alphazero_learn in ${BUILD_DIR} ==="
40     run bash -c "set -e
41         exec 9> /work/build/.lock_${CPU_TAG}
42         flock 9
43         mkdir -p ${BUILD_DIR}
44         TORCH_PREFIX=$(python3 -c 'import torch; print(torch.utils.cmake_prefix_path)')
45         cmake -S /Yolah -B ${BUILD_DIR} -DCMAKE_BUILD_TYPE=Release -DENABLE_TORCH=ON \
46             -DCMAKE_PREFIX_PATH=${TORCH_PREFIX} -Wno-dev > /work/build/cmake_${CPU_TAG}.log
47         || { cat /work/build/cmake_${CPU_TAG}.log; exit 1; }
48         cmake --build ${BUILD_DIR} --target alphazero_learn -j ${SLURM_CPUS_PER_TASK:-16} 2>&1 |
49         tail -3"
50 }
51
52 # Start a command in the container in the background, as a direct child:
53 # `exec` makes $! the singularity process itself, which forwards SIGTERM into
54 # the container (a bash subshell in between would swallow the signal and
55 # orphan the processes inside). Sets PID.
56 start_in_container() {
57     ( exec singularity exec --nv "${WSL_BIND[@]}" \
58         --bind "${YOLAH_DIR}:/Yolah" --bind "${WORK_DIR}:/work" "${SIF}" "$@" ) &
59     PID=$!
60 }
61
62 # Wait for PID, forwarding SIGUSR1 (SLURM's --signal) and SIGTERM (scancel)
63 # as SIGTERM: the program inside stops cleanly. Sets RC to its exit code.
64 wait_forwarding_signals() {
65     trap 'echo "[$(date '+%F %T')] signal received, stopping cleanly"; kill -TERM ${PID}
66         2>/dev/null || true' USR1 TERM
67     RC=0
68     wait ${PID} || RC=$? # returns early when the trap fires...
69     if kill -0 ${PID} 2>/dev/null; then # ...then wait for the clean shutdown
70         RC=0; wait ${PID} || RC=$?
71     fi
72 }
73
74 # Hours left in this SLURM job minus a margin (0 outside SLURM), so that the
75 # program stops by itself even if the signal is lost.
76 max_hours_for_job() {
77     local margin_s="$1" left secs
78     [[ -z "${SLURM_JOB_ID:-}" ]] && { echo 0; return; }
79     left=$(squeue -h -j "${SLURM_JOB_ID}" -o %L 2>/dev/null || echo "")
80     # formats: [D-]HH:MM:SS, MM:SS
81     if [[ "${left}" =~ ^([0-9]+)?([0-9]+):([0-9]+):([0-9]+)$ ]]; then
82         secs=$(( ${BASH_REMATCH[2]}:-0)*86400 + 10#${BASH_REMATCH[3]}*3600 +
83             10#${BASH_REMATCH[4]}*60 + 10#${BASH_REMATCH[5]} )
84         awk -v s="${secs}" -v m="${margin_s}" 'BEGIN { h = (s - m) / 3600; if (h < 0.1) h = 0.1;
85             printf "%.3f", h }'
86     else
87         echo 0
88     fi
89 }

```

11.5.9 nnue/alphazero_learn.sh

122 lines.

The main SLURM job: 3 GPUs, 32 CPUs, 128 GB, 14 days (sizing: Section 6.7). It compiles for the node, runs the trainer, and stops cleanly on the time-limit signal.

```
1  #!/bin/bash
2  # -----
3  # alphazero_learn.sh – AlphaZero/KataGo-style self-play learning on the cluster.
4  #
5  #     sbatch alphazero_learn.sh           # first job AND every following one
6  #
7  # The run lives in WORK_DIR (on /data). A job that ends (time limit,
8  # preemption, scancel) checkpoints; submitting the same script again continues
9  # it: 2 weeks + 2 weeks = one month of learning. Nothing is lost but the games
10 # in progress.
11 #
12 # Per job:
13 #   1. compile alphazero_learn (C++ self-play + matches) for this node's CPU,
14 #      inside the container, into WORK_DIR/build/<cpu> (skipped when cached);
15 #   2. run nnue/alphazero_learn.py, which starts one self-play process per GPU
16 #      of this job, trains on the first GPU, exports a new network every
17 #      EXPORT_EVERY samples and, every EVAL_HOURS, plays EVAL_GAMES games
18 #      (EVAL_SECONDS per move, colours alternated) on the last GPU against the
19 #      previously evaluated network.
20 #
21 # GPUs: 3 per job (the per-user maximum). The GPU nodes have 2 or 3 cards
22 # (A40 or RTX 8000, 48 GB); if a 3-GPU node is long to get, run this job with
23 # fewer GPUs and add the missing ones as alphazero_selfplay.sh jobs, which
24 # can land on any GPU node: everything goes through WORK_DIR on /data.
25 #     sbatch --gres=gpu:1 alphazero_learn.sh
26 #     sbatch alphazero_selfplay.sh ; sbatch alphazero_selfplay.sh
27 # To pin a GPU model, use the cluster's gres type, e.g. --gres=gpu:a40:3
28 # (`sinfo -o "%N %G"` lists them).
29 #
30 # Sizing (see "Sizing for the cluster" in doc/alphazero_mcts.pdf): self-play
31 # is the bottleneck; on an RTX 3080 self-play uses ~2.6 cores, and CPU work
32 # grows with the evaluation rate, so 3 GPUs twice as fast need ~16 cores (+
33 # trainer and matches): 32 CPUs. RAM holds a 16 M-row replay window (5.4 GB)
34 # and 768 network caches of 32 MB (25 GB): 128 GB.
35 #
36 # Follow the progress: WORK_DIR/evals.csv (Elo), WORK_DIR/train_log.csv,
37 #                       alphazero_learn_<jobid>.out, WORK_DIR/logs/.
38 #
39 # SLURM sends SIGUSR1 30 min before the time limit (--signal); it is forwarded
40 # to the trainer, which saves everything and stops the self-play cleanly.
41 # MAX_HOURS is a second safety net computed from the job's time limit.
42 # -----
43
44 #SBATCH --job-name=alphazero_learn
45 #SBATCH --output=alphazero_learn_%j.out
46 #SBATCH --error=alphazero_learn_%j.out
47 #SBATCH --partition=insa-gpu
48 #SBATCH -x crn23
49 #SBATCH --gres=gpu:3
50 #SBATCH --cpus-per-task=32
51 #SBATCH --mem=128G
52 #SBATCH --time=14-00:00:00
53 #SBATCH --signal=B:USR1@1800
54
55 set -euo pipefail
56
```

```

57 YOLAH_DIR="${YOLAH_DIR:-${HOME}/Yolah}" # git checkout
58 source "${YOLAH_DIR}/nnue/alphazero_common.sh" # SIF, WORK_DIR, run(), ...
59
60 # — Tunables (override via env / `sbatch --export=ALL,VAR=...`) —————
61 # Initial network (used only when WORK_DIR is new), path on the host.
62 INIT_MODEL="${INIT_MODEL:-${YOLAH_DIR}/nnue/cnn_resnet_256x30_value_policy.pt}"
63
64 SELFPLAY_GAMES="${SELFPLAY_GAMES:-256}" # concurrent games per GPU (128 saturate an RTX
↪ 3080)
65 SELFPLAY_NN_CACHE="${SELFPLAY_NN_CACHE:-32}" # MB of network cache per game
66 BATCH_SIZE="${BATCH_SIZE:-1024}"
67 LR="${LR:-1e-4}"
68 REUSE="${REUSE:-4}" # training samples per self-play row
69 MIN_ROWS="${MIN_ROWS:-250000}" # N0: rows before training starts (KataGo: 250k)
70 MAX_WINDOW="${MAX_WINDOW:-16000000}" # replay buffer capacity (336 B/row)
71 EXPORT_EVERY="${EXPORT_EVERY:-524288}" # training samples between networks
72 EVAL_HOURS="${EVAL_HOURS:-4}" # time between evaluations
73 EVAL_GAMES="${EVAL_GAMES:-20}"
74 EVAL_SECONDS="${EVAL_SECONDS:-2}"
75 EVAL_OPPONENTS="${EVAL_OPPONENTS:-previous}" # previous, initial, lag:K (comma separated)
76 EXTRA_ARGS="${EXTRA_ARGS:-}" # anything else for alphazero_learn.py
77 RESUBMIT="${RESUBMIT:-0}" # 1 = sbatch this script again when the time is up
78
79 mkdir -p "${WORK_DIR}"
80 # The network is stored with Git LFS: a clone made without git-lfs (as on the
81 # cluster) holds a 134-byte pointer instead. Copy the real file by hand:
82 # scp nnue/cnn_resnet_256x30_value_policy.pt cluster:Yolah/nnue/
83 if [[ ! -f "${WORK_DIR}/init_model.pt" ]] && head -c 100 "${INIT_MODEL}" 2>/dev/null | grep -q
↪ "git-lfs"; then
84     echo "ERROR: ${INIT_MODEL} is a Git LFS pointer, not the network."
85     echo "Copy the real file: scp nnue/cnn_resnet_256x30_value_policy.pt
↪ <cluster>:${INIT_MODEL}"
86     exit 1
87 fi
88 cp -n "${INIT_MODEL}" "${WORK_DIR}/init_model.pt" 2>/dev/null || true
89 MAX_HOURS=$(max_hours_for_job 1200) # stop 20 min before the limit
90
91 echo "===== "
92 echo " Job : ${SLURM_JOB_ID:-(local)} on $(hostname), GPUs ${CUDA_VISIBLE_DEVICES:-?}"
93 echo " Yolah : ${YOLAH_DIR} ($(git -C "${YOLAH_DIR}" rev-parse --short HEAD 2>/dev/null ||
↪ echo '?'))"
94 echo " SIF : ${SIF}"
95 echo " Work dir : ${WORK_DIR}"
96 echo " Max hours : ${MAX_HOURS}"
97 echo "===== "
98 nvidia-smi --query-gpu=index,name,memory.total --format=csv,noheader 2>/dev/null || true
99
100 # — 1. compile the C++ part for this node —————
101 build_alphazero_learn
102
103 # — 2. learn —————
104 echo "[$(date '+%F %T')] === Learning ==="
105 start_in_container bash -c "cd /Yolah/nnue && exec python3 -u alphazero_learn.py \
106 --work /work --init-model /work/init_model.pt --bin ${BUILD_DIR}/alphazero_learn \
107 --selfplay-config /Yolah/config/alphazero_mcts_selfplay_player.cfg \
108 --eval-config /Yolah/config/alphazero_mcts_eval_player.cfg \
109 --selfplay-games ${SELFPLAY_GAMES} --batch-size ${BATCH_SIZE} --lr ${LR} --reuse ${REUSE} \
110 --selfplay-set 'nn cache=${SELFPLAY_NN_CACHE}' --max-window ${MAX_WINDOW} \
111 --min-rows ${MIN_ROWS} --export-every ${EXPORT_EVERY} --eval-hours ${EVAL_HOURS} \
112 --eval-games ${EVAL_GAMES} --eval-seconds ${EVAL_SECONDS} --eval-opponents ${EVAL_OPPONENTS} \
113 --max-hours ${MAX_HOURS} ${EXTRA_ARGS}"
114 wait_forwarding_signals
115

```

```

116 echo "[$(date '+%F %T')] === Stopped (exit code ${RC}) ==="
117 tail -n 5 "${WORK_DIR}/evals.csv" 2>/dev/null || true
118 # Resubmit only after a clean stop (not after a crash: no resubmission loop).
119 if [[ "${RESUBMIT}" == "1" && "${RC}" == "0" && -n "${SLURM_JOB_ID:-}" ]]; then
120     echo "resubmitting ${SLURM_SUBMIT_DIR}/alphazero_learn.sh"
121     cd "${SLURM_SUBMIT_DIR}" && sbatch --export=ALL alphazero_learn.sh
122 fi

```

11.5.10 nnue/alphazero_selfplay.sh

61 lines.

An extra self-play GPU on any node, for a run split over several jobs (Figure 16).

```

1  #!/bin/bash
2  # -----
3  # alphazero_selfplay.sh – an extra self-play GPU for a running alphazero_learn
4  # job, on any GPU node.
5  #
6  #     sbatch alphazero_selfplay.sh           # same WORK_DIR as alphazero_learn.sh
7  #
8  # The learning loop only communicates through files in WORK_DIR (on /data,
9  # visible from every node): this job reads latest.json, hot-swaps each new
10 # network, and writes its games to WORK_DIR/selfplay/, where the trainer picks
11 # them up like those of its own GPUs. Use it when a node with 3 free GPUs is
12 # long to get: `sbatch --gres=gpu:1 alphazero_learn.sh` plus two of these
13 # (3 GPUs in total, the per-user maximum).
14 #
15 # It waits for the main job to have published a network, and stops by itself
16 # when no new network has appeared for STALE_HOURS (the main job is over), at
17 # its own time limit, or on scancel – finished games are always written.
18 # -----
19
20 #SBATCH --job-name=alphazero_selfplay
21 #SBATCH --output=alphazero_selfplay_%j.out
22 #SBATCH --error=alphazero_selfplay_%j.out
23 #SBATCH --partition=insa-gpu
24 #SBATCH -x crn23
25 #SBATCH --gres=gpu:1
26 #SBATCH --cpus-per-task=12
27 #SBATCH --mem=48G
28 #SBATCH --time=14-00:00:00
29 #SBATCH --signal=B:USRI@600
30
31 set -euo pipefail
32
33 YOLAH_DIR="${YOLAH_DIR:-${HOME}/Yolah}"
34 source "${YOLAH_DIR}/nnue/alphazero_common.sh"           # SIF, WORK_DIR, run(), ...
35
36 SELFPLAY_GAMES="${SELFPLAY_GAMES:-256}"                # concurrent games on this GPU
37 SELFPLAY_NN_CACHE="${SELFPLAY_NN_CACHE:-32}"           # MB of network cache per game
38 STALE_HOURS="${STALE_HOURS:-6}"                         # stop after this long without a new network
39
40 echo "=====
41 echo "   Job       : ${SLURM_JOB_ID:-(local)} on $(hostname), GPU ${CUDA_VISIBLE_DEVICES:-?}"
42 echo "   Work dir   : ${WORK_DIR}"
43 echo "=====
44 nvidia-smi --query-gpu=index,name,memory.total --format=csv,noheader 2>/dev/null || true
45
46 # The main job creates latest.json when it starts a new run.
47 until [[ -f "${WORK_DIR}/latest.json" ]]; do

```

```

48     echo "[$(date '+%F %T')] waiting for ${WORK_DIR}/latest.json (is alphazero_learn.sh running?)"
49     sleep 60
50 done
51
52 build_alphazero_learn
53
54 echo "[$(date '+%F %T')] === Self-play ==="
55 mkdir -p "${WORK_DIR}/logs"
56 start_in_container ${BUILD_DIR}/alphazero_learn selfplay \
57     --config /Yolah/config/alphazero_mcts_selfplay_player.cfg --work /work \
58     --games "${SELFPLAY_GAMES}" --set "nn cache=${SELFPLAY_NN_CACHE}" \
59     --max-stale-hours "${STALE_HOURS}" --tag "${hostname}_${SLURM_JOB_ID:-$_}$extra"
60 wait_forwarding_signals
61 echo "[$(date '+%F %T')] === Stopped (exit code ${RC}) ==="

```